ZYNQ 开发平台 Linux 应用教程

2020/9/29 10:07:39

ALINX

版权声明

Copyright ©2012-2020 芯驿电子科技（上海）有限公司

公司网址:
Http://www.alinx.com.cn

技术论坛:
http://www.heijin.org

官方旗舰店:
http://alinx.jd.com

邮箱:
avic@alinx.com.cn

电话:
021-67676997

传真:
021-37737073

ALINX 微信公众号:



我们承诺本教程并非一劳永逸，固守不变的文档。我们会根据论坛上大家的反馈意见，以及实际的开发实践经验积累不断的修正和优化教程

文档修订记录:

版本	时间	描述
1.01	2018/7/16	初始版本
1.02	2018/11/29	添加 QT 运行注意事项
1.03	2018/12/25	交代一点琐事
1.04		
1.05	2020/8/11	修改 AN926 为 AN9238
1.06	2020/9/29	添加 AX7Z020 硬件连接图

前言

首先感谢大家购买芯驿电子科技（上海）有限公司出品的 ZYNQ 的开发板！ 您对我们和我们产品的支持和信任,给我们增添了永往直前的信心和勇气。

“播下一粒种子，收获一片森林”，更是芯驿电子科技（上海）有限公司的美好愿望，同时我们会在黑金动力社区 *http://www.heijin.org* 和大家一起讨论，一起学习，一起进步，一起成长。

通过前面的教程我们熟悉了 ZYNQ 开发的基本流程，学习了 SDK 下 ZYNQ 程序的开发，但是很多应用可能需要使用嵌入式 Linux，由于 Linux 系统庞大，芯驿电子科技（上海）有限公司只能抛砖引玉，把最常用的知识点收集分享给大家。

目录

版权声明 2
前言 4
目录 5
准备工作及注意事项 7
第一章 QT 在 ZYNQ 上的应用 8
1.1 QT Creator 安装 8
1.2 QT 库交叉编译 16
1.3 配置 ZYNQ 版 QT Kits 19
1.4 创建 QT 测试工程 24
1.5 运行 ZYNQ 版本 QT 程序 31
1.6 常见问题 33
1.6.1 如何使用开发板 Debian 系统自带的 QT Creator 33
1.6.2 系统运行打印 I2C 错误信息 35
第二章 OpenCV 应用（USB 摄像头显示） 37
2.1 host 版 OpenCV 库编译 37
2.2 zynq 版 OpenCV 库编译 38
2.3 测试 host 版 OpenCV 39
2.4 测试 zynq 版 OpenCV 44
第三章 双目摄像头模块 OpenCV 显示 49
3.1 vivado 工程简介 49
3.2 摄像头驱动 50
3.3 设备树配置 51
3.4 摄像头 Linux 应用程序 53
第四章 触摸屏模块应用 58
4.1 vivado 工程简介 58
4.2 触摸屏驱动 59
4.3 内核驱动配置 59
4.4 设备树配置 61
4.5 触摸屏测试 62
第五章 AXI DMA 读写测试 65
5.1 vivado 工程 65
5.2 petalinux 工程配置 66
5.3 运行测试 67
第六章 基于 DMA 的 ADC 波形显示（AN9238） 68

6.1 ADC 驱动 68
6.2 petalinux 工程配置 68
6.3 设备树配置 69
6.4 应用程序 70
6.5 运行结果 70
6.6 常见问题 73
6.6.1 内核崩溃 73
第七章 基于 DMA 的 ADC 波形显示（AN706） 75
7.1 ADC 驱动 75
7.2 petalinux 工程配置 75
7.3 设备树配置 76
7.4 应用程序 77
7.5 运行结果 77
第八章 自动运行的 petalinux 应用程序 80
8.1 准备工作 80
8.2 startup 应用程序 80
8.3 使用 startup 82
第九章 附录： Linux 常用命令 84
9.1 文件和文件夹操作 84
9.1.1 创建一个文件夹 84
9.1.2 创建多层目录的文件夹 84
9.1.3 改变当前目录 84
9.1.4 复制文件 84
9.1.5 文件重命名 84
9.1.6 删除文件和文件夹 84
9.2 文件列表 85
9.3 压缩解压 85
9.3.1 创建一个压缩包 85
9.3.2 解压一个 tar.gz 文件 85
9.4 系统管理 85

准备工作及注意事项

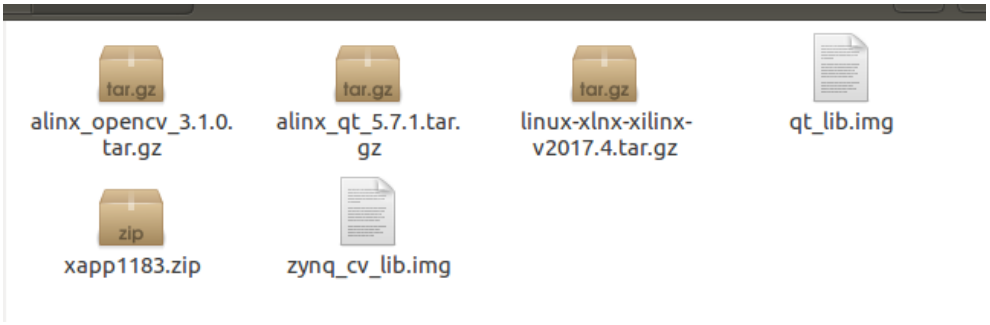
例程需要使用 petalinux 2017.4 环境，请参考基础教程，安装好 petalinux，按照前面的基础教程学习好 petalinux 工具的使用。如果使用其他版本的 petalinux，可能无法正常工作。

例程需要熟练掌握裸机实验例程，请先学习 SDK 裸机例程。

例程需要非常好的软件开发基础，请打好基础，量力而行。

例程所有源文件位于名为 course_s4_linux 压缩包中。

例程中可能用到的源码包、安装软件在 zynq_soft_2017/00_resource(需单独下载)目录下



例程目录一般结构：app 表示 Linux 下的应用软件；petalinux 为 petalinux 工程目录，Linux 内核配置、设备树、根文件系统配置都在这里；sd_boot 目录为编译好的可运行程序，复制所有文件到 sd 卡，可快速复现实验；vivado 目录是例程需要的 vivado 工程。

例程中 petalinux 工程使用的 Linux 内核使用本地源，如果直接使用例程 petalinux 工程，请先使用 petalinux-config 命令配置 Linux 内核源码路径。如何使用复制过来的 petalinux 工程，在前面的基础教程里已经讲过，不会的要先回去复习一下。

再次申明，本实验内容要求熟练掌握前面所学内容，对 xSDK 裸机开发、petalinux 使用熟练掌握才能学习，免得操作上遇到低级错误。

第一章 QT 在 ZYNQ 上的应用

例程相关文件夹： use_qt

QT 在嵌入式开发中非常实用，大部分嵌入式 GUI 都是使用 QT 开发，本章讲解如何安装 QT Creator，如何交叉编译 QT 库，如何交叉编译 QT 应用程序。

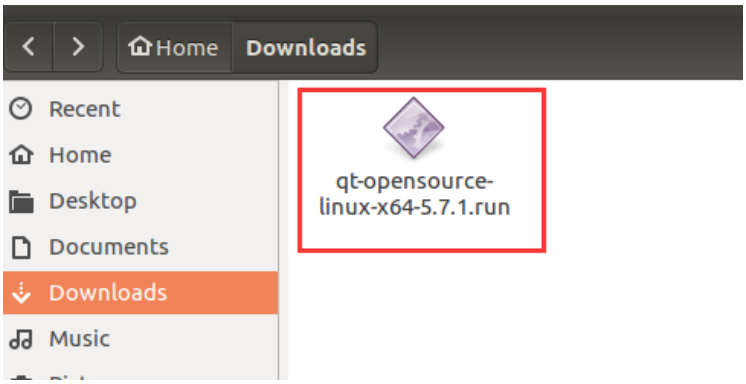
从本章开始需要开发者具备良好的 C、C++编程功底，本章只涉及到 QT 环境搭建问题，如何使用 QT 开发程序，大家还是要自己先学习，所以建议不具备这些基础的开发者先找其他资料打好相关基础再学习。

需要注意：这里的 QT 只能运行与非桌面环境，例如前面的 Debian 8，不能运行这里的 QT。

1.1 QT Creator 安装

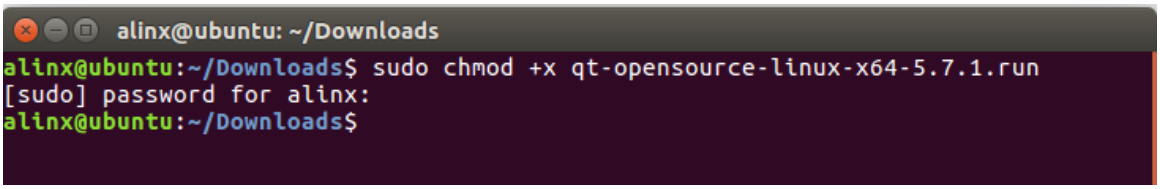
我们在 Ubuntu 中安装 QT Creator，安装文件名称为“qt-opensource-linux-x64-5.7.1.run”，从文件名我们可以看出 QT 版本为 5.7.1，这些版本也是我们精心挑选，适合当前环境的版本。

1) 将安装文件复制到 Ubuntu 主机中



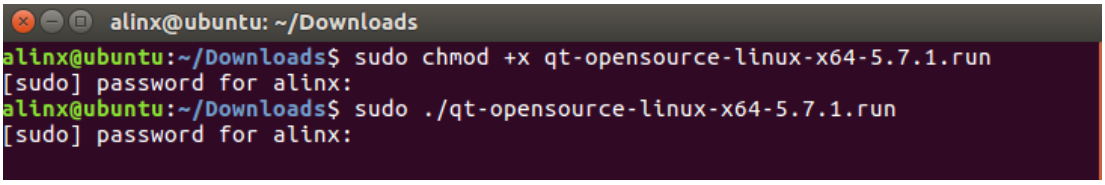
2) 添加运行权限

```
sudo chmod +x qt-opensource-linux-x64-5.7.1.run
```

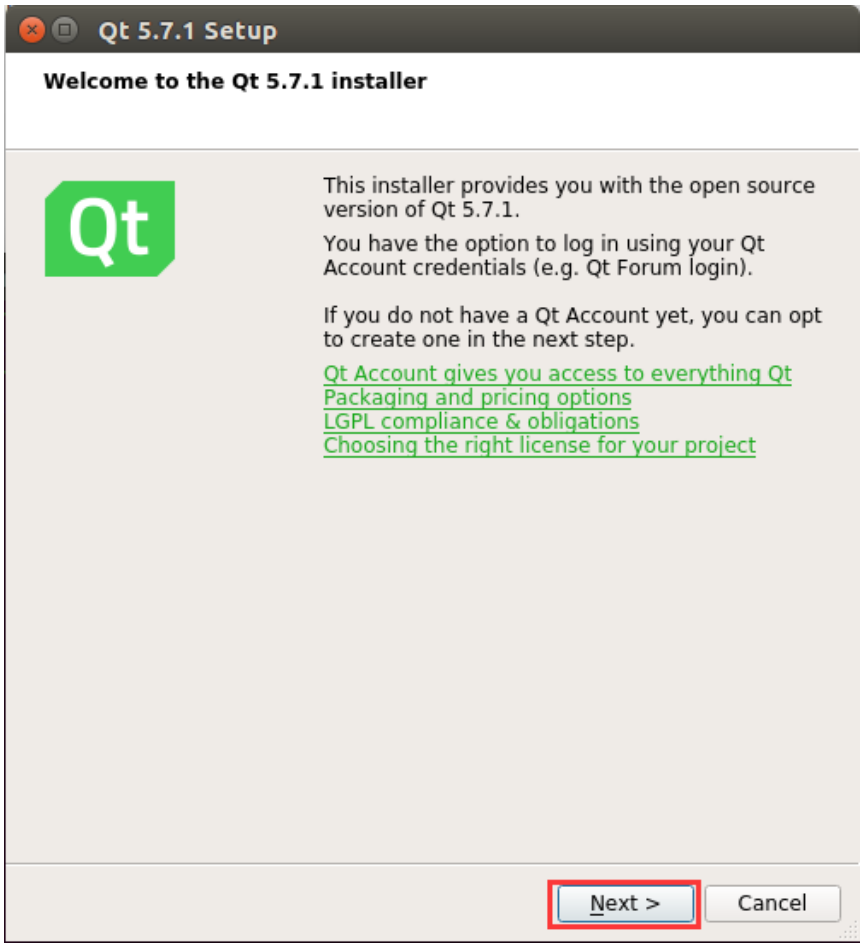


3) 运行安装软件

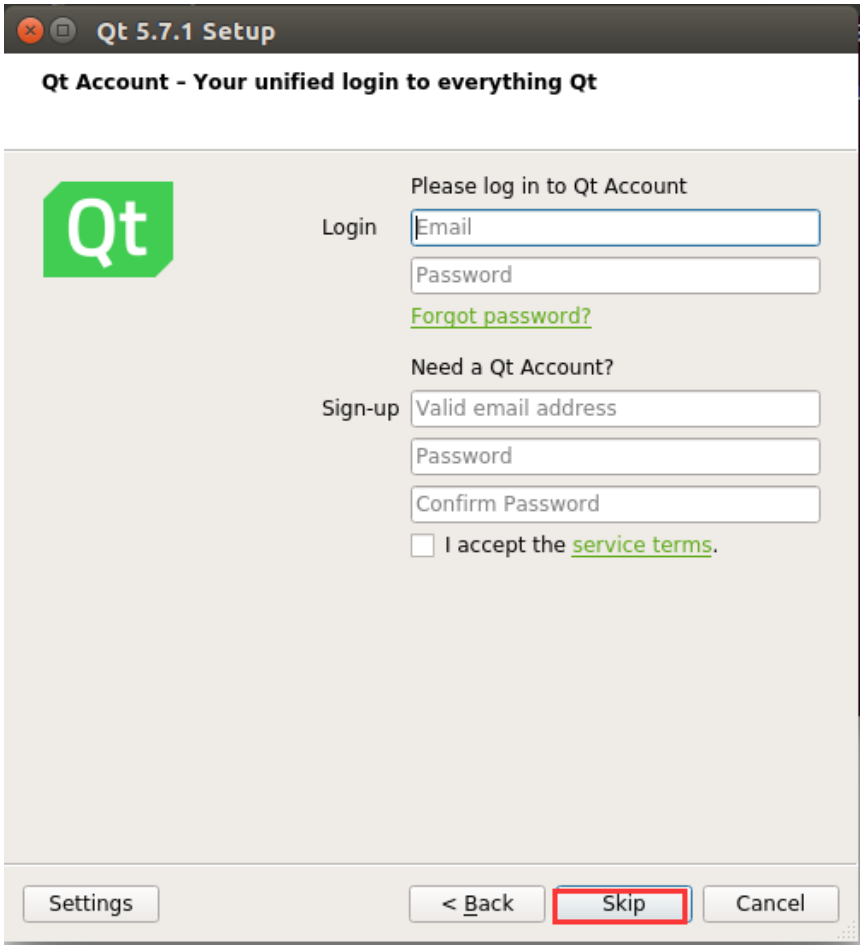
```
sudo ./qt-opensource-linux-x64-5.7.1.run
```



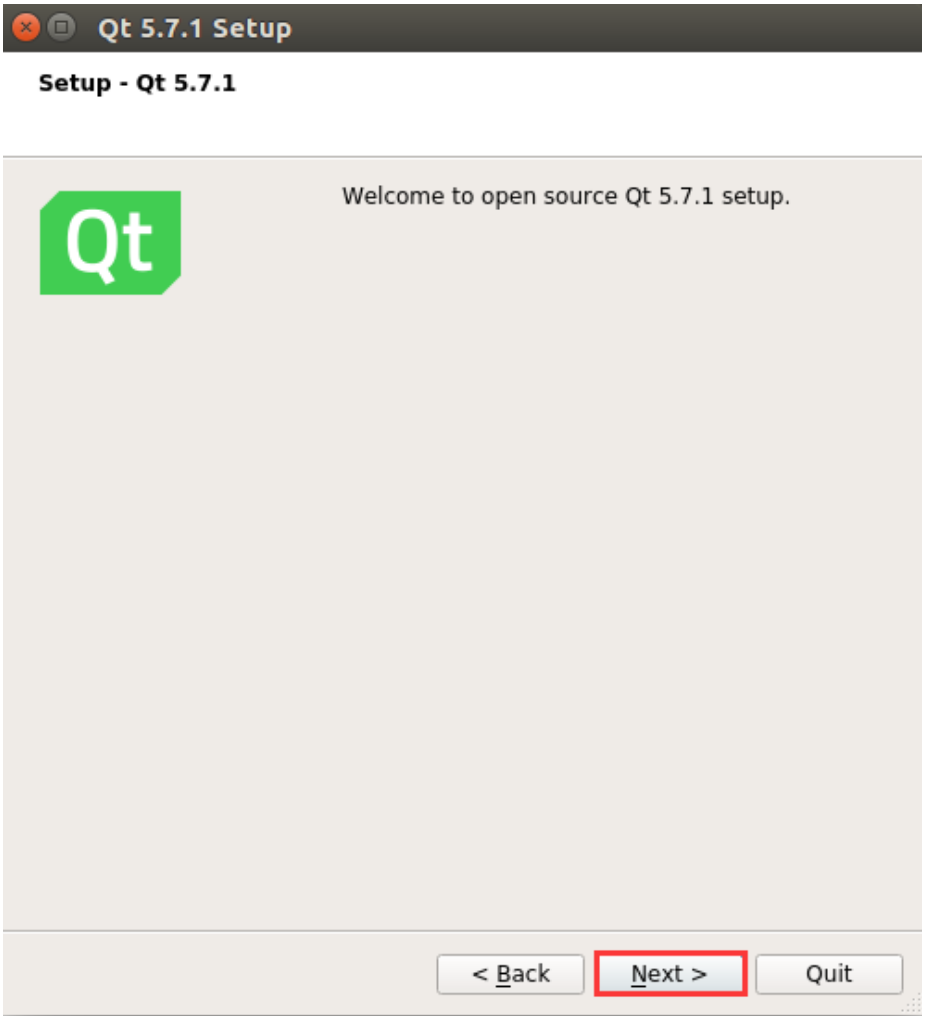
4) 点击 Next 开始安装



5) 点击“Skip”跳过账号信息填写



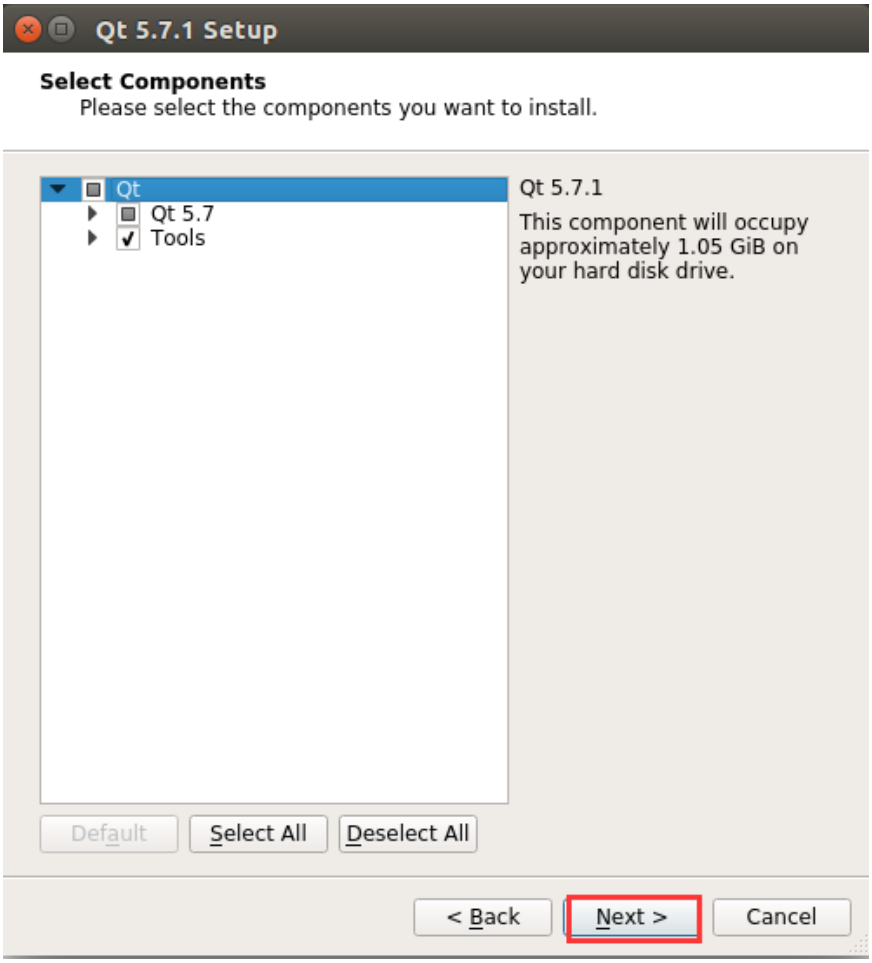
6) 点击“Next”继续安装



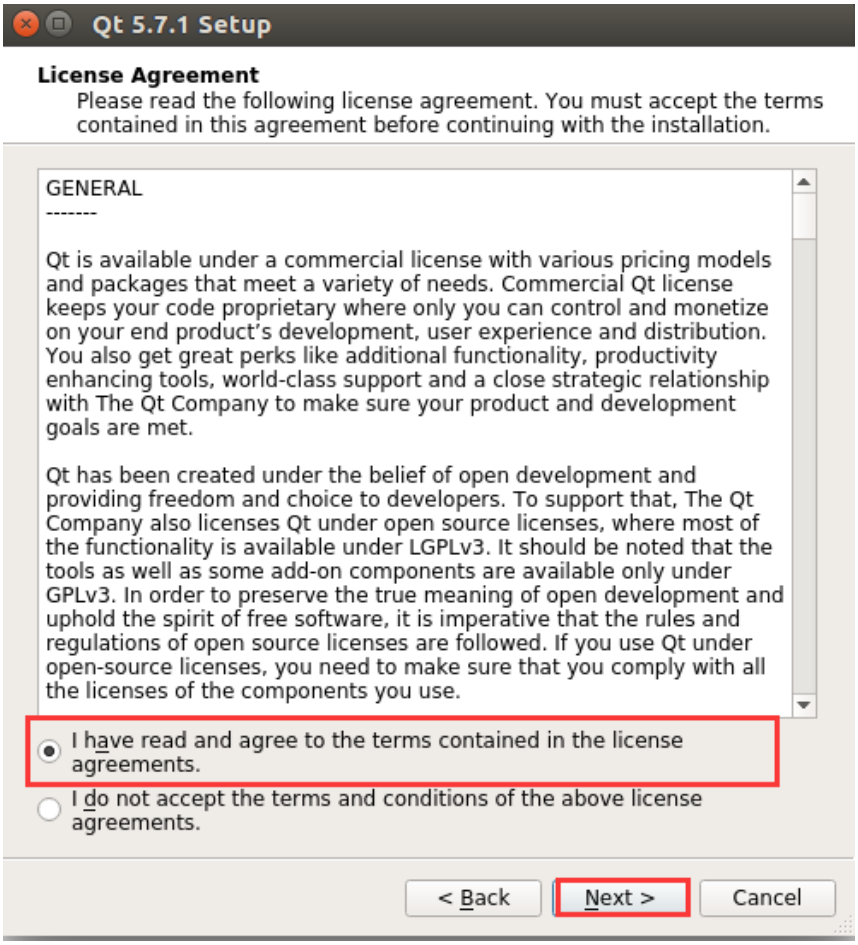
7) 安装路径保持默认，点击“Next”继续安装



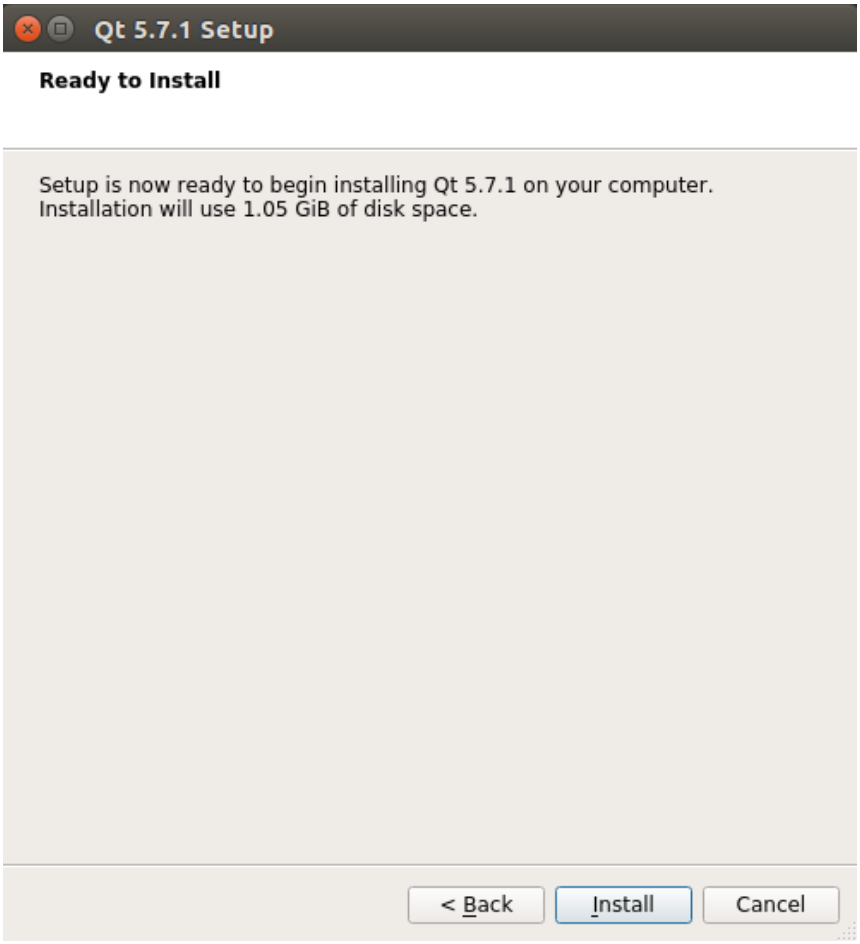
8) 保持默认安装选项，点击“Next”继续安装



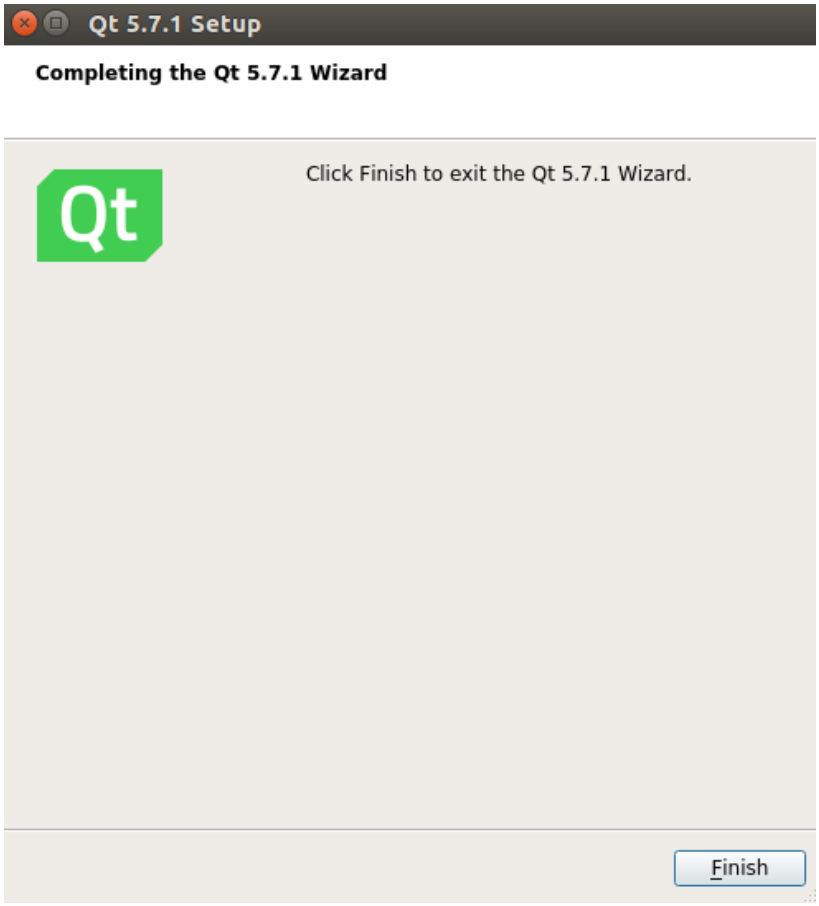
9) 选择同意 License 许可



10) 点击“Install”安装



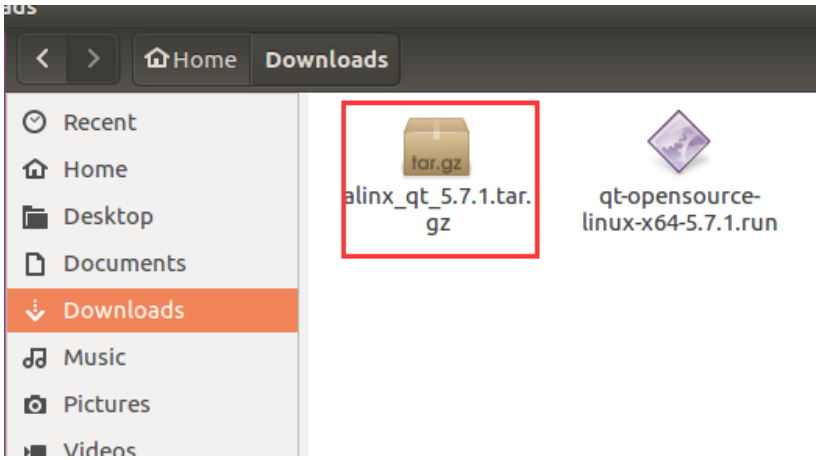
11) 点击“Finish”完成安装



1.2 QT 库交叉编译

前面安装的 QT 可以创建在 x86 处理器上运行的应用程序，不能建立 ZYNQ 能运行的程序，我们需要交叉编译 QT 的源代码才能在 zynq 上运行。为了简化编译流程，芯驿电子准备好了 QT 5.7.1 的源代码包，并制作了一些编译脚本。

1) 将压缩包复制到 Ubuntu 主机中

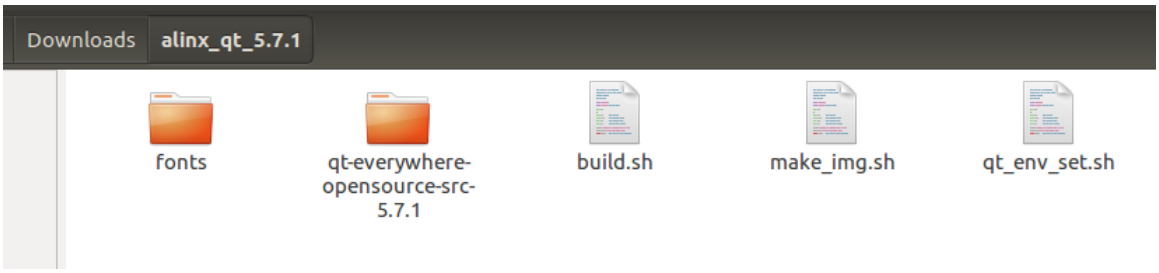


2) 将压缩包解压

```
tar -zxvf alinx_qt_5.7.1.tar.gz
```

```
alinx_qt_5.7.1/qt-everywhere-opensource-src-5.7.1/qtdatavis3d/src/datavisualizat
ion/global/datavisualizationglobal_p.h
alinx@ubuntu:~/Downloads$ tar -zxvf alinx_qt_5.7.1.tar.gz
```

3) 解压后的文件夹，“fonts”是 QT 应用程序显示需要用到的字体文件，“qt-everywhere-opensource-src-5.7.1”是 QT 源代码文件夹，“build.sh”是编译脚本，“make_img.sh”是把编译的库打包成 img 镜像文件。



4) “build.sh”脚本中指定了 XSDK 的安装目录，主要使用 XSDK 自带的交叉编译器，如果你的 SDK 安装目录和脚本里不同，请修改 XSDK 路径，同时指定 ZYNQ 版本 QT 库的安装路径为“/opt/alinx/zynq_qt5.7.1”,编译完成以后可以在这个路径里找到交叉编译过的 QT 库，不建议修改此路径。

```
#!/bin/sh
SDK_PATH=/opt/Xilinx/SDK/2017.4
if [ ! -d $SDK_PATH ]; then
  echo "no dir ${SDK_PATH}"
  exit 1
fi
source ${SDK_PATH}/settings64.sh
export ZYNQ_QT_INSTALL=/opt/alinx/zynq_qt5.7.1
export PKG_CONFIG_LIBDIR=${SDK_PATH}/gnu/aarch32/lin/gcc-arm-linux-gnueabi/arm-linux-gnueabihf/lib
export CROSS_COMPILE=arm-linux-gnueabihf-

if [ ! -d $PKG_CONFIG_LIBDIR ]; then
  echo "no ${PKG_CONFIG_LIBDIR}"
  exit 1
fi

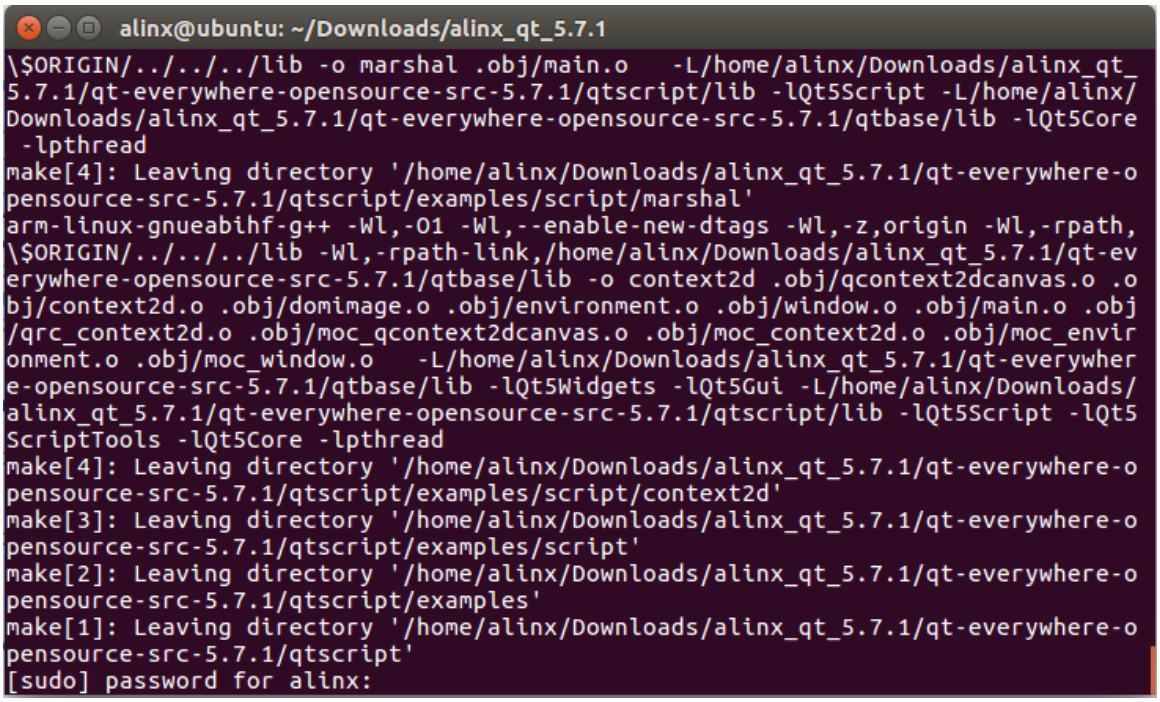
Cur_Dir=$(pwd)
QT_SORCE=qt-everywhere-opensource-src-5.7.1
echo $Cur_Dir
export ZYNQ_QT_BUILD=$Cur_Dir
export PATH=$ZYNQ_QT_INSTALL/bin:$PATH
cd $QT_SORCE
./configure -xplatform arm-linux-gnueabihf-g++ \
    -opensource \
    -confirm-license \
```

5) 运行编译脚本

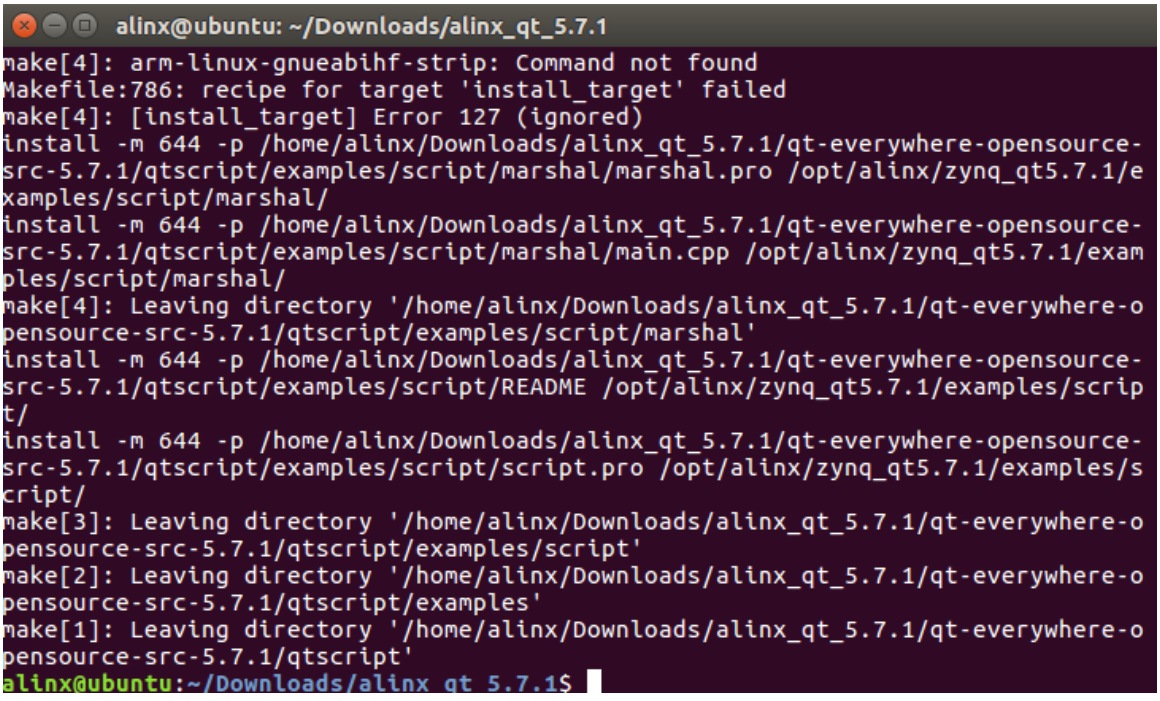
```
./build.sh
```

```
alinx@ubuntu: ~/Downloads/alinx_qt_5.7.1
alinx@ubuntu:~/Downloads/alinx_qt_5.7.1$ ./build.sh
```

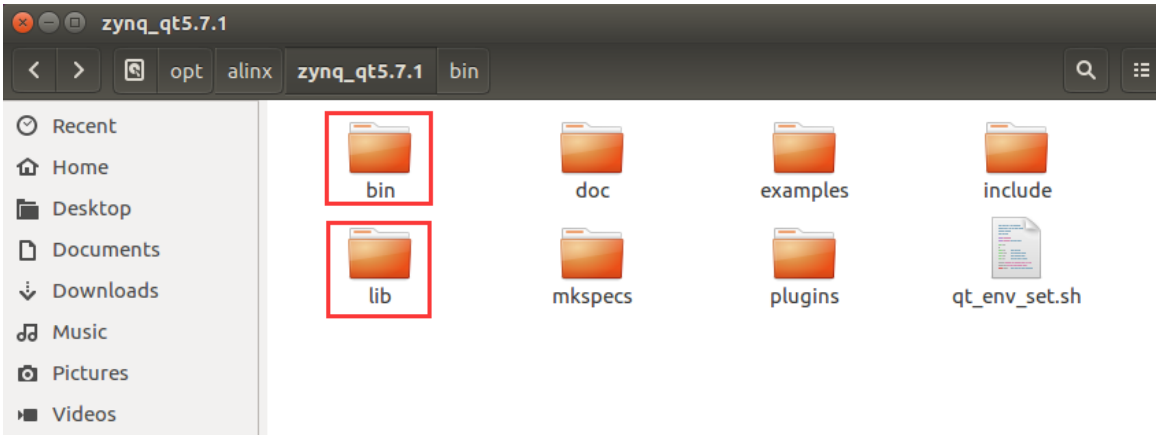
6) 等待一段时间，编译快完成时需要输入 Ubuntu 账户密码



7) 输入密码后，编译安装完成

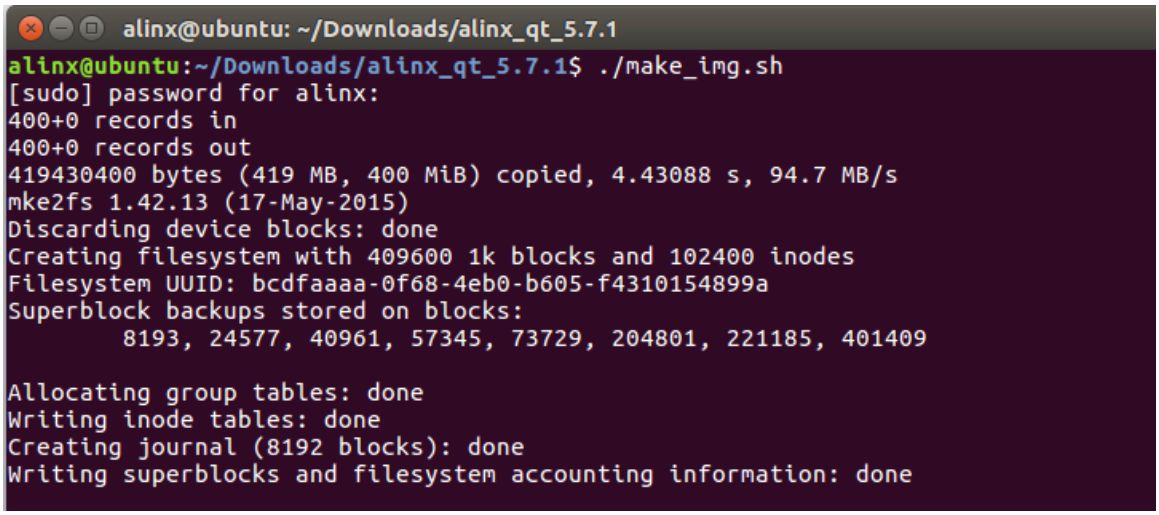


8) 到目录“/opt/alinx/zynq_qt5.7.1”下我们可以找到 bin 目录，里面有我们要用到的 qmake 文件，和 lib 目录，这个目录就是我们需要的 QT 库。

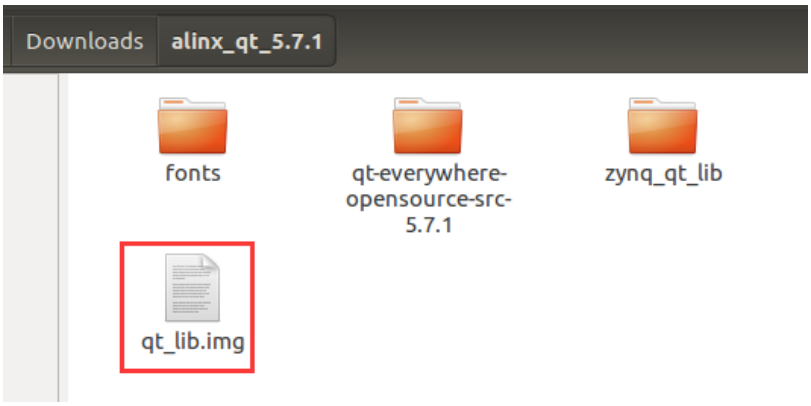


9) 运行脚本制作 QT 库的 img 文件，需要输入账号密码

```
./make_img.sh
```

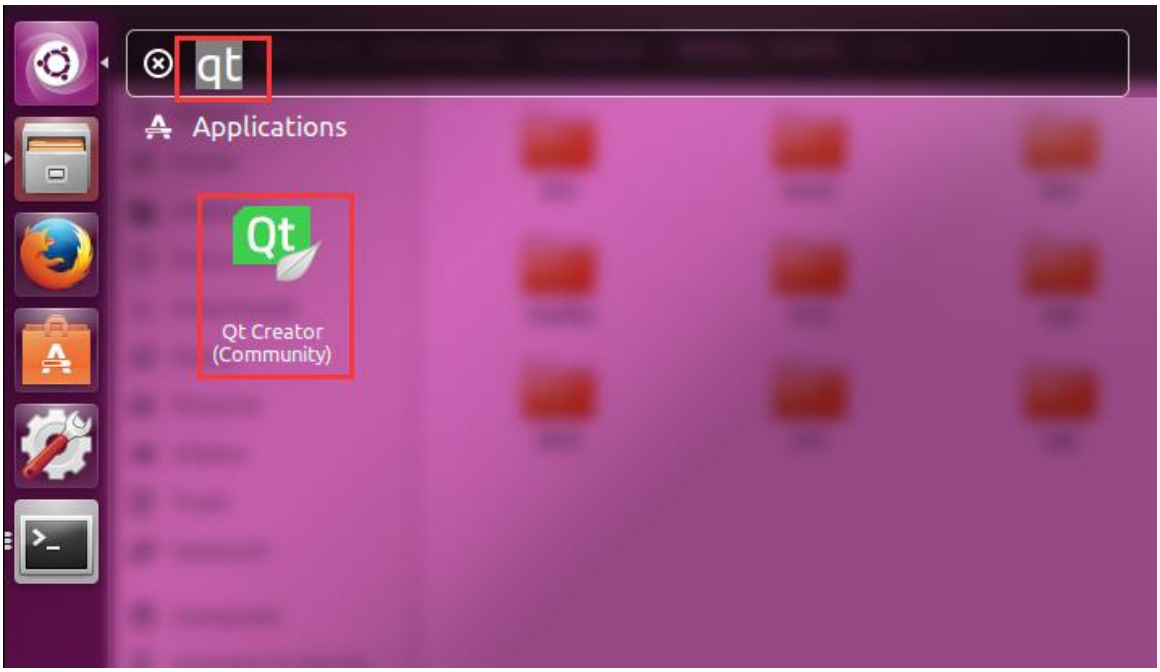


10) 得到 qt_lib.img 可以在 zynq 板上挂载

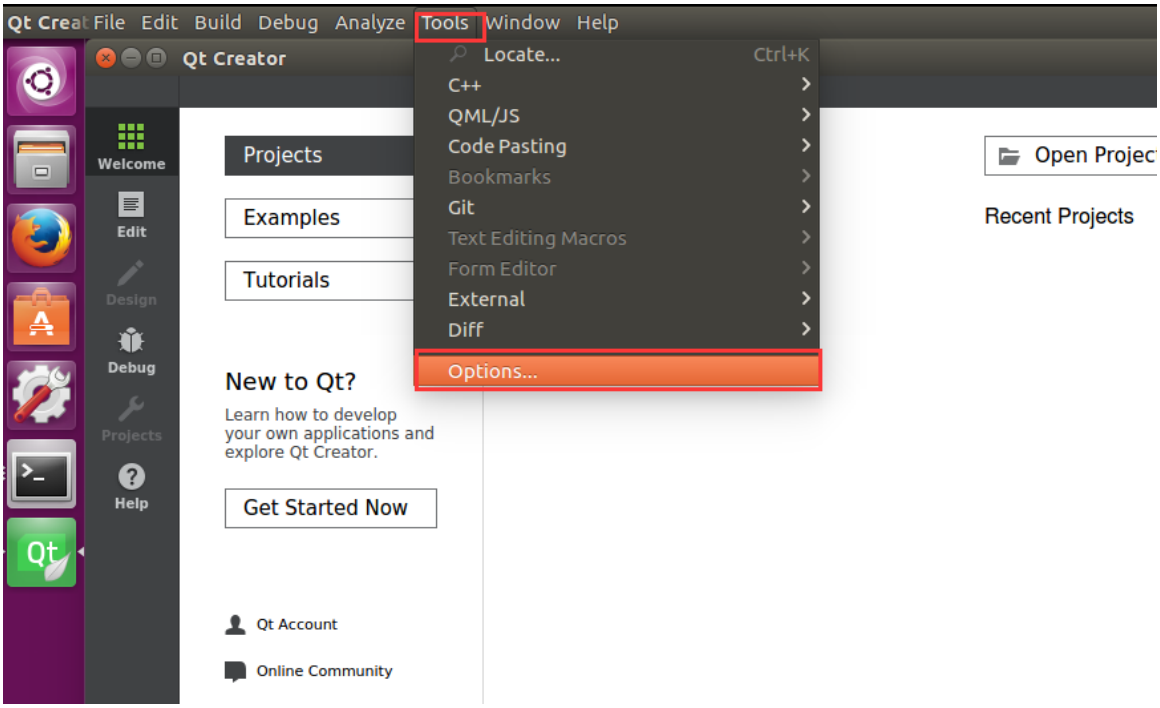


1.3 配置 ZYNQ 版 QT Kits

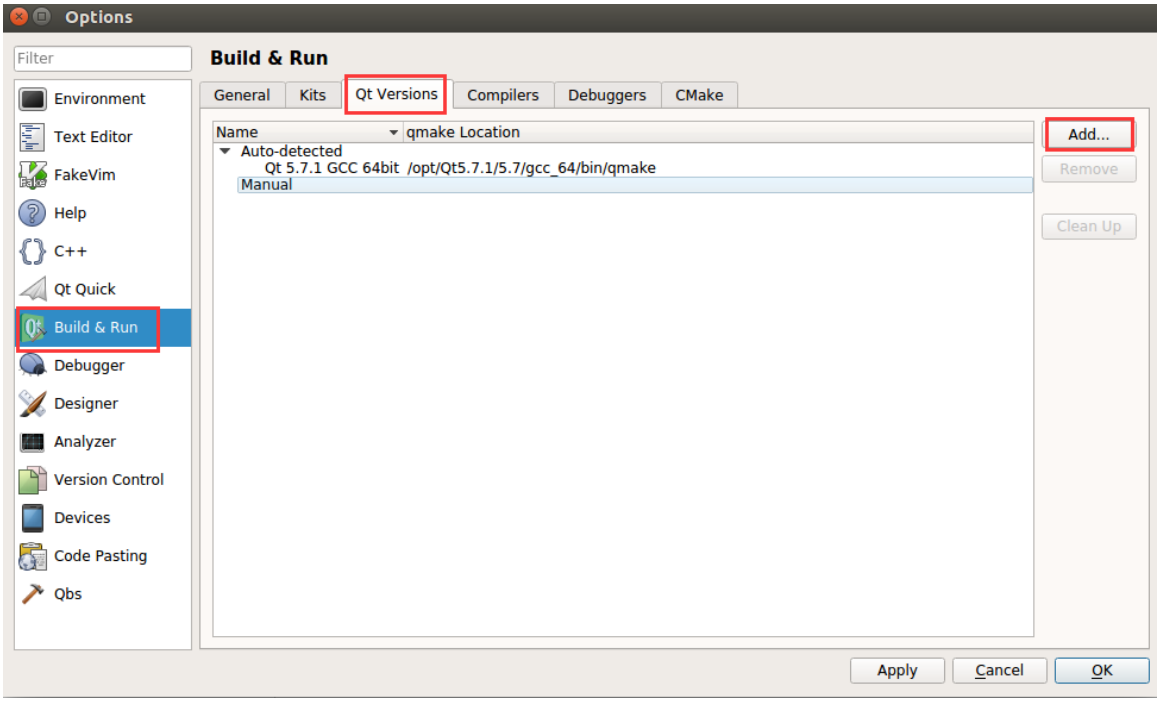
1) 运行 QT Creator，找不到应用程序快捷方式，可以搜索一下。



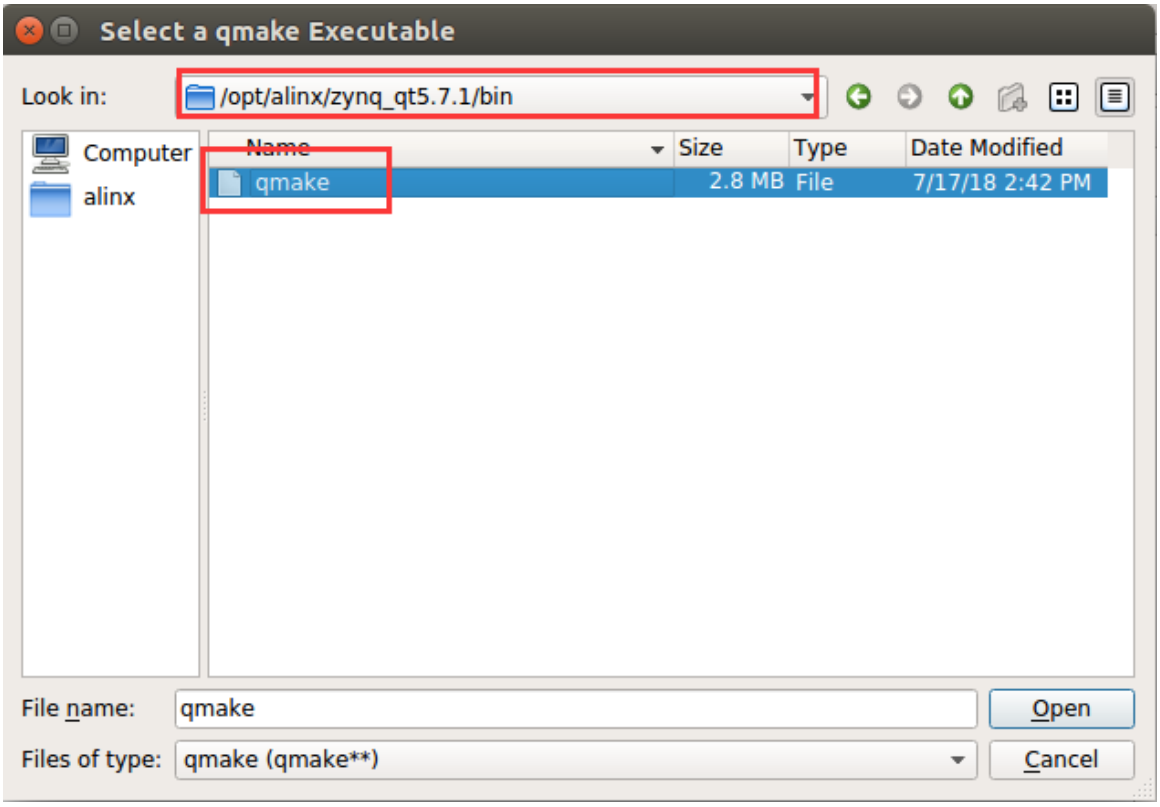
2) 点击 Tools -> Options



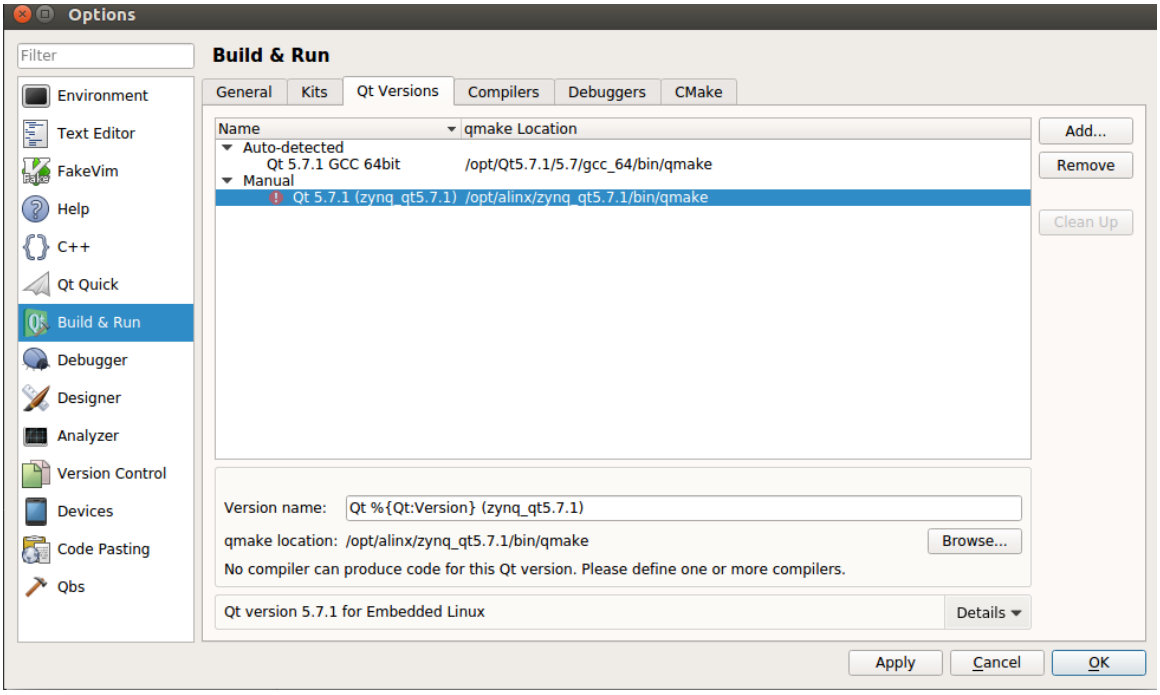
3) 在 Build & Run 选项的 Qt Versions 页点击 Add..



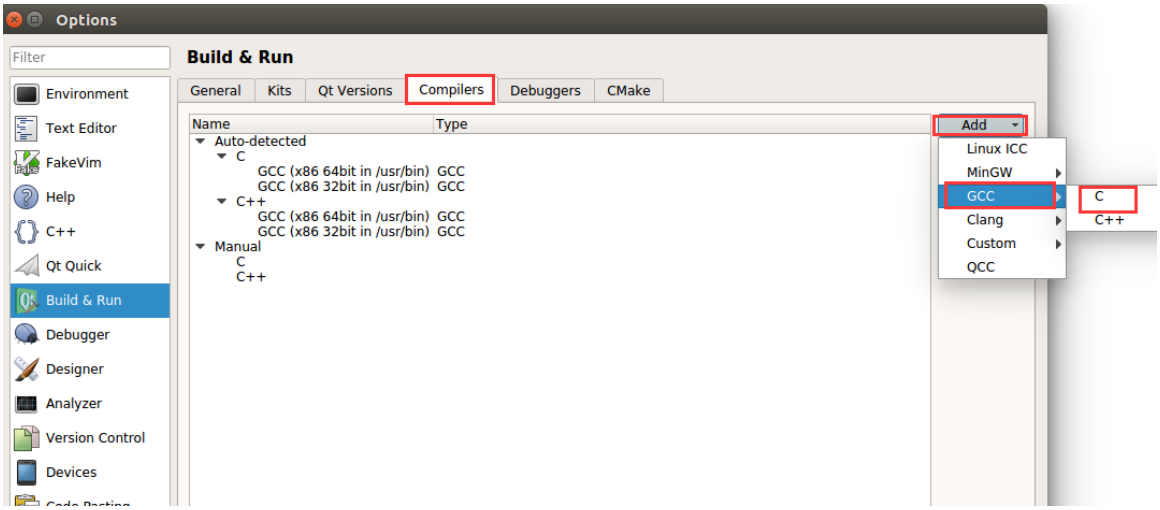
4) 选择前面编译生成的 qmake 文件



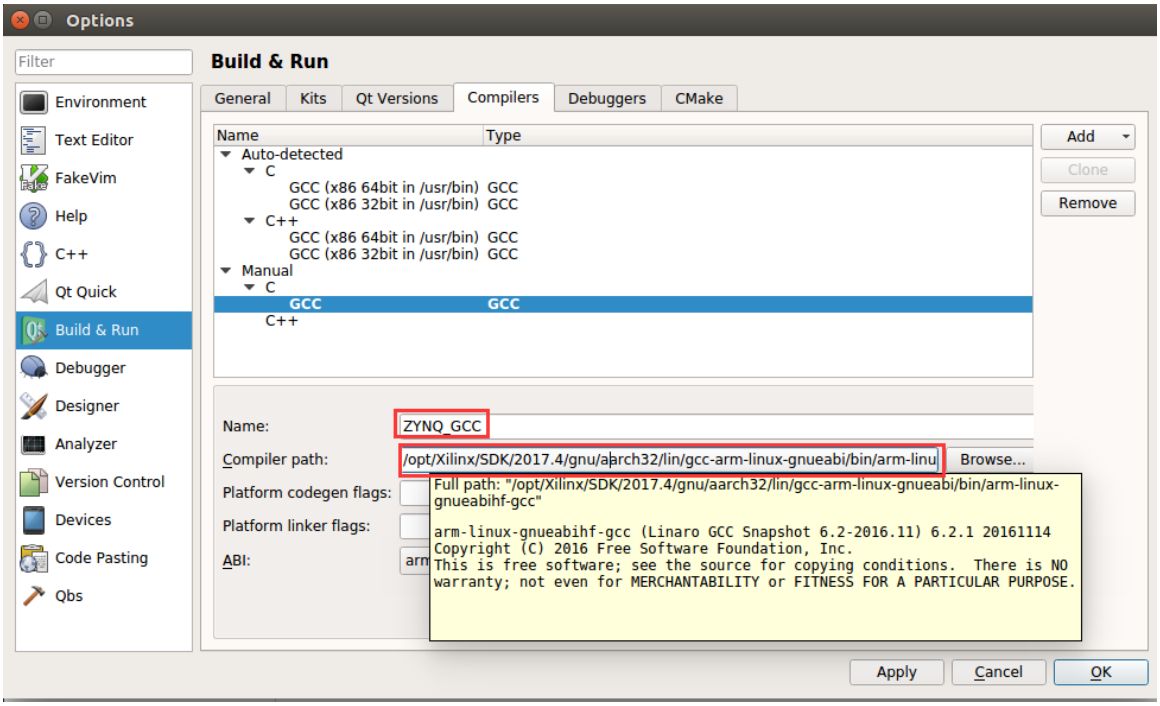
5) 添加 qmake 以后，保持默认 name，不修改



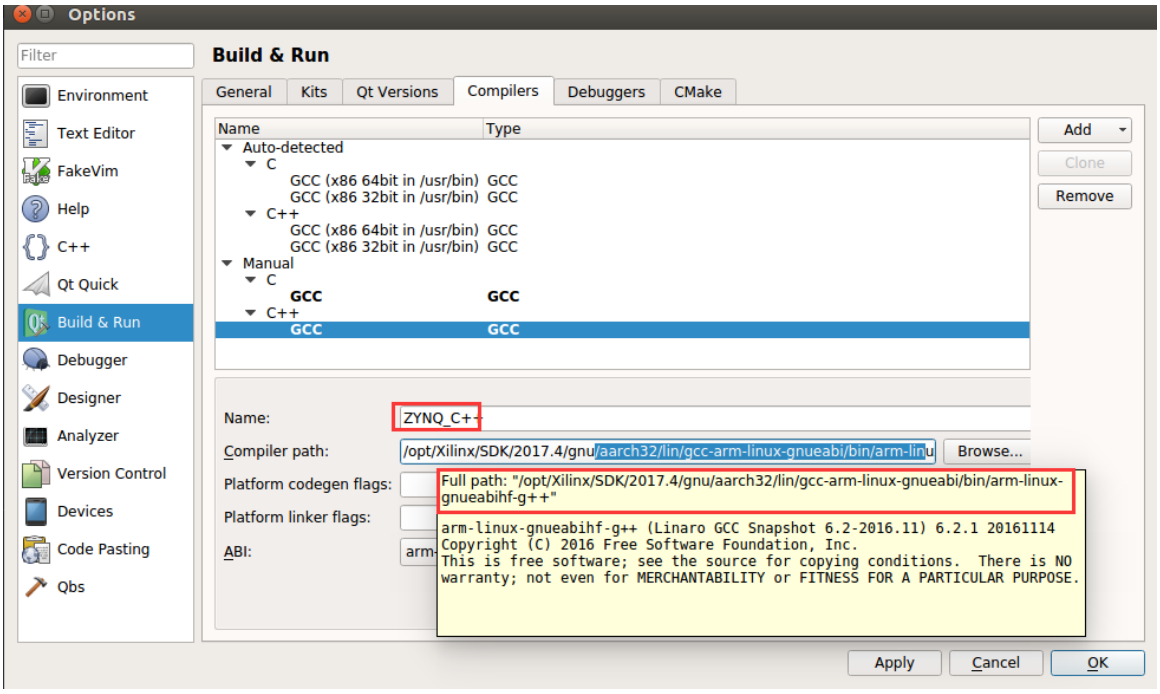
6) 在 Compilers 页面，点击 Add..，选择 GCC -> C



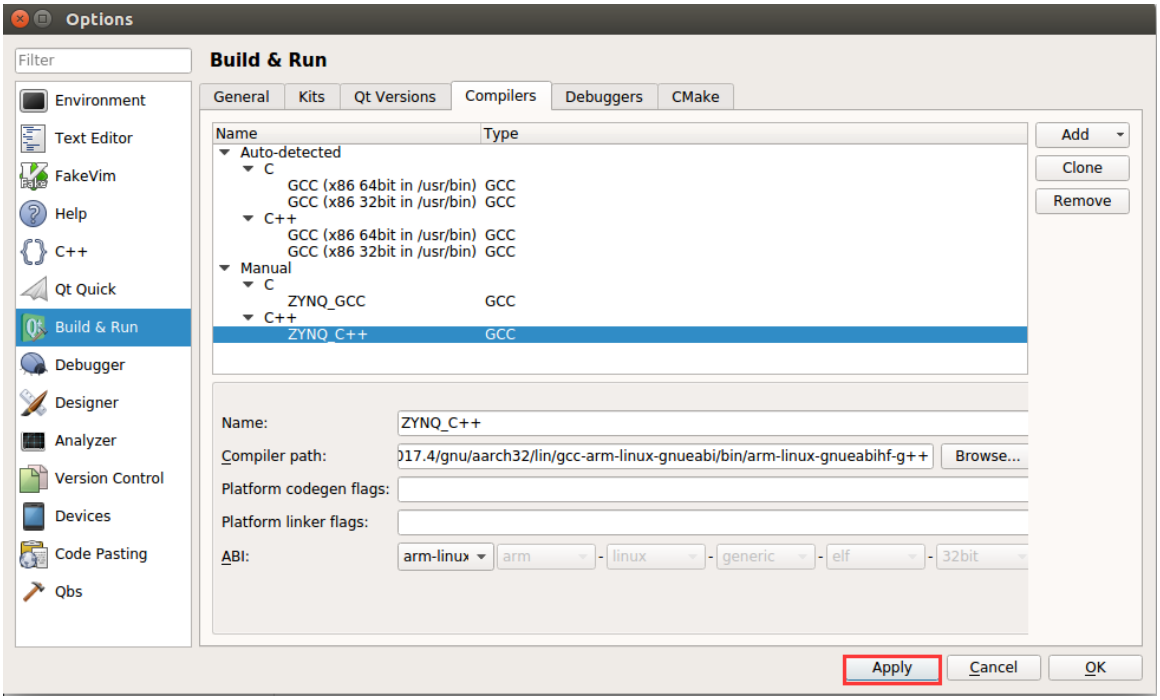
7) Name 修改为 ZYNQ_GCC，Compiler path 选择 /opt/Xilinx/SDK/2017.4/gnu/aarch32/lin/gcc-arm-linux-gnueabi/bin/arm-linux-gnueabihf-gcc



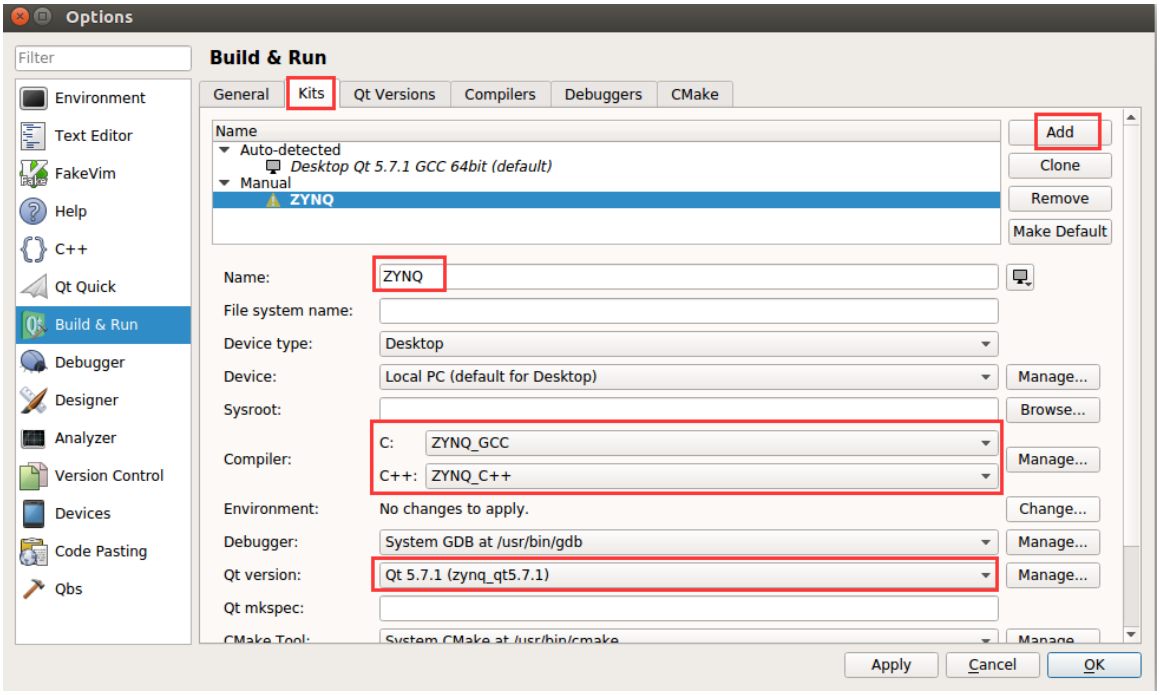
8) 在 Compilers 页面，点击 Add..，选择 GCC -> C++，Name 修改为 ZYNQ_C++，路径选择 /opt/Xilinx/SDK/2017.4/gnu/aarch32/lin/gcc-arm-linux-gnueabi/bin/arm-linux-gnueabihf-g++



9) 再点击“Apply”，更新信息



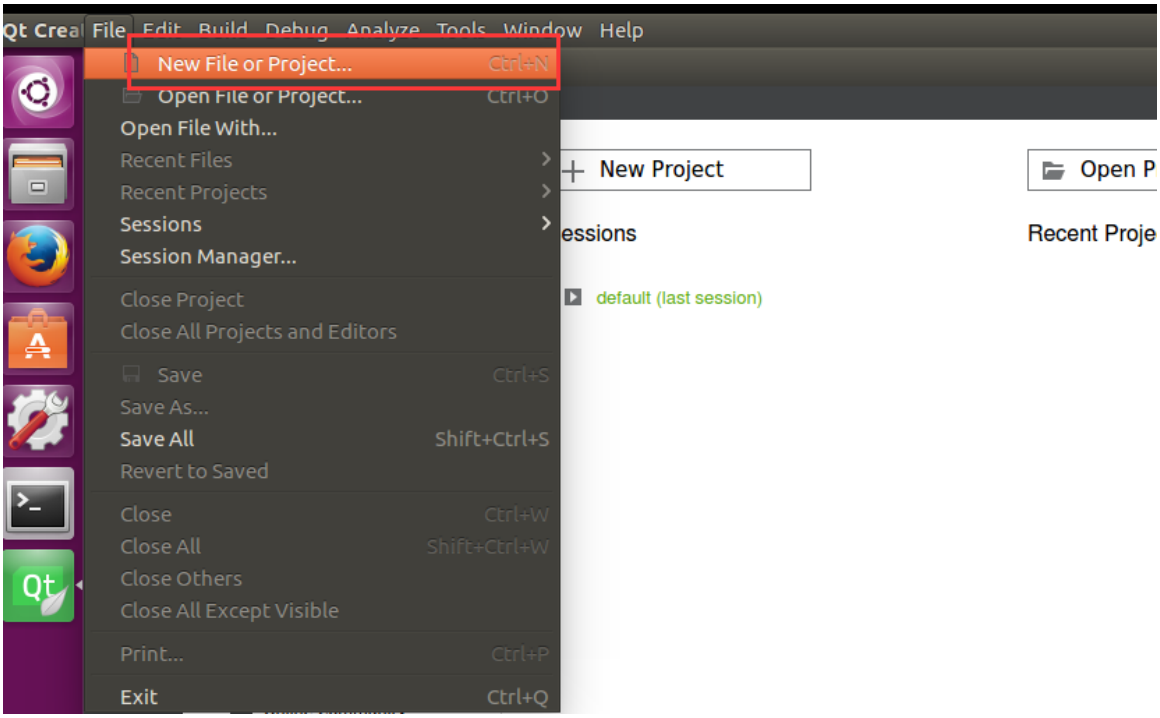
在 Kits 页，点击 Add 添加一个新的 Kit，Name 修改为 ZYNQ，Compiler C：选择 ZYNQ_GCC，Compiler C++选择 ZYNQ_C++，Qt version 选择 Qt 5.7.1(zynq_qt5.7.1)



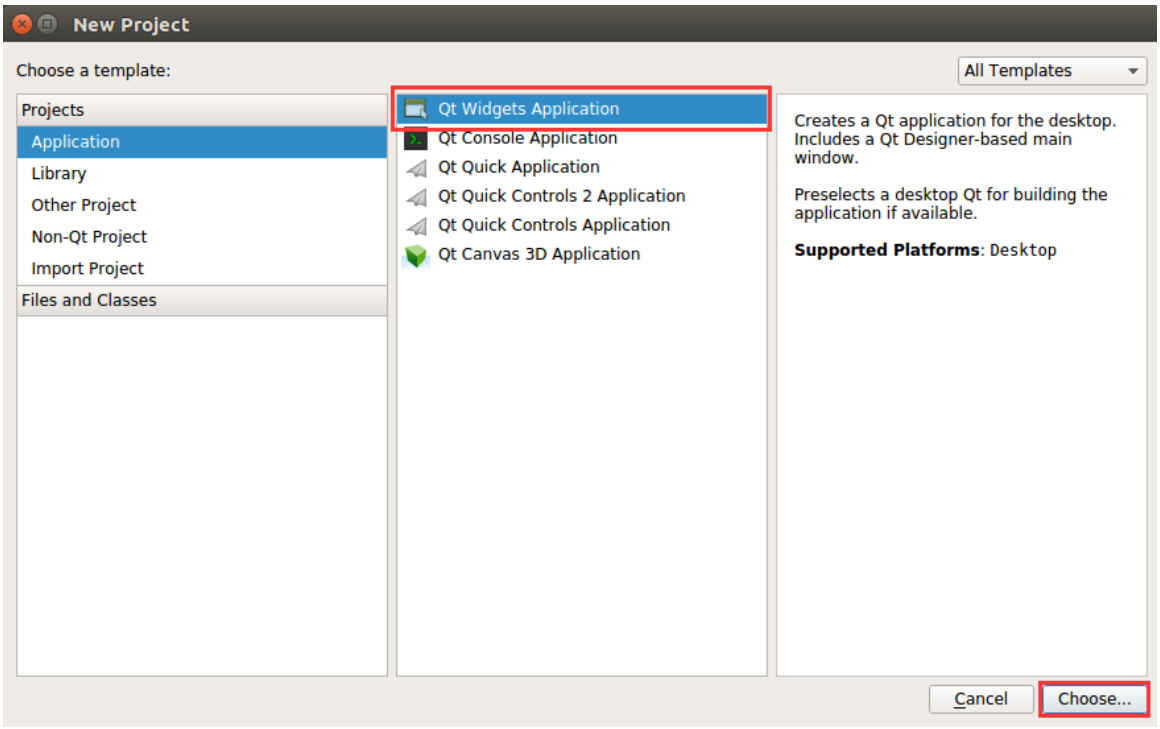
10) 点击 OK 配置 Kits 完成

1.4 创建 QT 测试工程

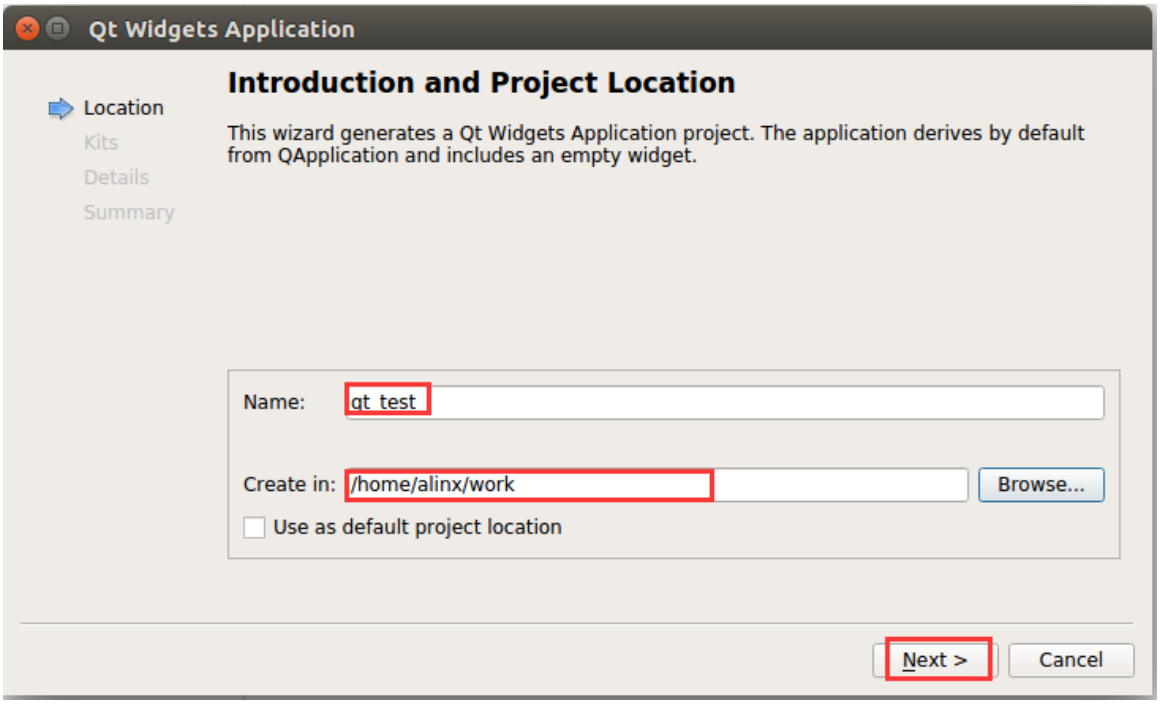
1) 点击 File -> New File or Project



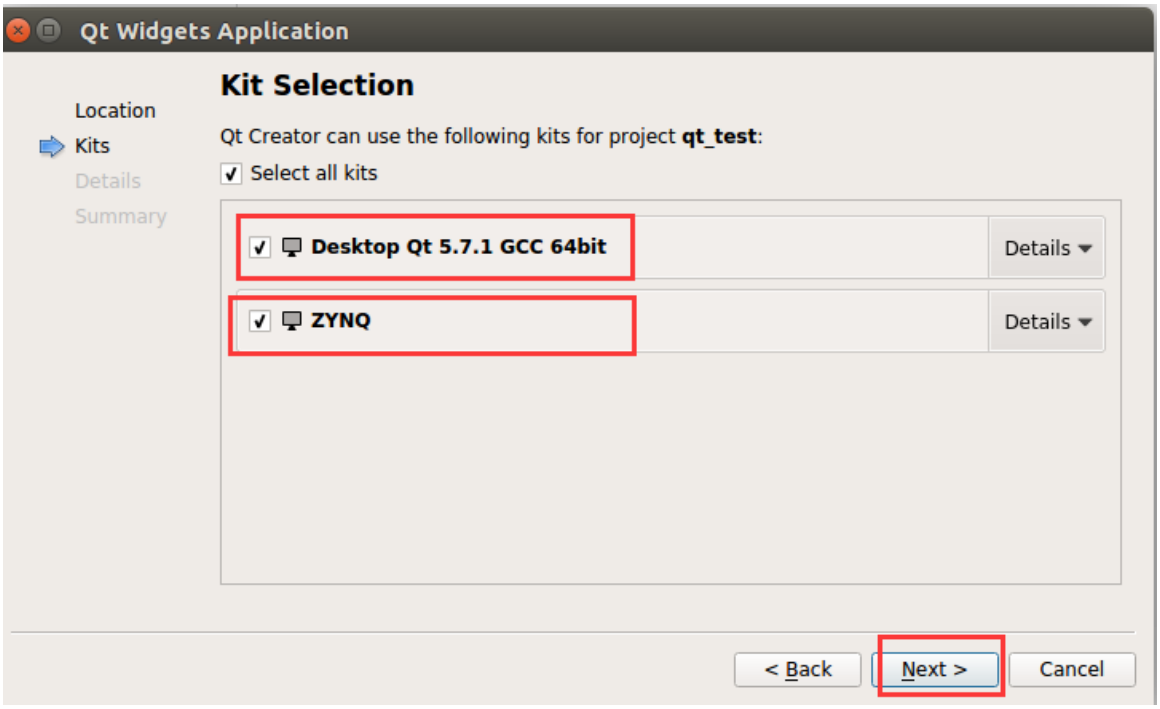
2) 工程模板选择“Application”“Qt Widgets Application”，然后点击“Choose...”



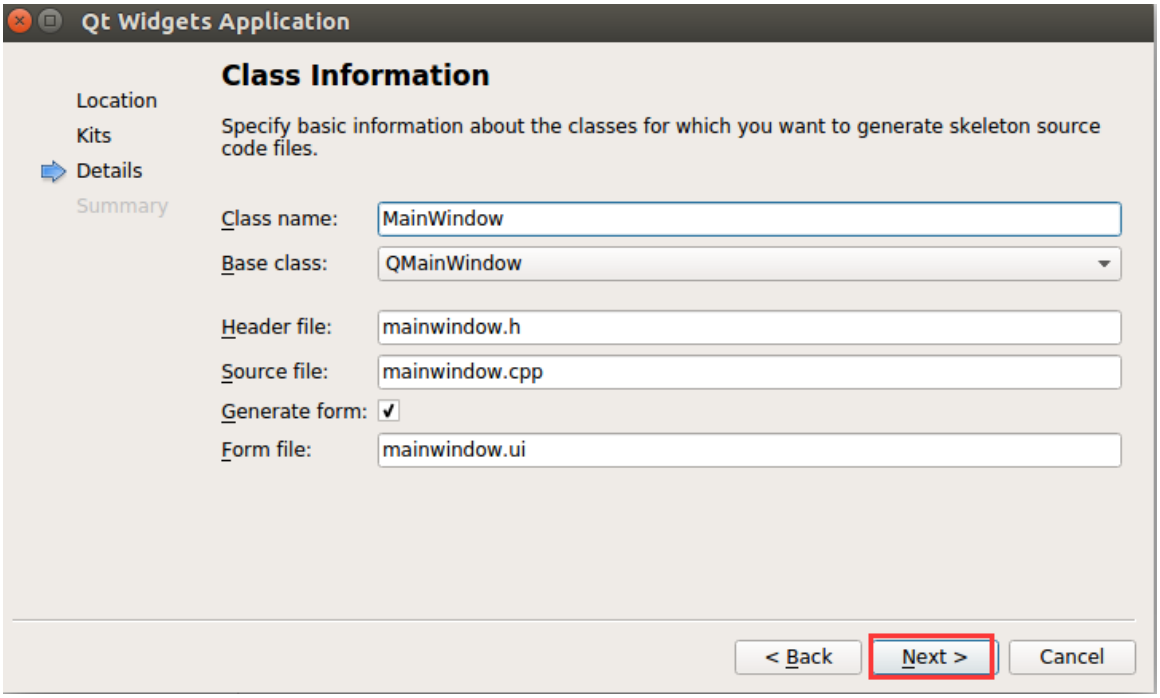
3) Name 修改为 qt_test，路径本实验选择/home/alinx/work



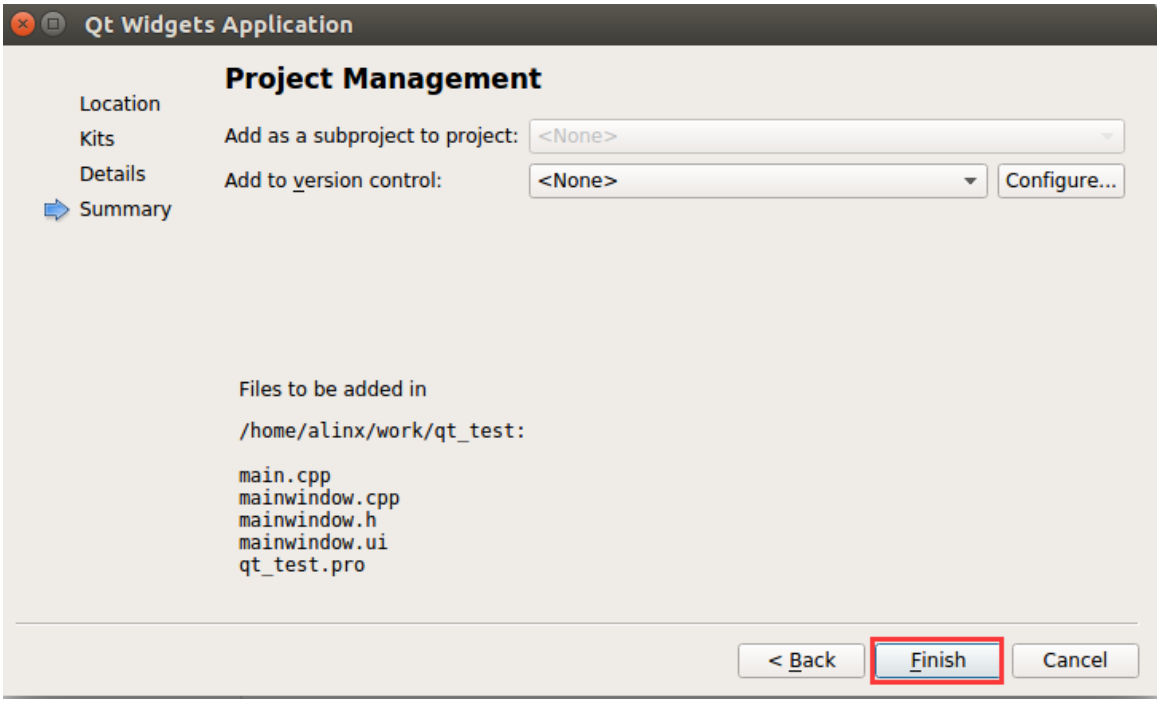
4) kit 选择，这里选择 ZYNQ 版本



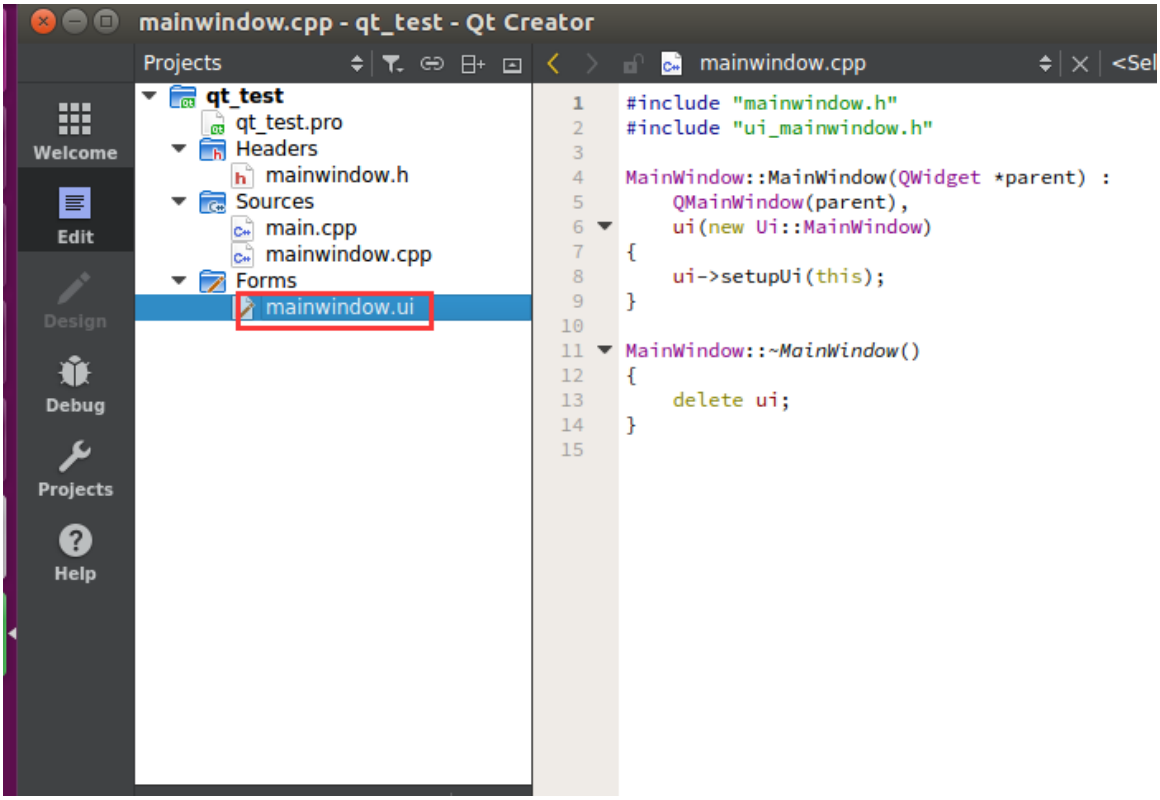
5) 选择 Next 继续



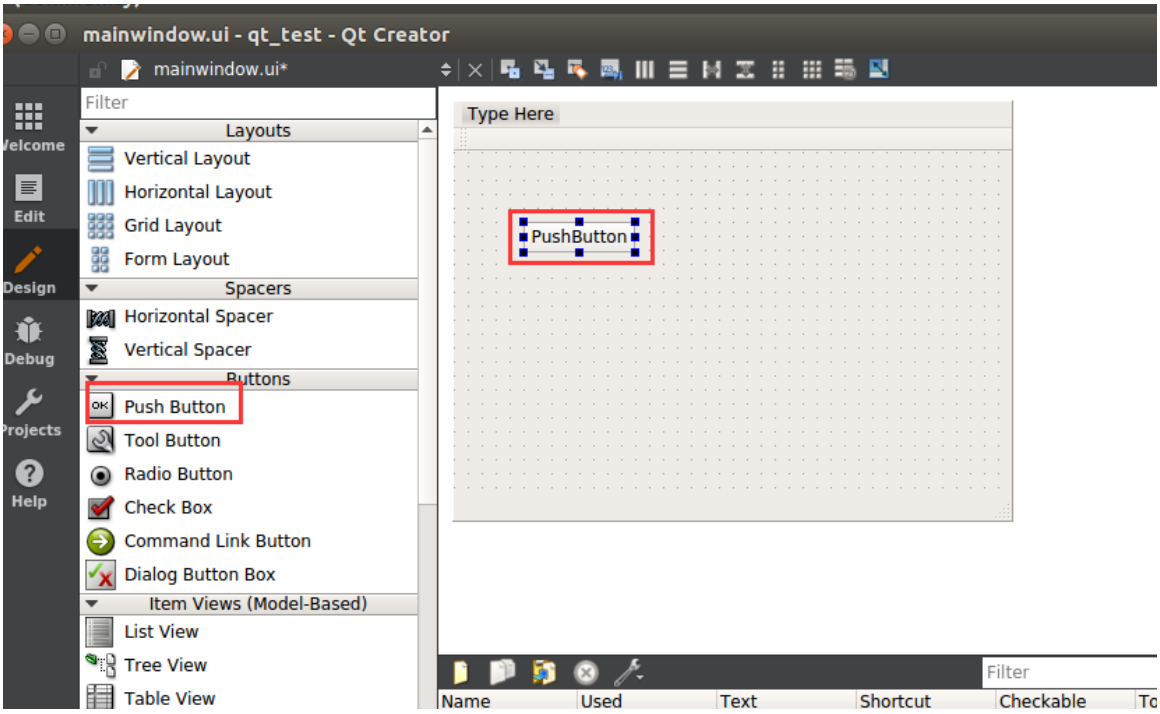
6) 选择 Finish 结束工程创建向导



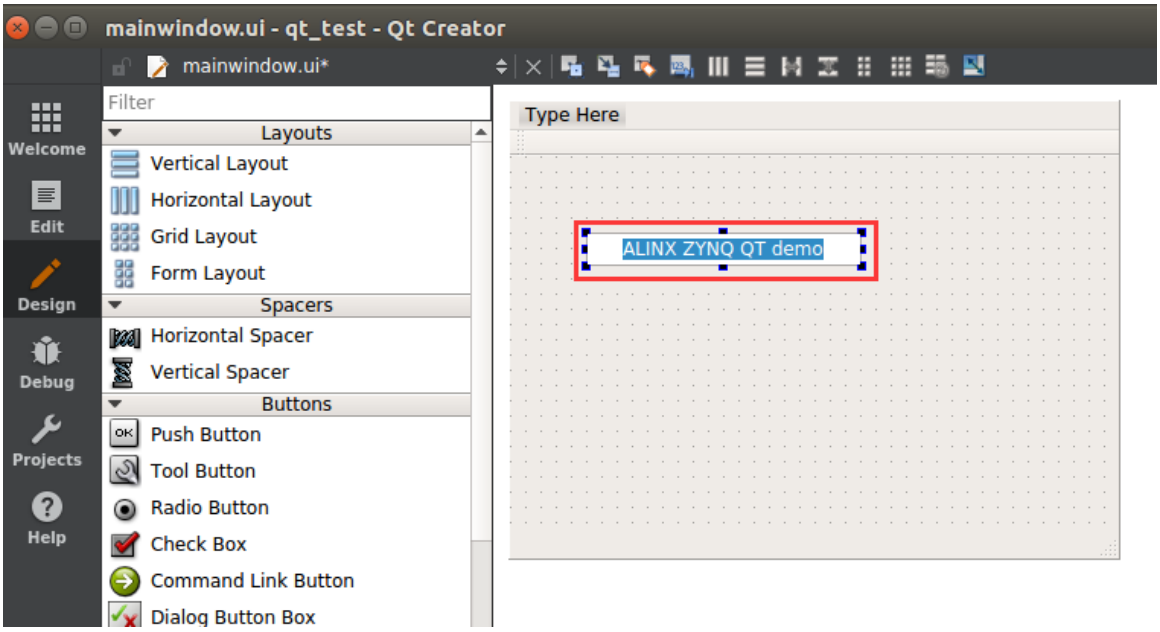
7) 双击 mainwindow.ui 文件



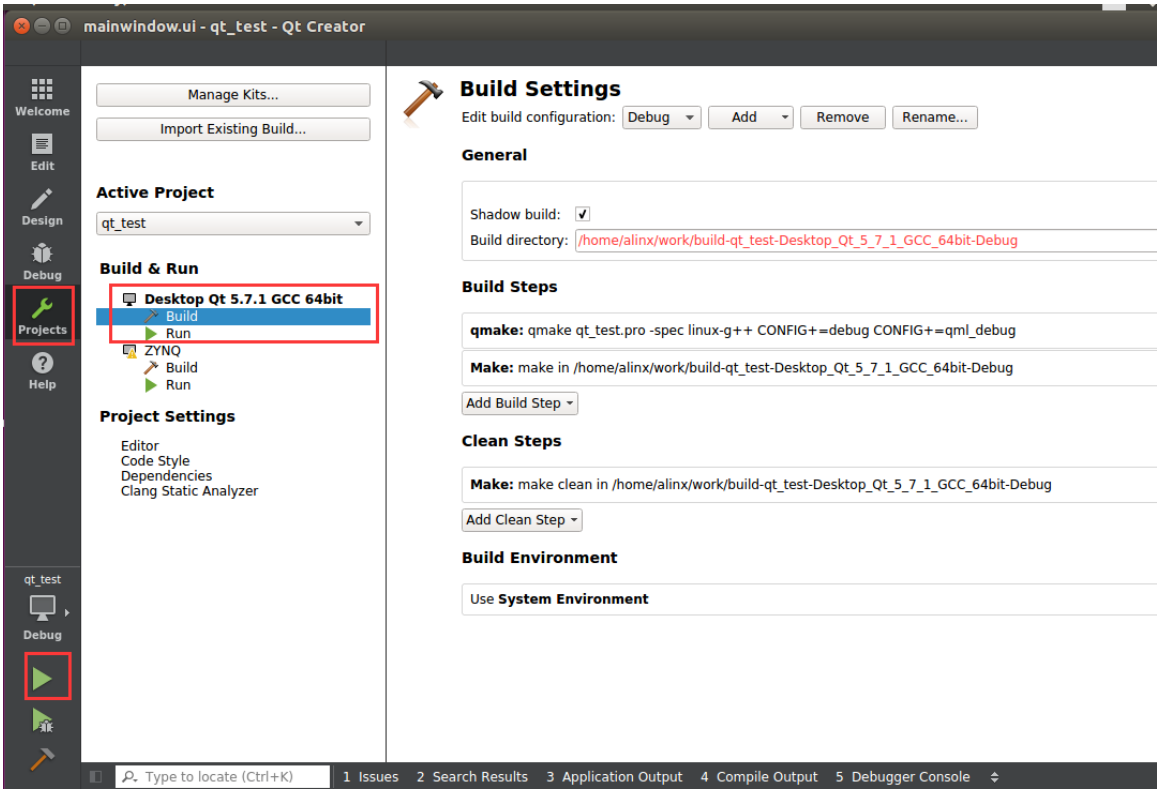
8) 拖拽一个 Push Button 到主界面中



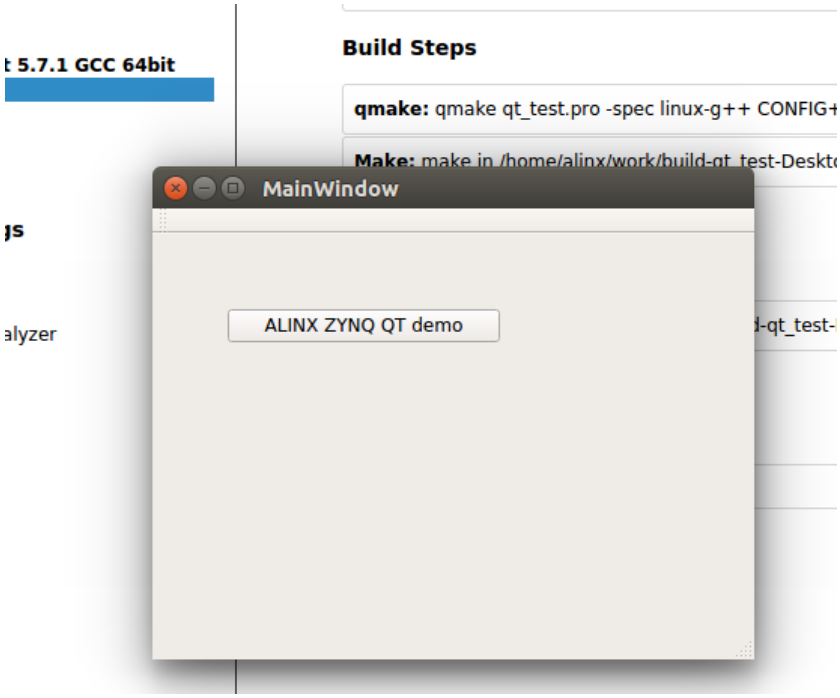
9) 双击文字部分，可以修改按钮文字



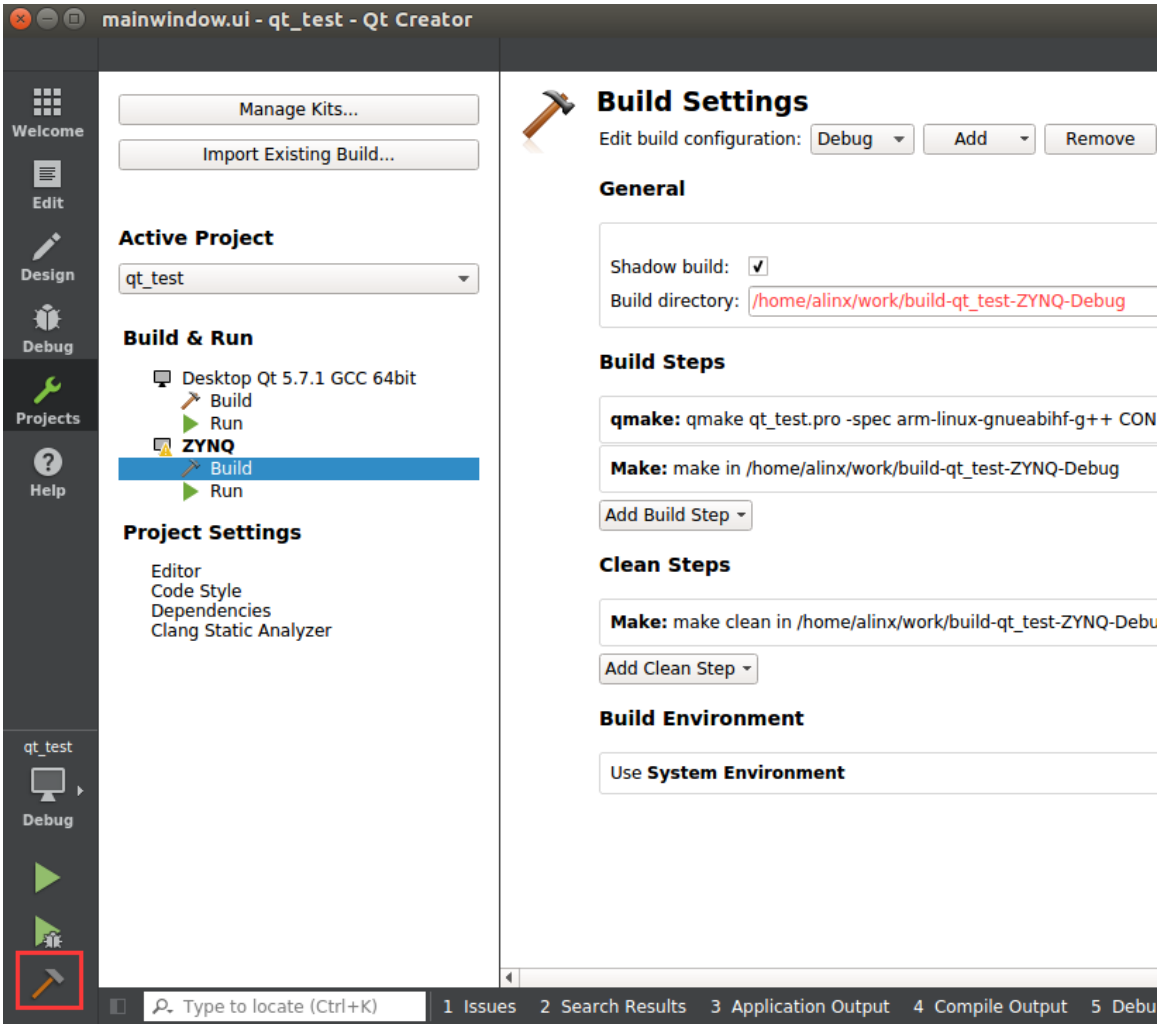
10) 在 Projects 选择，选择“Desktop Qt 5.7.1 GCC 64bit”,然后点击绿色三角运行 Host 版本



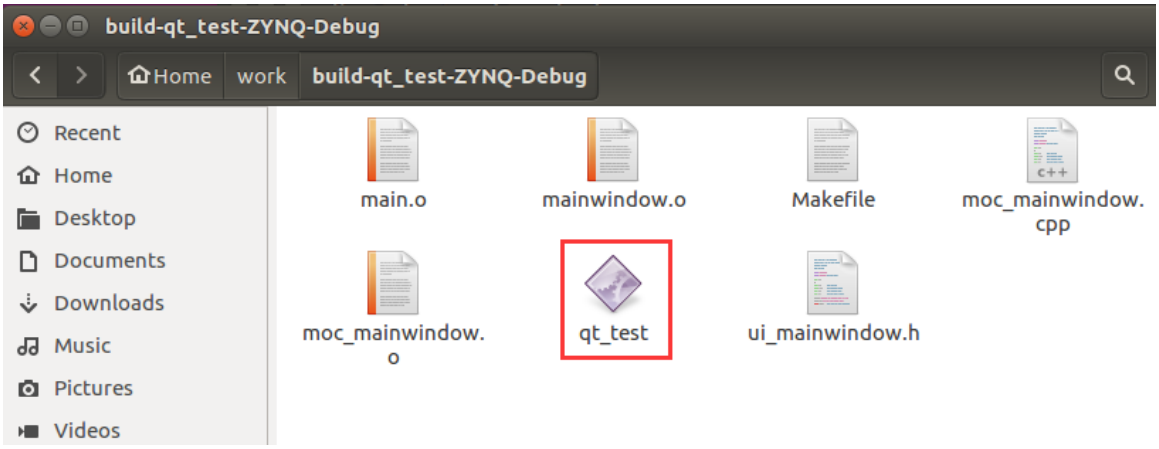
11) 运行效果



12) 选择 ZYNQ Kit，点击锤子图标，只编译程序，不运行



13) 在 build-qt_test-ZYNQ-Debug 目录可以看到生成了一个 qt_test 的文件，这个文件要在 ZYNQ 上运行。



1.5 运行 ZYNQ 版本 QT 程序

运行 QT 首先需要显示设备，在前面的教程中学习了如何通过 petalinux 创建 Linux，本章不再讲解如何创建，例程中提供了相关开发板的 petalinux 工程。

1) 准备好开发板，连接好显示器，将例程给的 BOOT.BIN，image.ub 复制到 sd 卡（sd_boot 目录），建议先格式化 sd 卡，再复制。

2) 使用串口终端登录系统，默认账户 root，密码 root

```
IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
Sending discover...
Sending select for 192.168.1.46...
Lease of 192.168.1.46 obtained, lease time 86400
/etc/udhcpc.d/50default: Adding DNS 192.168.1.1
done.
Starting Dropbear SSH server: Generating key, this may take a while...
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDS6PnYncg3QkvRX9ykZpeL1B9GEmuj83s8PqHb+OZs
72dZpqajFznfrRtE1NP7agPaD646HACC4AsFYT34ds+eAf8tx4hxNRxn8ZdTmFhONdq/A960OZ1Uy2qV
Fingerprint: md5 b8:c8:a4:a4:70:9e:ff:e4:00:15:41:d8:c2:21:f3:2d
dropbear.
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2017.4 an5642_peta /dev/ttyPS0

an5642_peta login: root
Password:
root@an5642_peta:~#
```

3) 挂载 Ubuntu 主机 NFS 服务，不会使用 NFS 的开发者可参考前面的教程，这里的 192.168.1.55 为实验时 Ubuntu 主机 IP 地址，和开发板在同一个网段。

```
mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
```

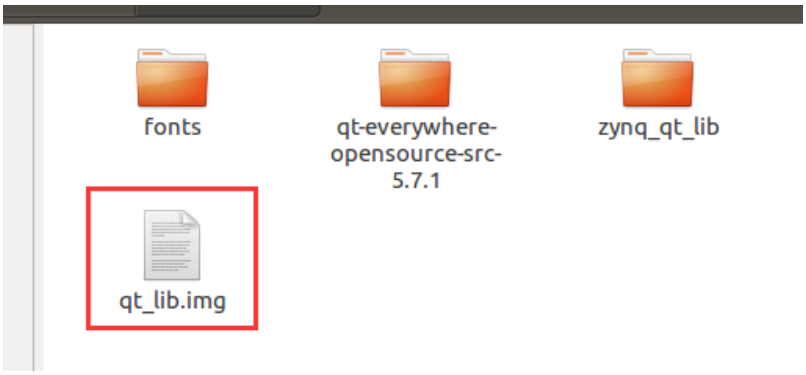
4) 然后进入/mnt 目录

```
cd  /mnt
```

5) 建立一个目录用于挂载 QT 库

```
mkdir  /tmp/qt
```

6) 将 QT 库复制到 NFS 服务目录，便于开发板使用



7) 挂载 QT 库

```
mount  qt_lib.img  /tmp/qt
```

```
root@an5642_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt#
```

8) 进入 qt 库目录，运行环境变量设置脚本

```
cd  /tmp/qt
source  ./qt_env_set.sh
```

```
root@an5642_peta:/mnt# cd /tmp/qt
root@an5642_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@an5642_peta:/tmp/qt#
```

9) 进入/mnt 目录，再进入 build-qt_test-ZYNQ-Debug 目录

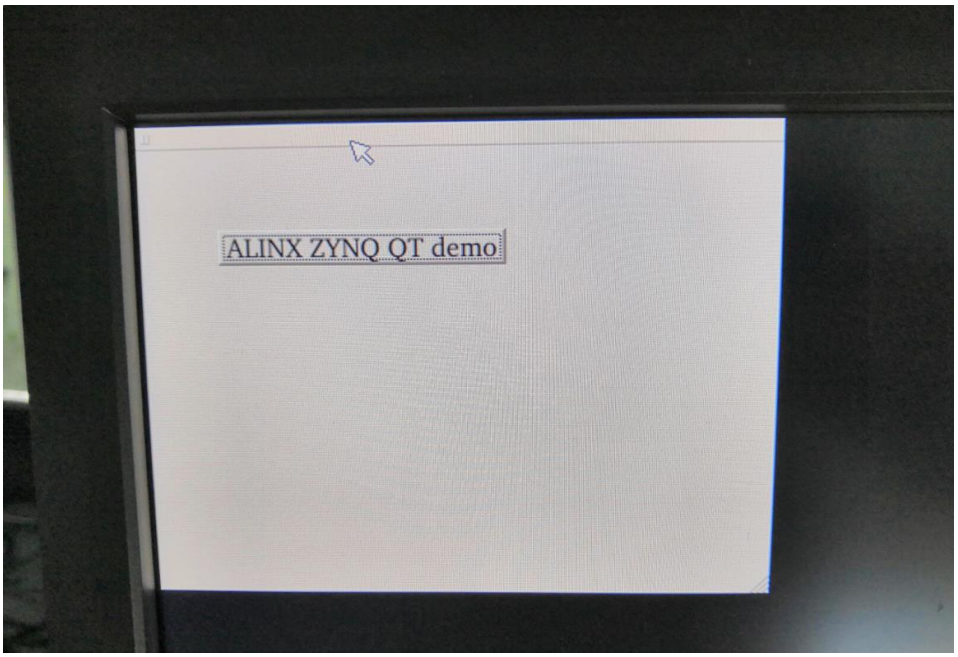
```
cd /mnt
cd build-qt_test-ZYNQ-Debug
```

10) 运行 ZYNQ 版本 QT 测试，需要注意的是从开发板系统启动到 QT 程序运行起来，时间不能超过 10 分钟，否则显示系统休眠，无法唤醒，QT 程序一旦运行，系统就不会再休眠。

```
./qt_test
```

```
root@an5642_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/qt
root@an5642_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@an5642_peta:/tmp/qt# cd /mnt
root@an5642_peta:/mnt# cd build-qt_test-ZYNQ-Debug/
root@an5642_peta:/mnt/build-qt_test-ZYNQ-Debug# ./qt_test
```

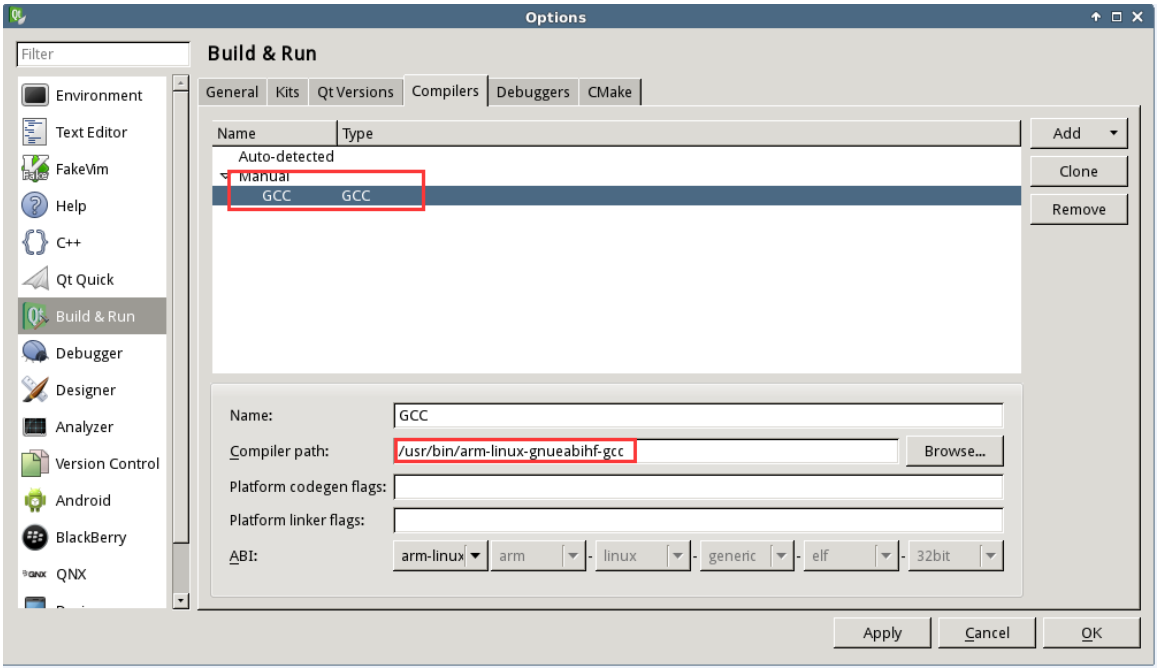
11) 运行效果，需要注意：这里的 QT 只能运行与非桌面环境，例如前面的 Debian 8，不能运行这里的 QT



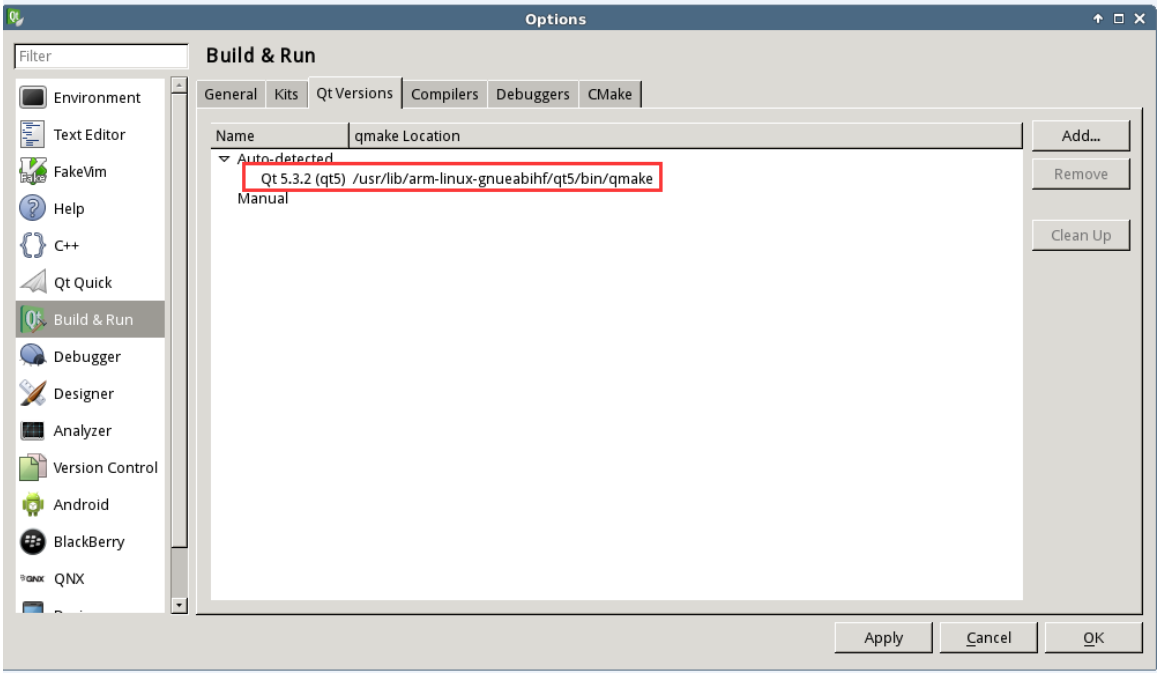
1.6 常见问题

1.6.1 如何使用开发板 Debian 系统自带的 QT Creator

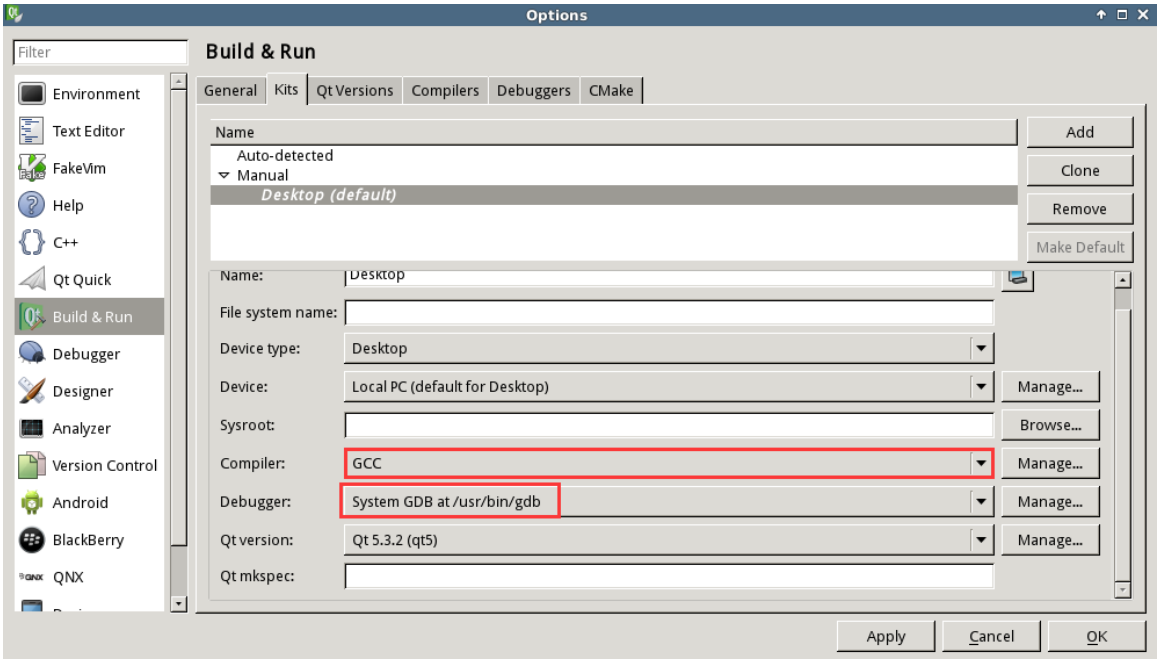
1) 配置编译器



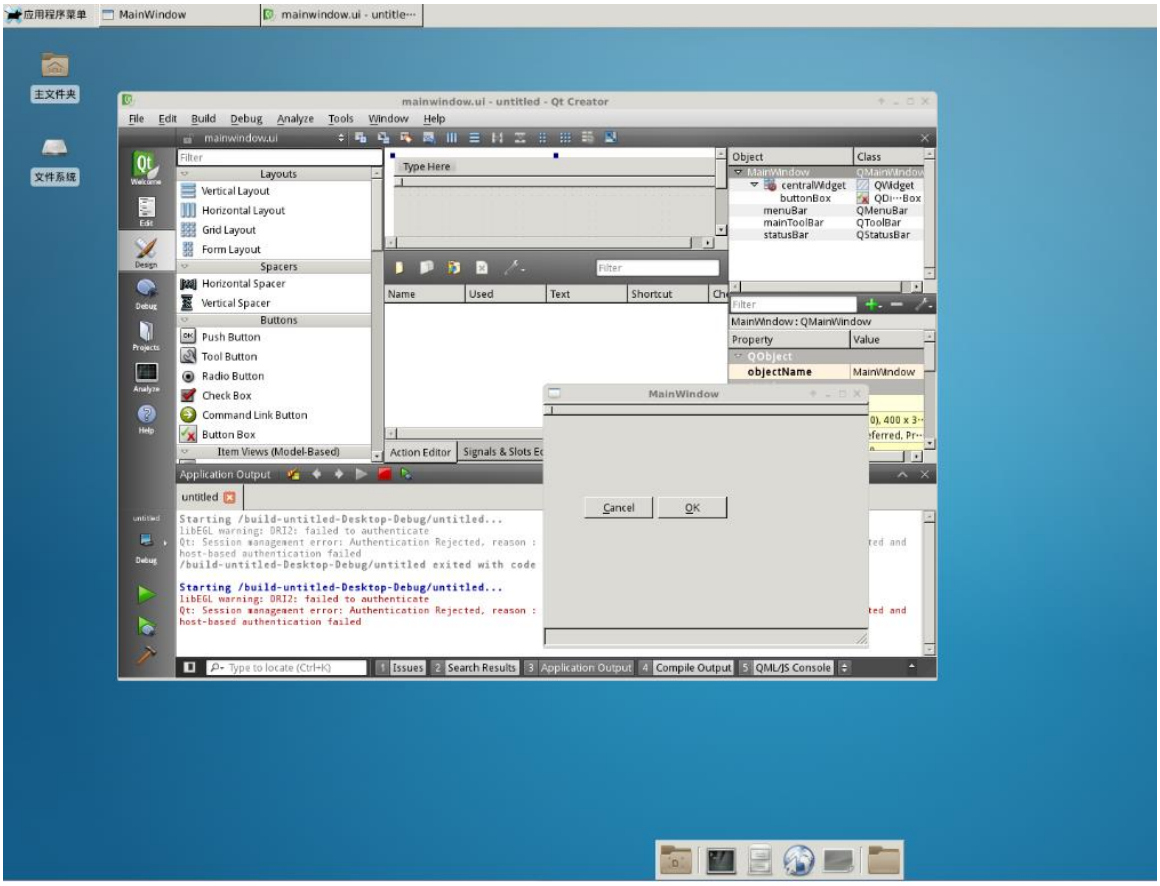
2) 查看 qmake 路径是否正确



3) 查看 Kits 设置是否和下图一致

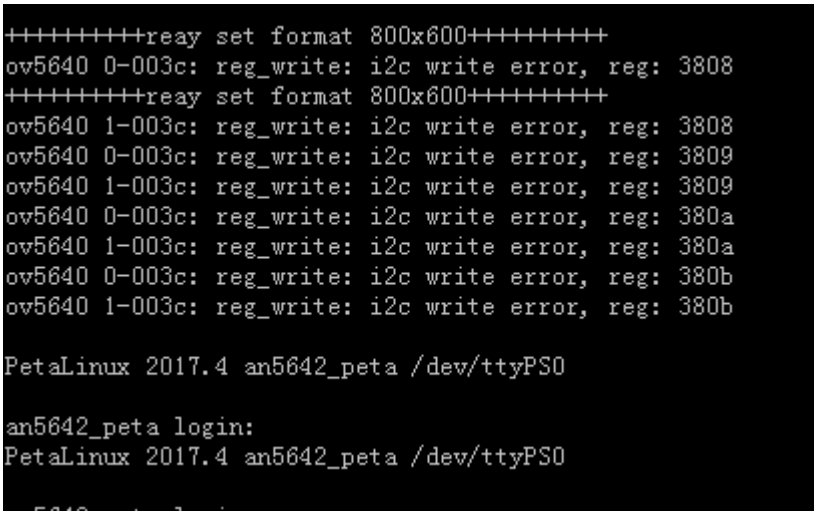


4) 配置完成后就可以建立一个工程测试一下



1.6.2 系统运行打印 I2C 错误信息

可以忽略这个错误信息


+++++++++reay set format 800x600+++++++++
ov5640 0-003c: reg_write: i2c write error, reg: 3808
+++++++++reay set format 800x600+++++++++
ov5640 1-003c: reg_write: i2c write error, reg: 3808
ov5640 0-003c: reg_write: i2c write error, reg: 3809
ov5640 1-003c: reg_write: i2c write error, reg: 3809
ov5640 0-003c: reg_write: i2c write error, reg: 380a
ov5640 1-003c: reg_write: i2c write error, reg: 380a
ov5640 0-003c: reg_write: i2c write error, reg: 380b
ov5640 1-003c: reg_write: i2c write error, reg: 380b
PetaLinux 2017.4 an5642_peta /dev/ttyPS0
an5642_peta login:
PetaLinux 2017.4 an5642_peta /dev/ttyPS0

第二章 OpenCV 应用（USB 摄像头显示）

例程相关文件夹： opencv_usb_camera

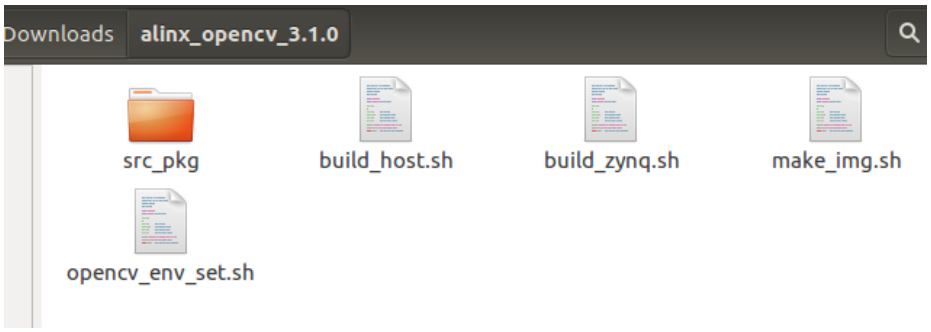
本章讲解如何编译 OpenCV 库，包括 Ubuntu 主机版和 ZYNQ 版。同样，芯驿电子为大家准备好了源代码包和编译脚本，从名称可以看出，OpenCV 版本为 3.1.0。



2.1 host 版 OpenCV 库编译

1) 解压源码包

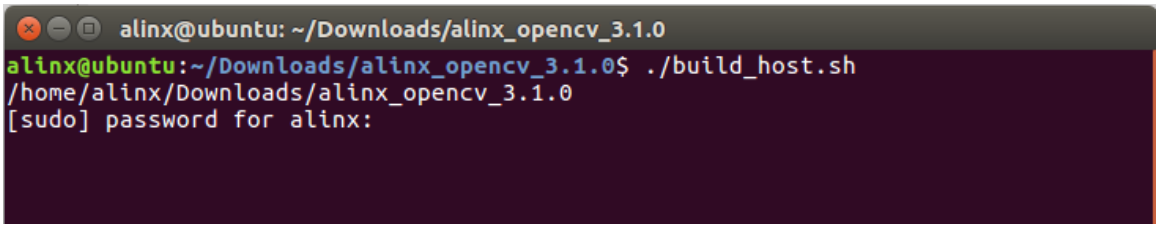
2) “src_pkg”里就是源码，“build_host.sh”是编译 Ubuntu 版本 OpenCV 的脚本，“build_zynq.sh”是编译 ZYNQ 版本的脚本，“make_img.sh” 是制作 ZYNQ 版本库镜像的脚本



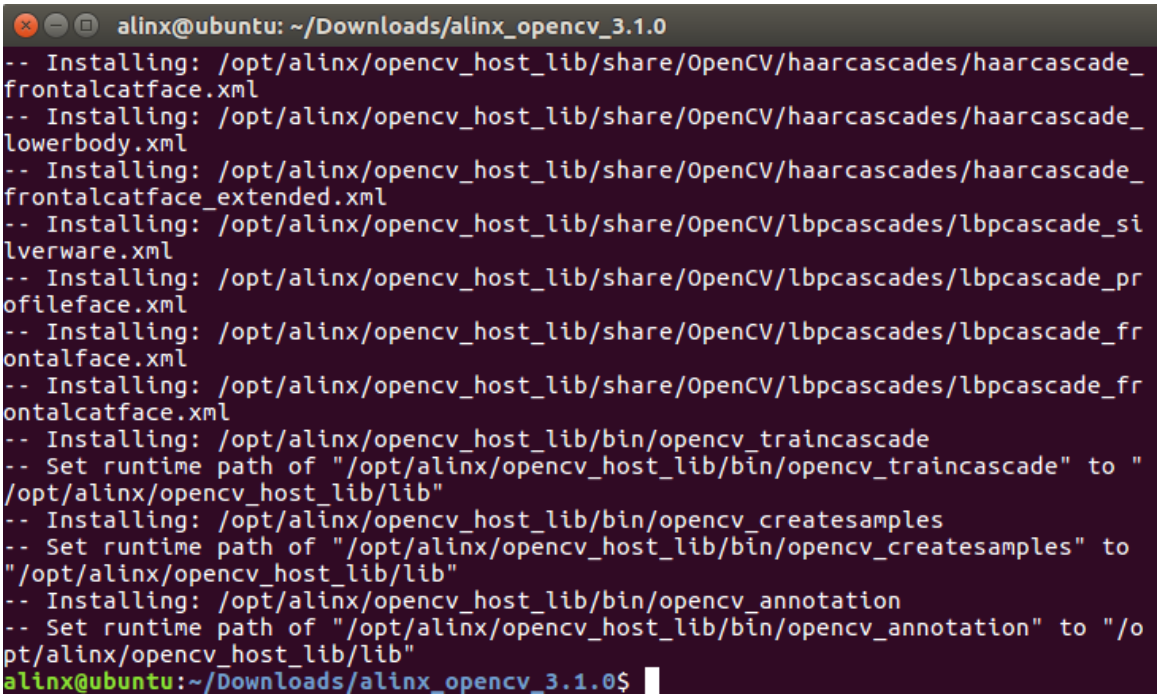
3) 安装 cmake

```
sudo apt-get install cmake
```

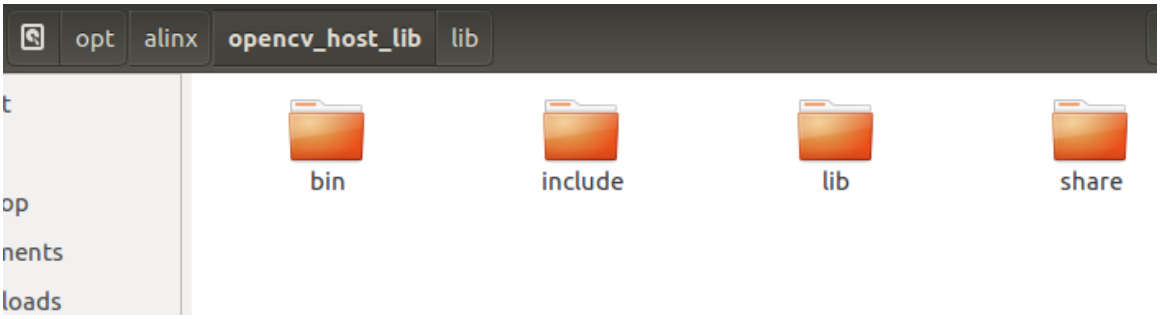
4) 在终端运行“build_host.sh”，会提示输入账号密码，输入密码，继续运行



5) 编译完成

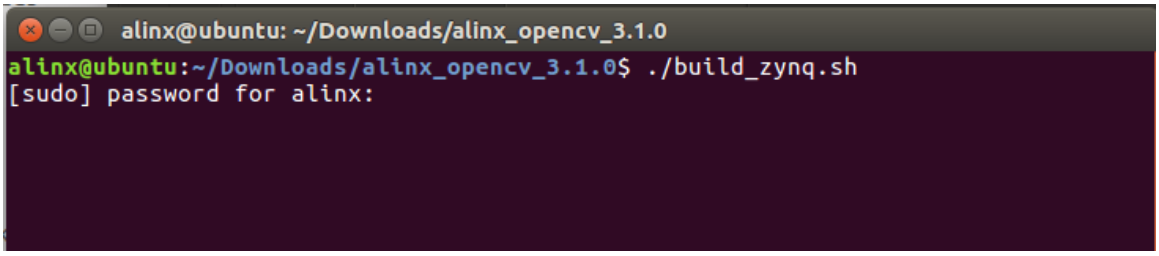


6) 在/opt/alinx/opencv_host_lib 目录下可以看到编译好的 OpenCV 库，“include”目录为头文件，“lib”为库。

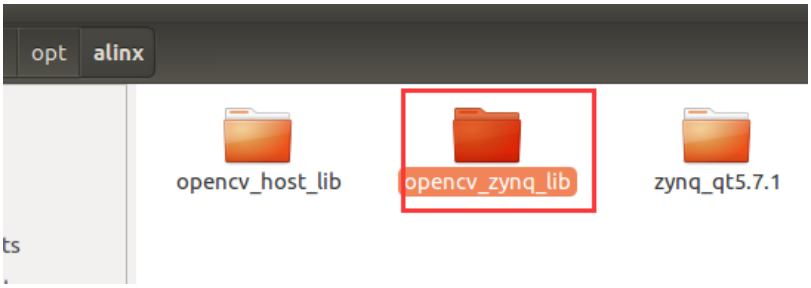


2.2 zynq 版 OpenCV 库编译

1) 和 host 版本编译类型，运行“build_zynq.sh”编译 ZYNQ 版本的 OpenCV 库，需要输入账号密码后就开始了漫长的编译等待时间



2) /opt/alinx/opencv_zynq_lib 目录就是编译出来的 ZYNQ 版本 OpenCV 库

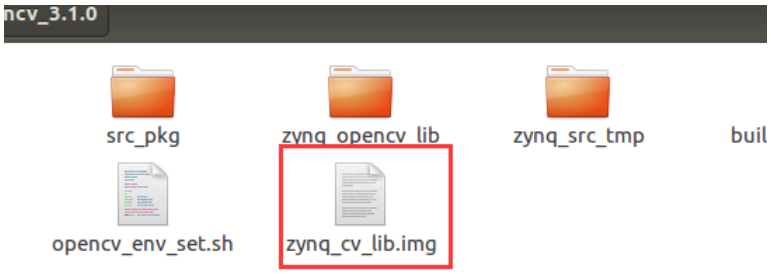


3) 运行 make_img.sh 制作 OpenCV 库的镜像，需要输入账号密码

```
alinx@ubuntu:~/Downloads/alinx_opencv_3.1.0$ ./make_img.sh
arm-linux-gnueabihf-strip: '/home/alinx/Downloads/alinx_opencv_3.1.0/zynq_opencv
_lib/v4l1compat.so': No such file
arm-linux-gnueabihf-strip: '/home/alinx/Downloads/alinx_opencv_3.1.0/zynq_opencv
_lib/v4l2convert.so': No such file
18+0 records in
18+0 records out
18874368 bytes (19 MB, 18 MiB) copied, 0.0178909 s, 1.1 GB/s
mke2fs 1.42.13 (17-May-2015)
Discarding device blocks: done
Creating filesystem with 18432 1k blocks and 4608 inodes
Filesystem UUID: b1899b6d-7828-4735-947a-0f7c1b1282bc
Superblock backups stored on blocks:
        8193

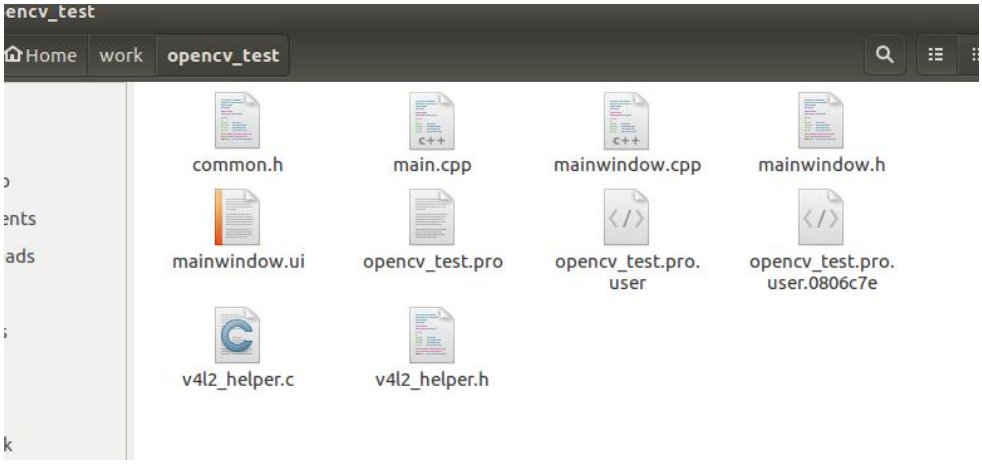
Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done
```

4) zynq_cv_lib.img 文件就是制作好的 OpenCV 库镜像

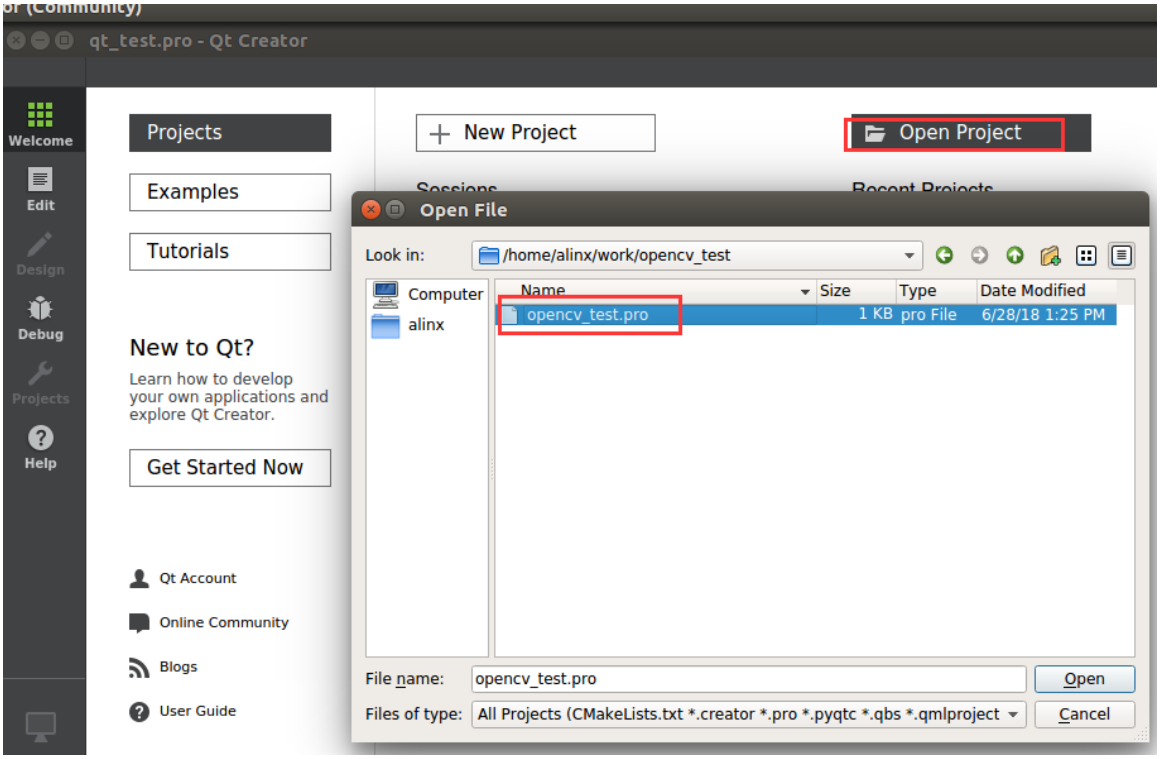


2.3 测试 host 版 OpenCV

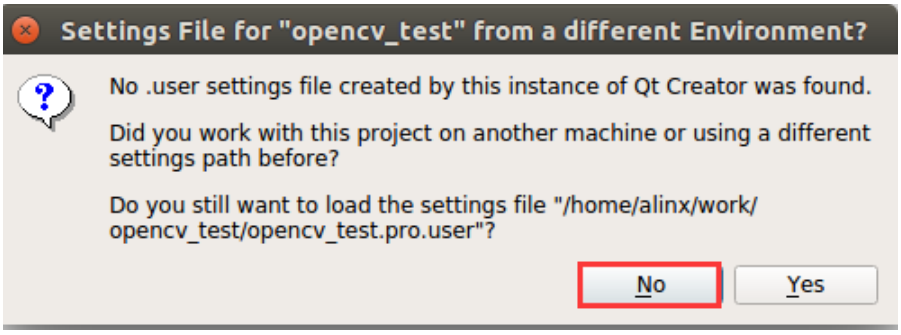
实验给出一个 QT 工程，工程中使用 OpenCV 进行 YUV 到 RGB 的颜色空间转换，其他更复杂的例子开发者可以自行开发，需要说明的是，在嵌入式上运行 OpenCV 和 Ubuntu 主机会有效差异。



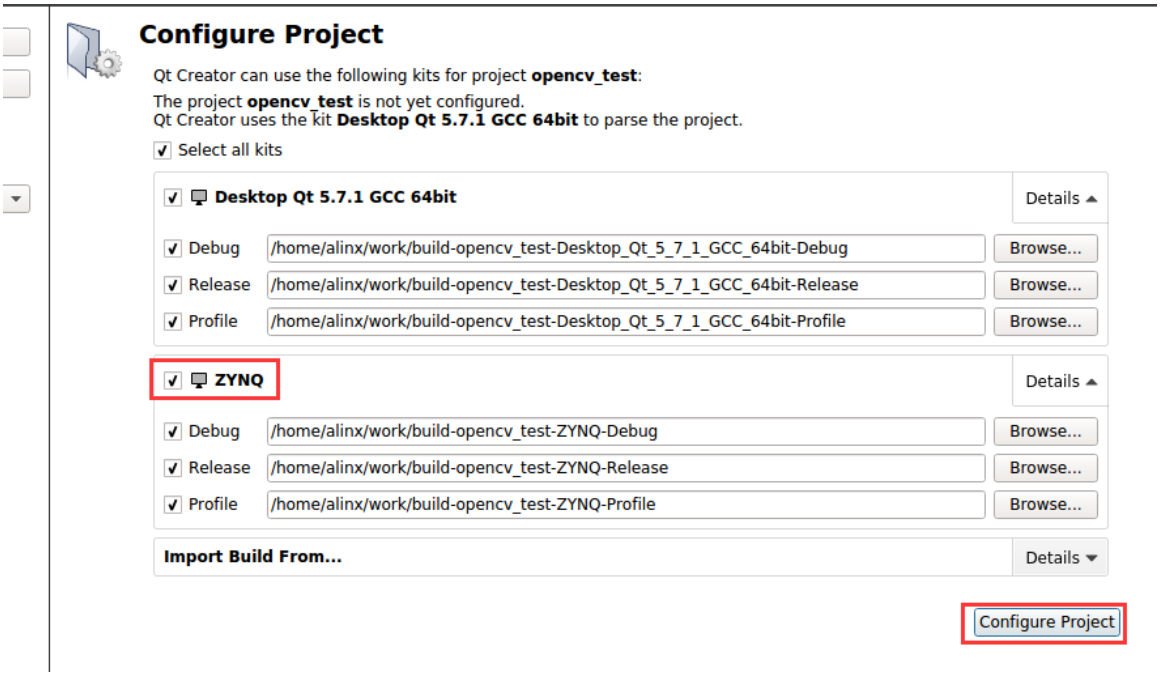
5) 运行 QT Creator，打开 opencv_test 工程



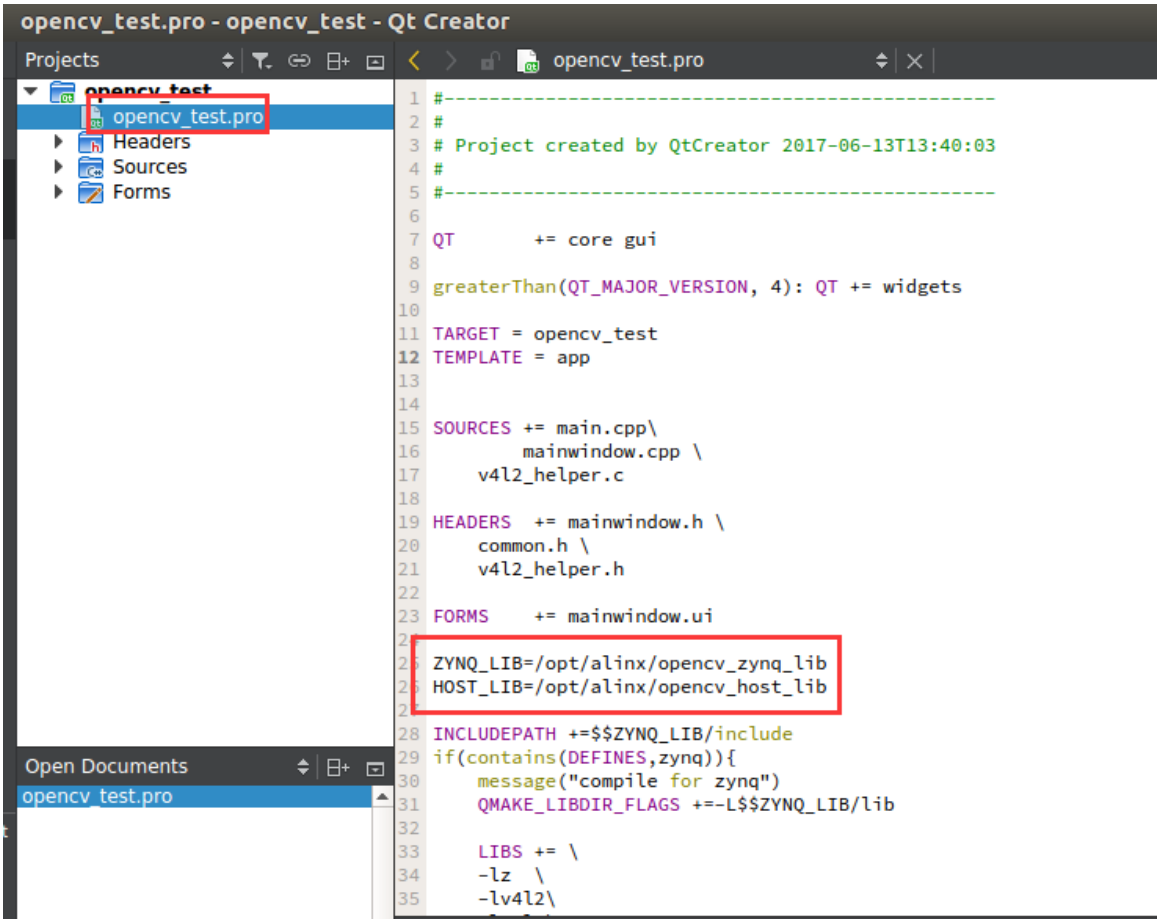
6) 如果出现下图对话框，选择“No”



7) 选中 ZYNQ kit，点击“Configure Project”

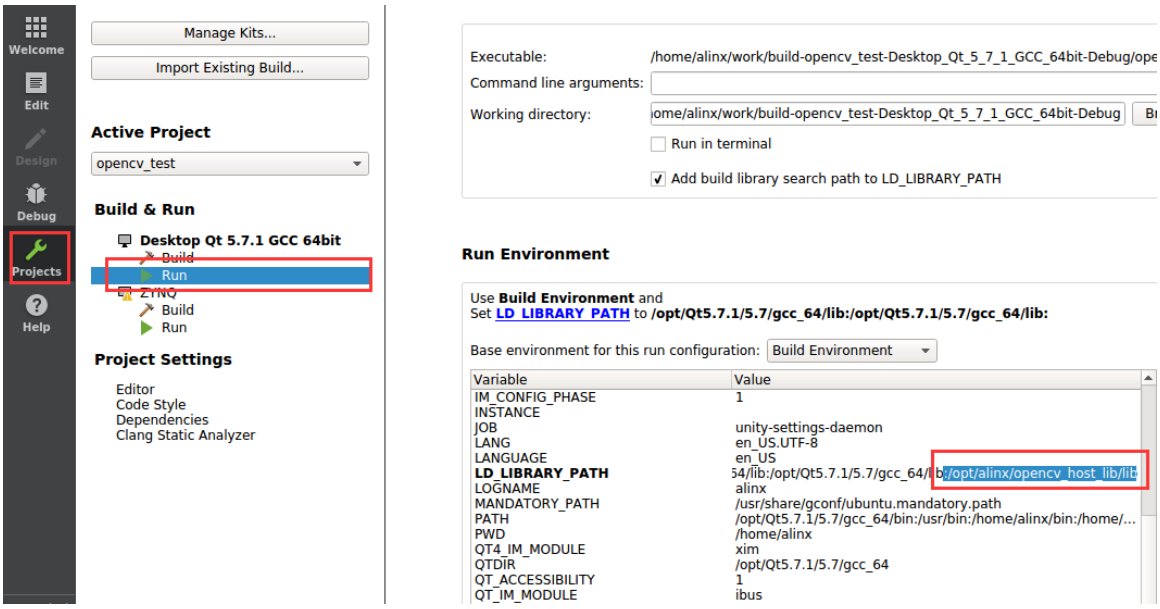


8) 双击“opencv_test.pro”文件可以看到设置了 ZYNQ 和 HOST 版本的 OpenCV 库的路径，根据实际情况修改，如果前面的库都是按默认配置编译这里不需要修改。

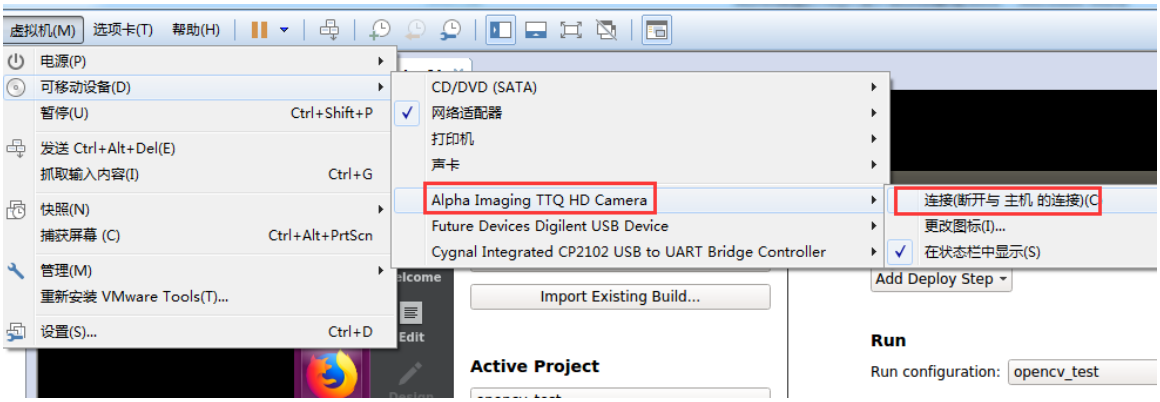


9) 设置 host 版运行环境变量，在 LD_LIBRARY_PATH 选项后面添加

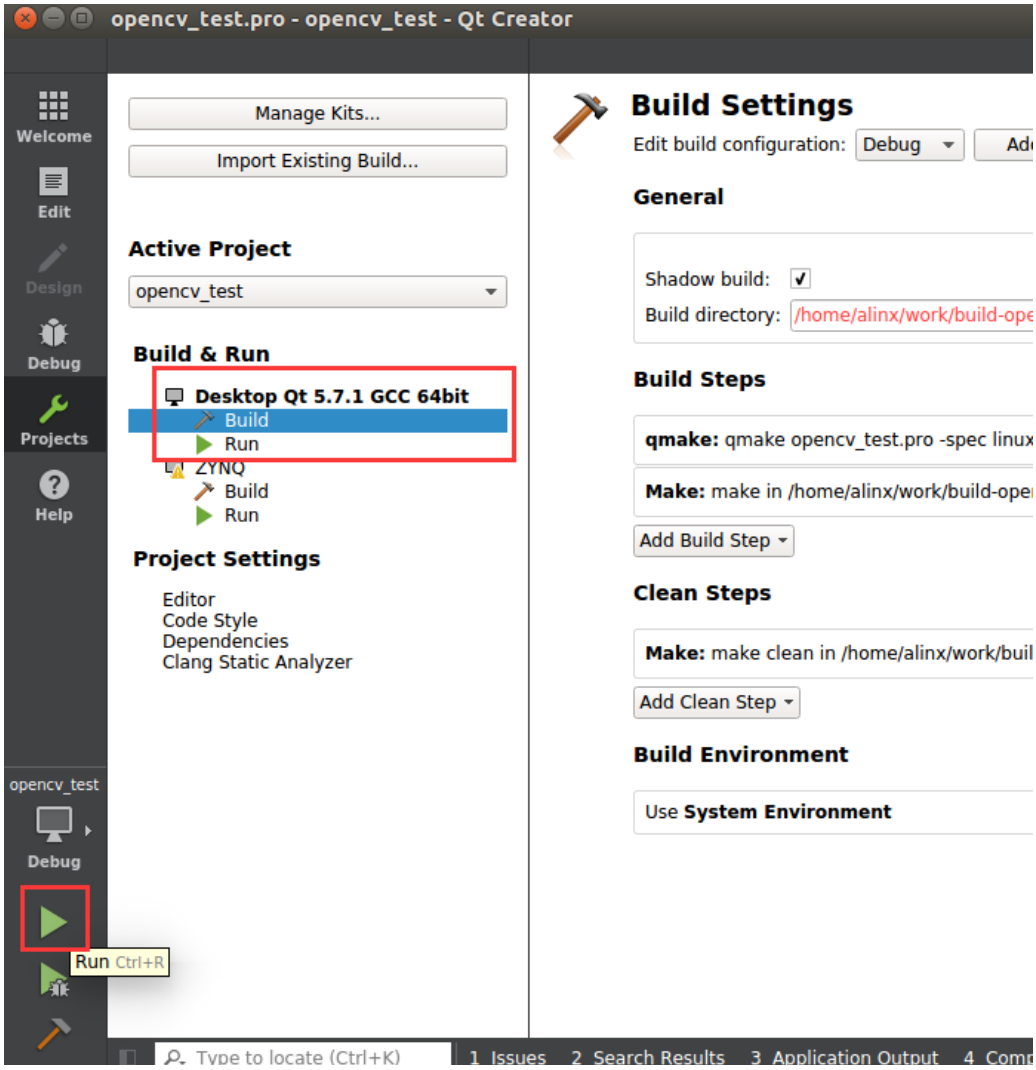
":/opt/alinx/opencv_host_lib/lib"



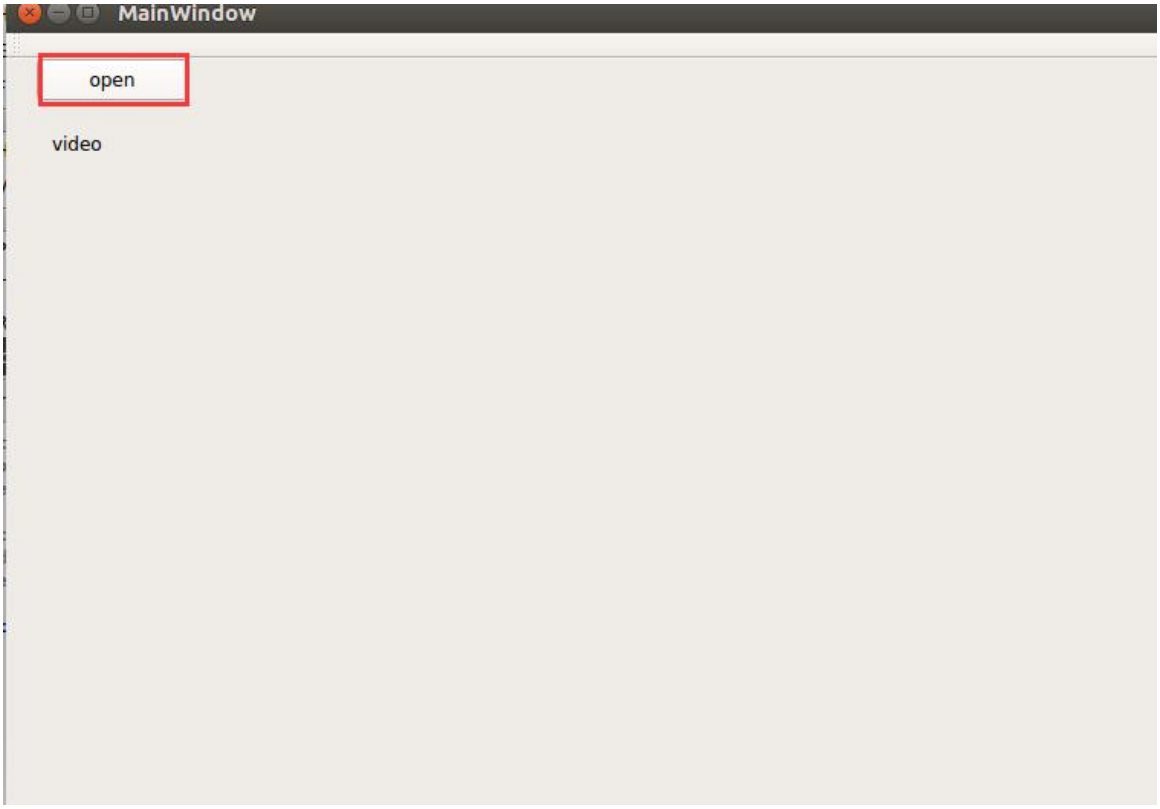
10) 连接 USB 摄像头到 Ubuntu 主机中



11) 运行 host 版本测试程序

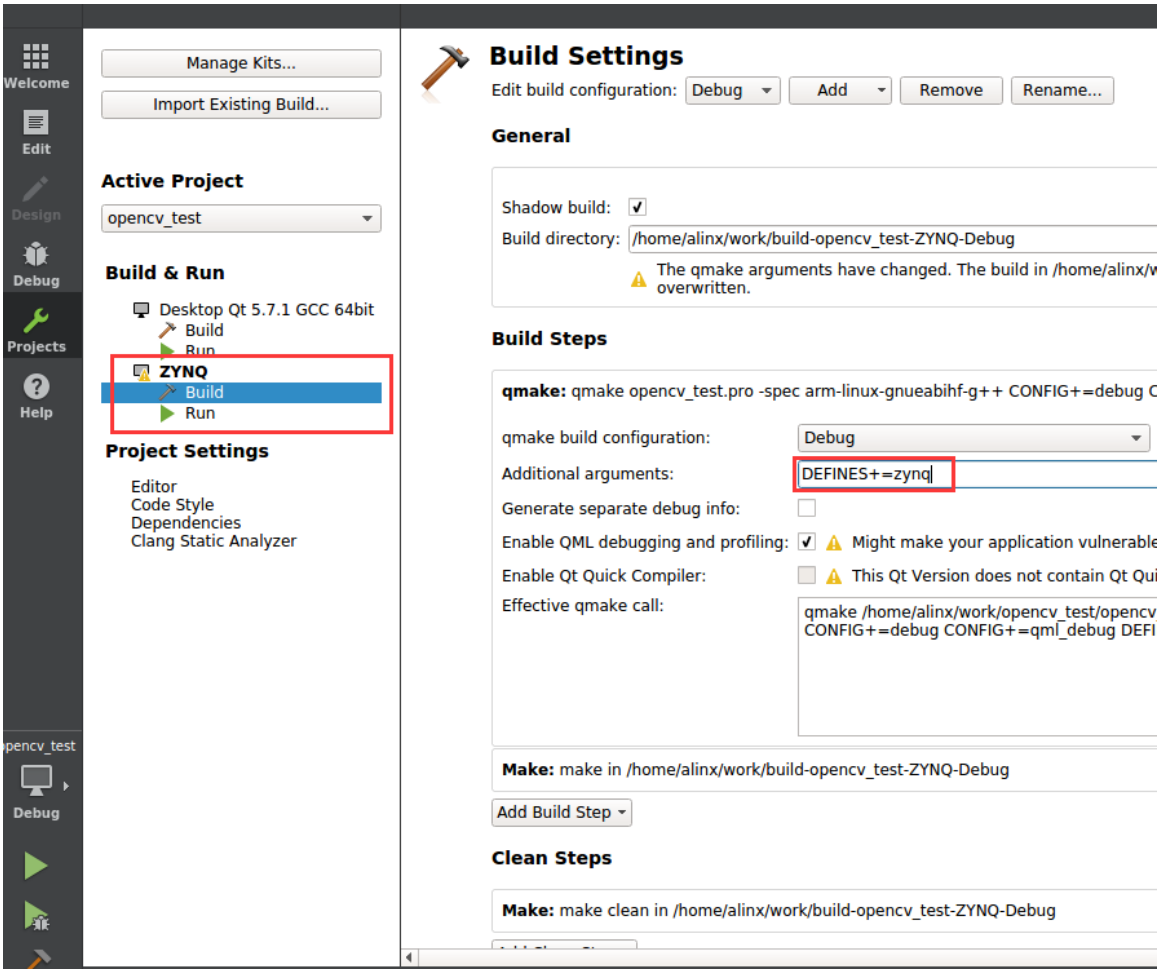


12) 点击 open 即可打开 USB 摄像头

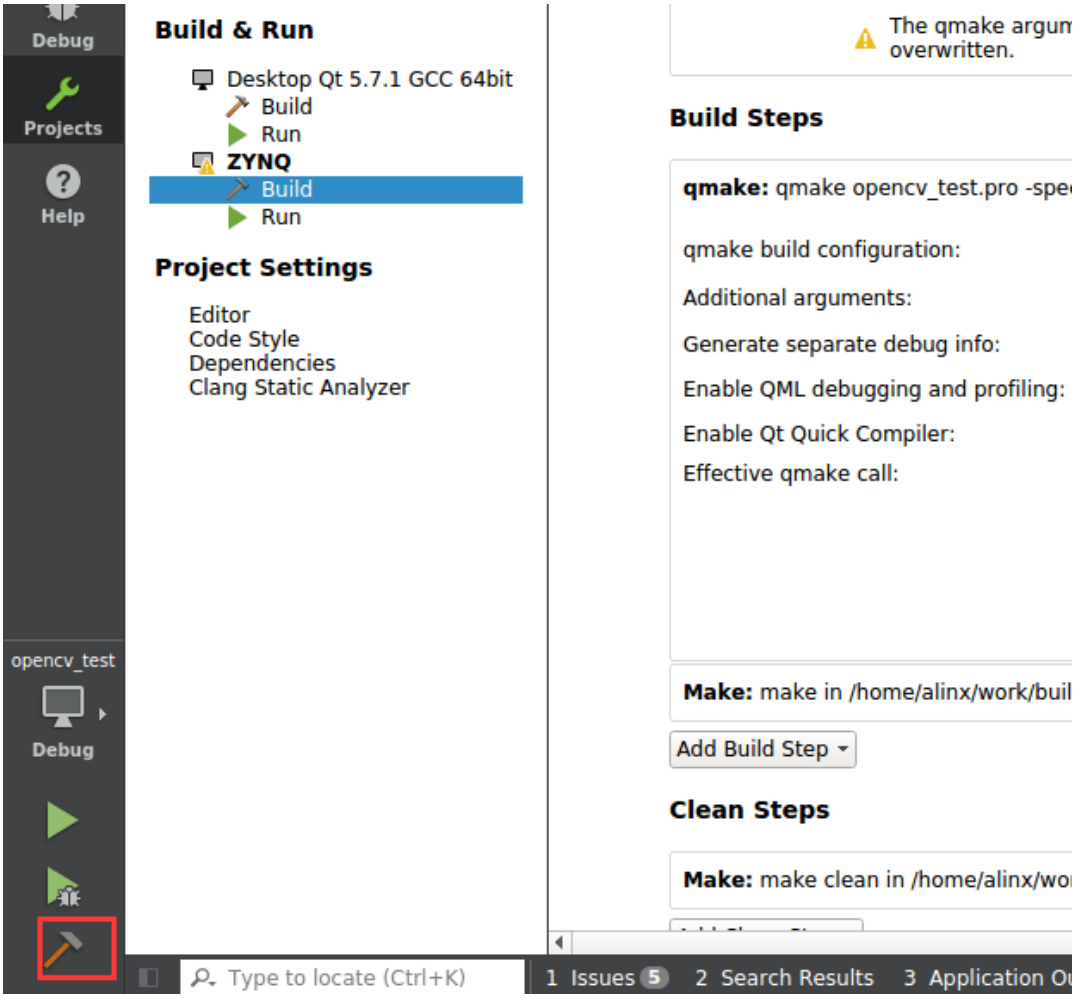


2.4 测试 zynq 版 OpenCV

1) 在 zynq 的 kit 设置里添加额外的编译选项 DEFINES+=zynq，定义一个 zynq 宏


Welcome
Edit
Design
Debug
Projects
Help
Manage Kits...
Import Existing Build...
Active Project
opencv_test
Build & Run
Desktop Qt 5.7.1 GCC 64bit
Build
Run
ZYNQ
Build
Run
Project Settings
Editor
Code Style
Dependencies
Clang Static Analyzer
Build Settings
Edit build configuration: Debug Add Remove Rename...
General
Shadow build:
Build directory: /home/alinx/work/build-opencv_test-ZYNQ-Debug
The qmake arguments have changed. The build in /home/alinx/w overwritten.
Build Steps
qmake: qmake opencv_test.pro -spec arm-linux-gnueabihf-g++ CONFIG+=debug C
qmake build configuration: Debug
Additional arguments: DEFINES+=zynq
Generate separate debug info:
Enable QML debugging and profiling: Might make your application vulnerable
Enable Qt Quick Compiler: This Qt Version does not contain Qt Qui
Effective qmake call: qmake /home/alinx/work/opencv_test/opencv CONFIG+=debug CONFIG+=qml_debug DEFI
Make: make in /home/alinx/work/build-opencv_test-ZYNQ-Debug
Add Build Step
Clean Steps
Make: make clean in /home/alinx/work/build-opencv_test-ZYNQ-Debug
opencv_test
Debug

2) 点击锤子图标编译



3) 准备好开发板，连接好显示器，将例程给的 BOOT.BIN，image.ub 复制到 sd 卡（sd_boot 目录），建议先格式化 sd 卡，再复制。

4) 使用串口终端登录系统，默认账户 root，密码 root

```
IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
Sending discover...
Sending select for 192.168.1.46...
Lease of 192.168.1.46 obtained, lease time 86400
/etc/udhcpc.d/50default: Adding DNS 192.168.1.1
done.
Starting Dropbear SSH server: Generating key, this may take a while...
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDS6PnYncg3QkvRX9ykZpeL1B9GEmuj83s8PqHb+OZs
72dZpqajFznfrRtE1NP7agPaD646HACC4AsFYT34ds+eAf8tx4hxNRxn8ZdTmFhONdq/A960OZ1Uy2qV
Fingerprint: md5 b8:c8:a4:a4:70:9e:ff:e4:00:15:41:d8:c2:21:f3:2d
dropbear.
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2017.4 an5642_peta /dev/ttyPS0

an5642_peta login: root
Password:
root@an5642_peta:~#
```

5) 挂载 NFS，设置 QT 运行环境

```
mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
```

```
root@an5642_peta:~# mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
root@an5642_peta:~# cd /mnt
root@an5642_peta:/mnt# mkdir /tmp/qt
root@an5642_peta:/mnt# mount qt_lib.img /tmp/qt
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/qt
root@an5642_peta:/tmp/qt# source ./qt_env_set.sh
```

6) 复制 zynq_cv_lib.img 到/home/alinx/work 文件夹，设置 OpenCV 库环境

```
mkdir /tmp/cv
cd /mnt
mount zynq_cv_lib.img /tmp/cv
cd /tmp/cv
source ./opencv_env_set.sh
```

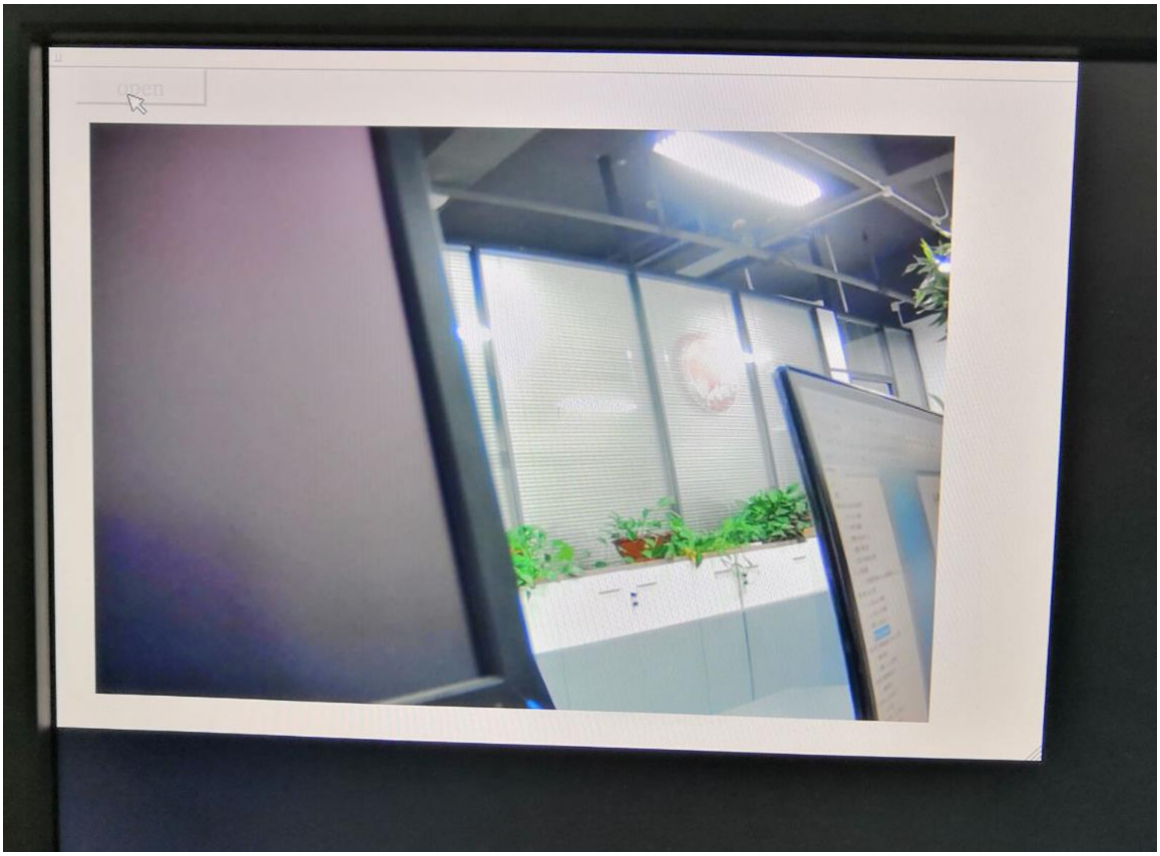
```
root@an5642_peta:/tmp/qt# mkdir /tmp/cv
root@an5642_peta:/tmp/qt# cd /mnt
root@an5642_peta:/mnt# mount zynq_cv_lib.img /tmp/cv
EXT4-fs (loop1): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/cv
root@an5642_peta:/tmp/cv# source ./opencv_env_set.sh
/tmp/cv
root@an5642_peta:/tmp/cv#
```

7) 运行测试

```
cd /mnt/build-opencv_test-ZYNQ-Debug/
./opencv_test
```

```
root@an5642_peta:/tmp/cv# cd /mnt/build-opencv_test-ZYNQ-Debug/
root@an5642_peta:/mnt/build-opencv_test-ZYNQ-Debug# ./opencv_test random: crng init done
```

8) 插入 USB 摄像头，点击 open，测试结果如下



第三章 双目摄像头模块 OpenCV 显示

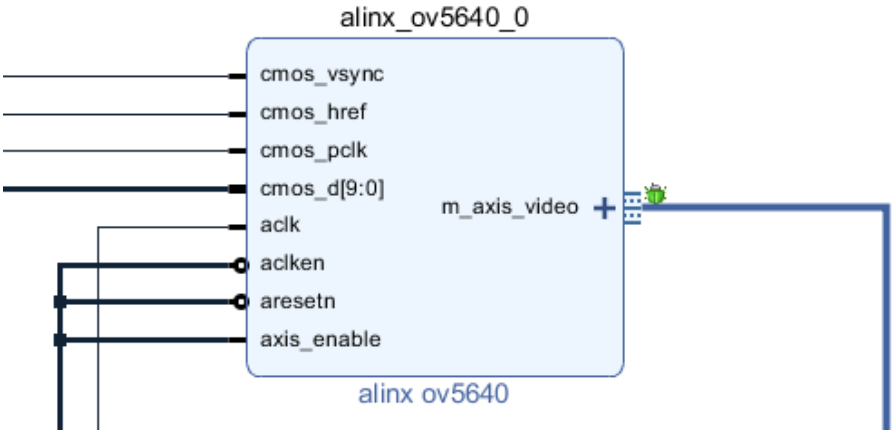
例程相关文件夹：opencv_an5642

前面的教程中讲到如何使用 OpenCV 和 QT 显示 USB 摄像头的应用，本章讲解如何显示芯驿电子配套模块双目摄像头（AN5642）的应用。

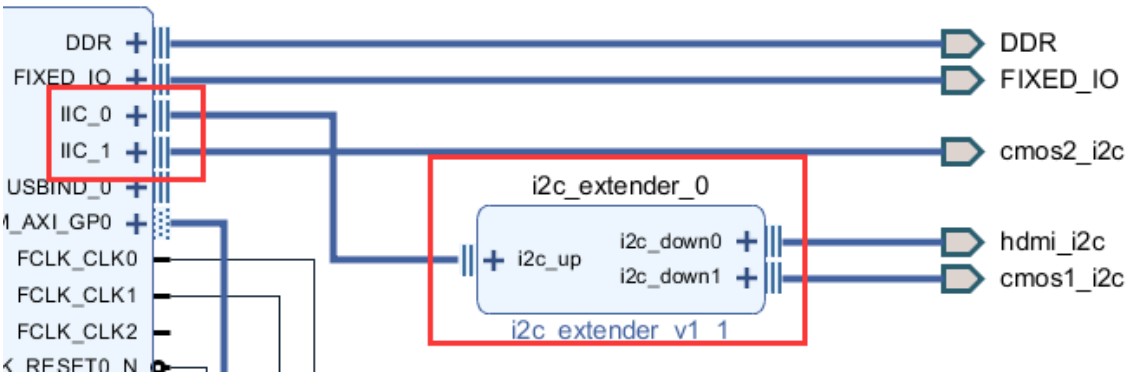
例程资料中给出了 vivado 工程，petalinux 工程。

3.1 vivado 工程简介

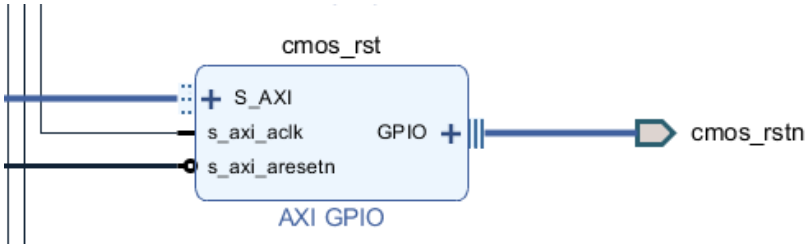
1) 摄像头输入转换为 AXI Stream 模块，将摄像头输入数据转换为 16bit 的 AXI Stream，然后通过 VDMA 写入 PS 端 DDR3。



2) IIC 处理，由于双目 OV5640 的 IIC 地址完全相同，所以这里使能 2 路 IIC，都是通过 EMIO 引出到 PL 端，其中一个通过 IIC 扩展模块扩展出多路。

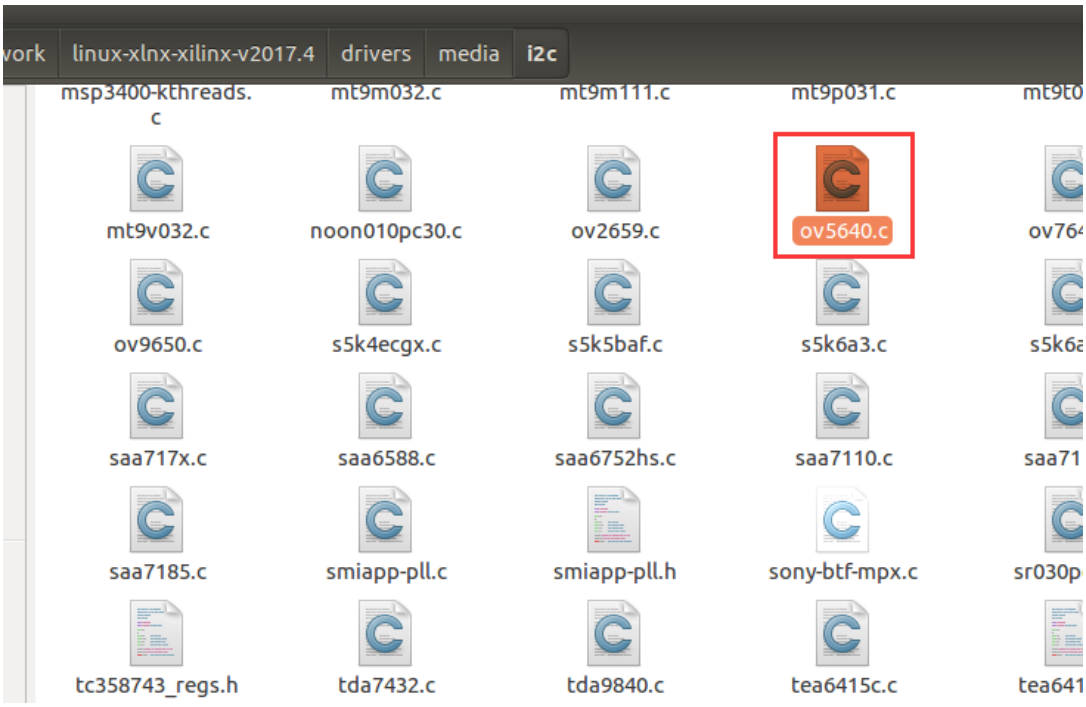


3) 通过 AXI GPIO 复位 2 路摄像头



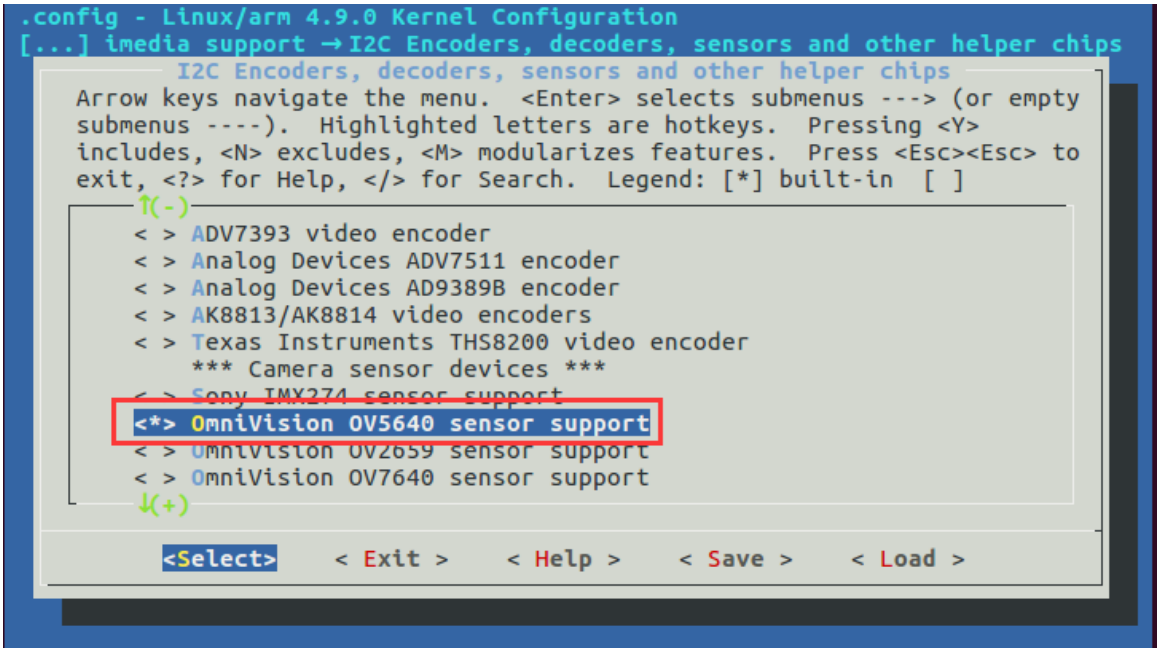
3.2 摄像头驱动

芯驿电子为 OV5640 写了简单的驱动，由于 Linux 驱动编写涉及内容较多，大部分驱动编写都是通过改写内核中已有驱动，这个驱动也是一样。驱动主要配置 OV5640 输出 YUV422 数据，分辨率根据应用程序来修改，这里简单介绍一下。

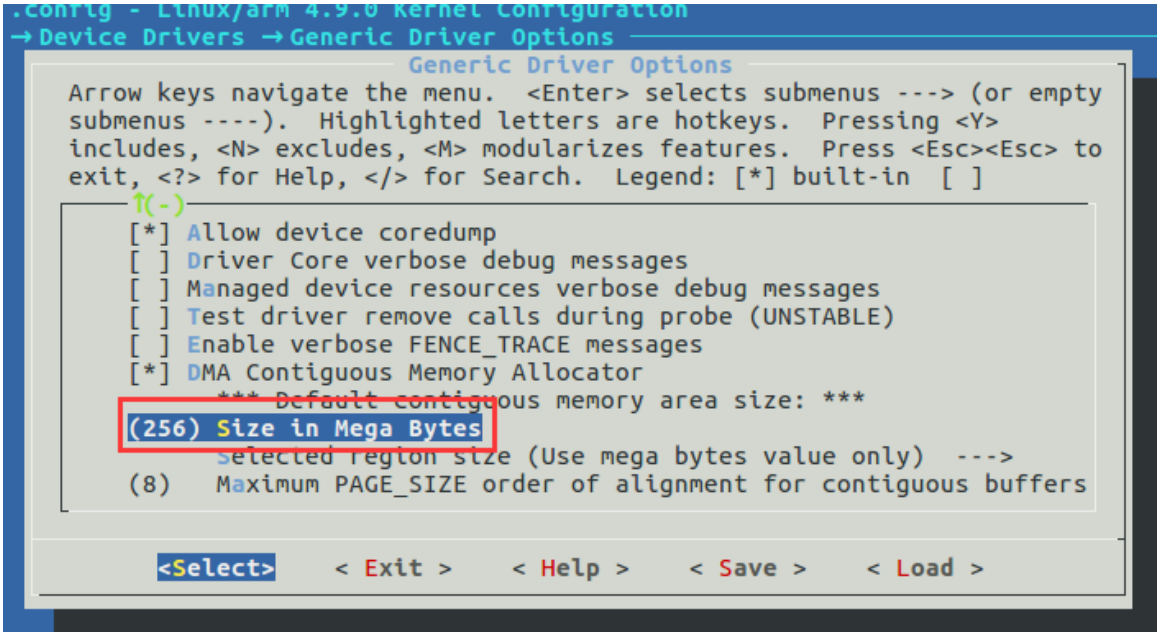


1) 驱动不做过多解释，本身就是 Linux 开发中比较难的点，请直接使用芯驿电子提供的 Linux 内核源码包。

2) 驱动配置，Device Drivers → Multimedia support→ I2C Encoders, decoders, sensors and other helper chips → <*> OmniVision OV5640 sensor support，HDMI 等显示驱动配置在前面的教程中已经讲解，不再复述。



3) CMA_SIZE_MBYTES 配置，由于视频路数增加，需要添加更多的 DMA 空间，配置 CMA_SIZE_MBYTES 大小为 256MB



3.3 设备树配置

1) 在 petalinux 工程 project-spec/meta-user/recipes-bsp/device-tree/files 目录下，找到 system-user.dtsi，这是我们要修改会添加的设备树，例程中已经修改好
2) 2 个 OV5640 分别挂载 i2c0 和 i2c1 总线下，地址为 0x3c（7bit 表示，是 8bit 表示地址的一半），详细设备树可以参考例程 petalinux 工程目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

```
&i2c0 {
        clock-frequency = <100000>;
        sensor@3c{
                compatible = "alinx,ov5640";
                reg = <0x3c>;
                #address-cells = <1>;
                #size-cells = <0>;
                reset-gpios = <&cmos_rst 0 0>;
                port@0 {
                        reg = <0>;
                        xlnx,video-format = <XVIP_VF_YUV_422>;
                        xlnx,video-width = <16>;
                        ov5640_out_0: endpoint {
                                remote-endpoint = <&vcap_ov5640_in_0>;
                        };
                };
        };
};

&i2c1 {
        clock-frequency = <100000>;
        sensor@3c{
                compatible = "alinx,ov5640";
                reg = <0x3c>;
                #address-cells = <1>;
                #size-cells = <0>;
                reset-gpios = <&cmos_rst 1 0>;
                port@0 {
                        reg = <0>;
                        xlnx,video-format = <XVIP_VF_YUV_422>;
                        xlnx,video-width = <16>;
                        ov5640_out_1: endpoint {
                                remote-endpoint = <&vcap_ov5640_in_1>;
                        };
                };
        };
};
```

3) vcap_ov5640 设备为 Xilinx 提供 VDMA 驱动封装起来，做成 V4L2 接口

```
vcap_ov5640_0 {
        compatible = "xlnx,video";
        dmas = <&axi_vdma_1 0>;
        dma-names = "port0";

        ports {
                #address-cells = <1>;
                #size-cells = <0>;

                port@0 {
                        reg = <0>;
                        direction = "input";
                        vcap_ov5640_in_0: endpoint {
                                remote-endpoint = <&ov5640_out_0>;
                        };

                };
        };
};

vcap_ov5640_1 {
        compatible = "xlnx,video";
        dmas = <&axi_vdma_2 0>;
        dma-names = "port0";

        ports {
                #address-cells = <1>;
                #size-cells = <0>;

                port@0 {
                        reg = <0>;
                        direction = "input";
                        vcap_ov5640_in_1: endpoint {
                                remote-endpoint = <&ov5640_out_1>;
                        };

                };
        };
};
```

3.4 摄像头 Linux 应用程序

1) 应用程序使用 QT 编写，结合 OpenCV 做颜色空间转换，在前面的教程中已经将过，不再复述
2) 需要注意的是，elf 文件生成位置，和前面教程不同，这次在 app 目录里
3) 运行例程 Linux，可以看到双目摄像头相关驱动打印信息。

```
hub 1-0:1.0: 1 port detected
mousedev: PS/2 mouse device common for all mice
i2c /dev entries driver
cdns-i2c e0004000.i2c: 100 kHz mmio e0004000 irq 23
cdns-i2c e0005000.i2c: 100 kHz mmio e0005000 irq 24
random: fast init done
ov5640 0-003c: Chip ID 0x5640 detected
ov5640 1-003c: Chip ID 0x5640 detected
xilinx-video amba_pl:vcap_ov5640_0: device registered
xilinx-video amba_pl:vcap_ov5640_1: device registered
uvcvideo: Unable to create debugfs directory
usbcore: registered new interface driver uvcvideo
USB Video Class driver (1.1.1)
cdns-wdt f8005000.watchdog: Xilinx Watchdog Timer at f096e000 with timeout 10s
EDAC MC: ECC not enabled
Xilinx Zynq CpuIdle Driver started
sdhci: Secure Digital Host Controller Interface driver
sdhci: Copyright(c) Pierre Ossman
sdhci-pltfm: SDHCI platform and OF driver helper
usb 1-1: new high-speed USB device number 2 using ci_hdrc
mmc0: SDHCI controller on e0100000.sdhci [e0100000.sdhci] using ADMA
mmc0: new high speed SDHC card at address aaaa
mmcblk0: mmc0:aaaa SL08G 7.40 GiB
mmc1: SDHCI controller on e0101000.sdhci [e0101000.sdhci] using ADMA
 mmcblk0: p1 p2
ledtrig-cpu: registered to indicate activity on CPUs
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
NET: Registered protocol family 10
sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
NET: Registered protocol family 17
```

4) 挂载 NFS，设置 QT 和 OpenCV 运行环境

```
mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
mkdir /tmp/cv
cd /mnt
mount zynq_cv_lib.img /tmp/cv
cd /tmp/cv
source ./opencv_env_set.sh
```

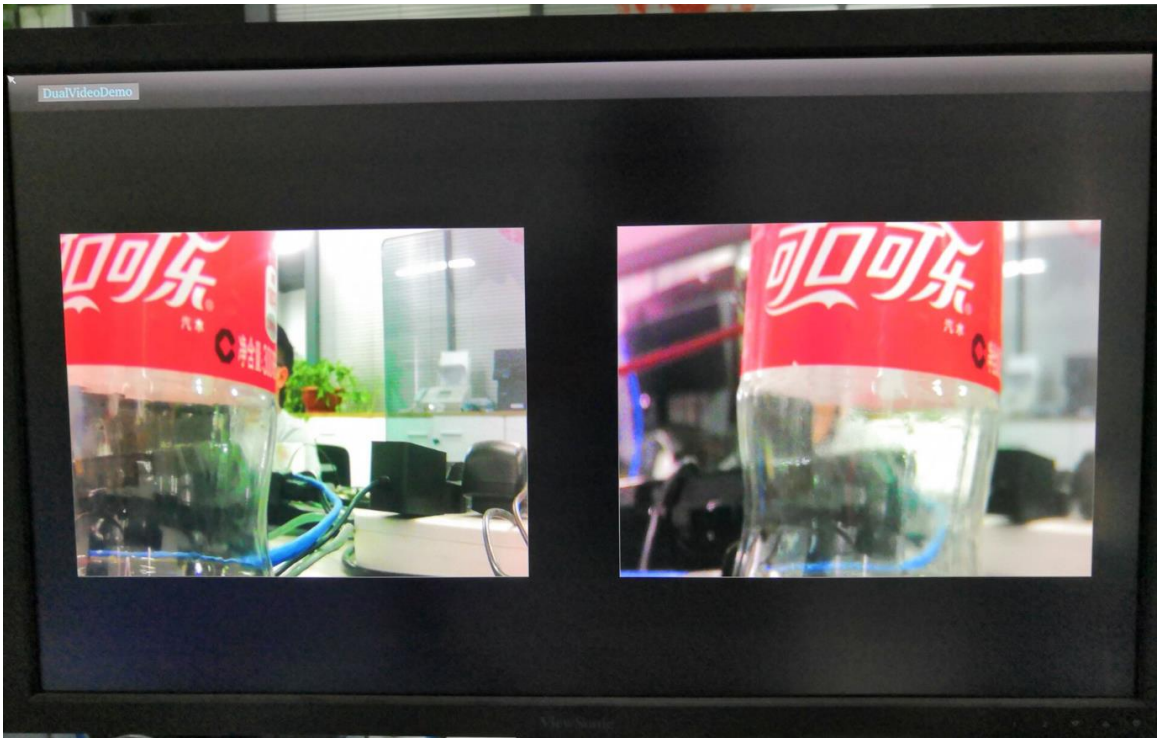
```
an5642_peta login: root
Password:
root@an5642_peta:~# mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
mount qt_lib.img /tmp/qt
cd /tmp/qt
root@an5642_peta:~# cd /mnt
root@an5642_peta:/mnt# mkdir /tmp/qt
cd /tmp/cv
root@an5642_peta:/mnt# mount qt_lib.img /tmp/qt
source ./opencv_env_set.sh
EXT4-fs (loop0): recovery complete
EXT4-fs (loop0): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/qt
root@an5642_peta:/tmp/qt# source ./qt_env_set.sh
/tmp/qt
root@an5642_peta:/tmp/qt# mkdir /tmp/cv
root@an5642_peta:/tmp/qt# cd /mnt
root@an5642_peta:/mnt# mount zynq_cv_lib.img /tmp/cv
EXT4-fs (loop1): recovery complete
EXT4-fs (loop1): mounted filesystem with ordered data mode. Opts: (null)
root@an5642_peta:/mnt# cd /tmp/cv
root@an5642_peta:/tmp/cv# source ./opencv_env_set.sh
/tmp/cv
root@an5642_peta:/tmp/cv#
```

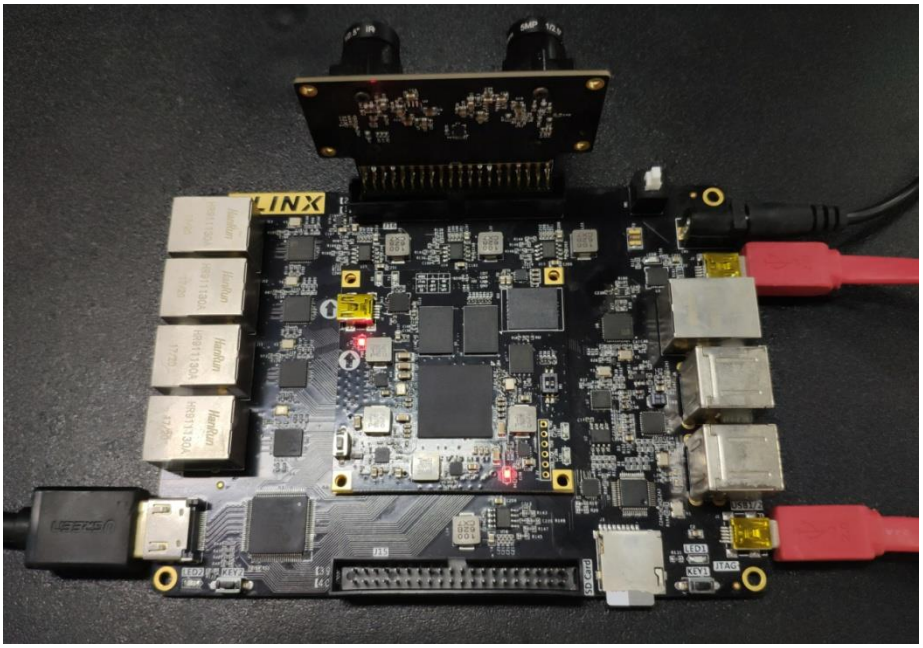
5) 运行测试程序

```
cd /mnt/app
./an5642_demo
```

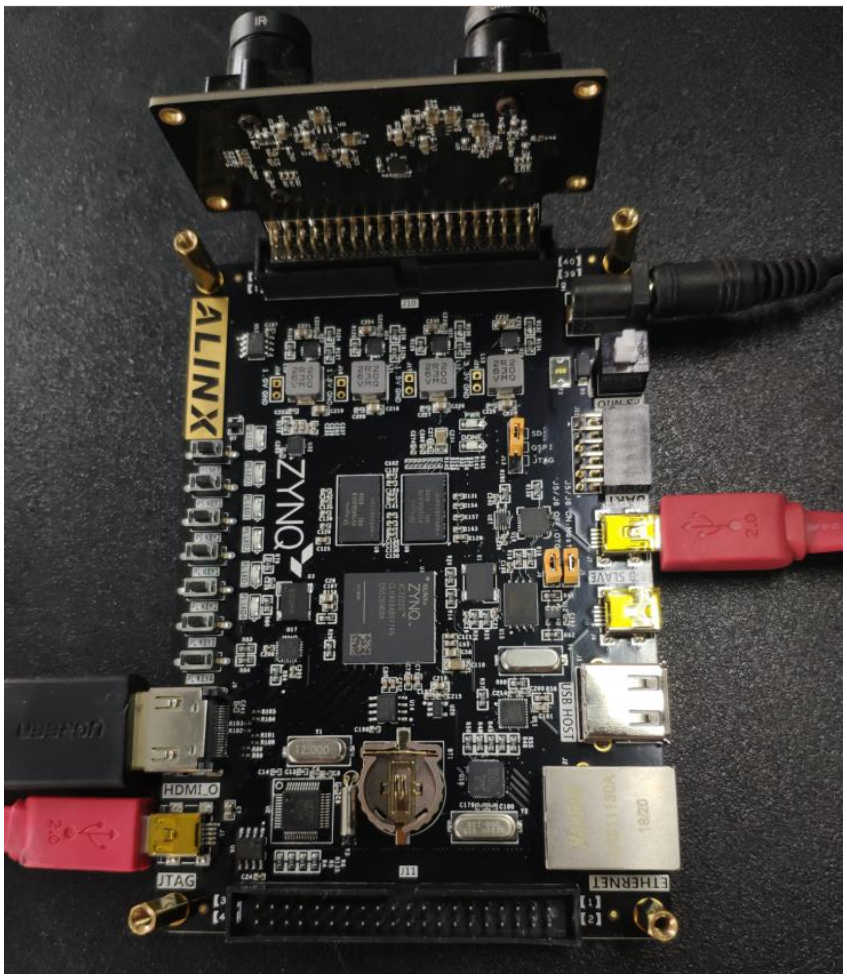
```
root@an5642_peta:/tmp/cv# cd /mnt/app
root@an5642_peta:/mnt/app# ./an5642_demo
```

6) 运行效果，资料包中给出了编译好的程序在 sd_boot 目录，所有文件复制到 sd 卡根目录运行即可。

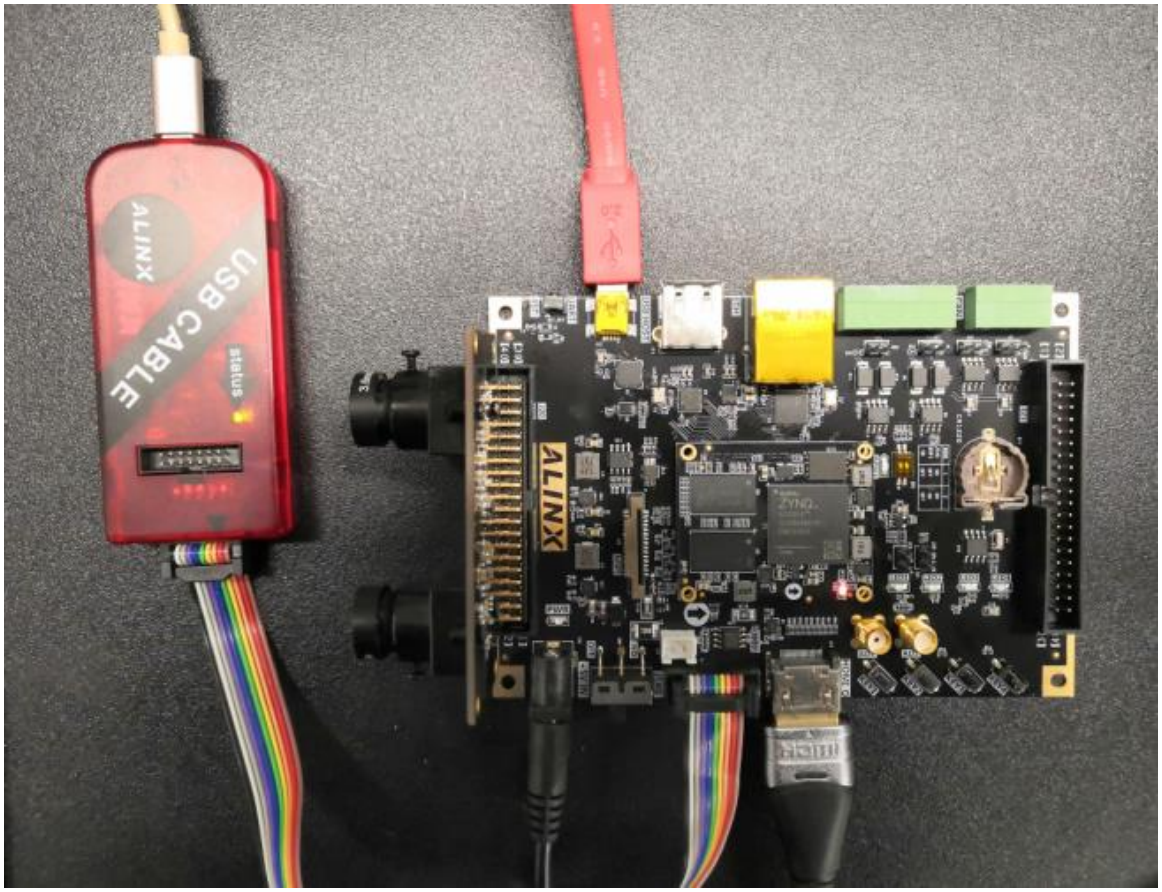




AX7021 硬件连接图（J16 扩展口）



AX7020/AX7010 硬件连接图(J10 扩展口)



AX7Z020/AX7Z010 硬件连接图（扩展口 J20）

第四章 触摸屏模块应用

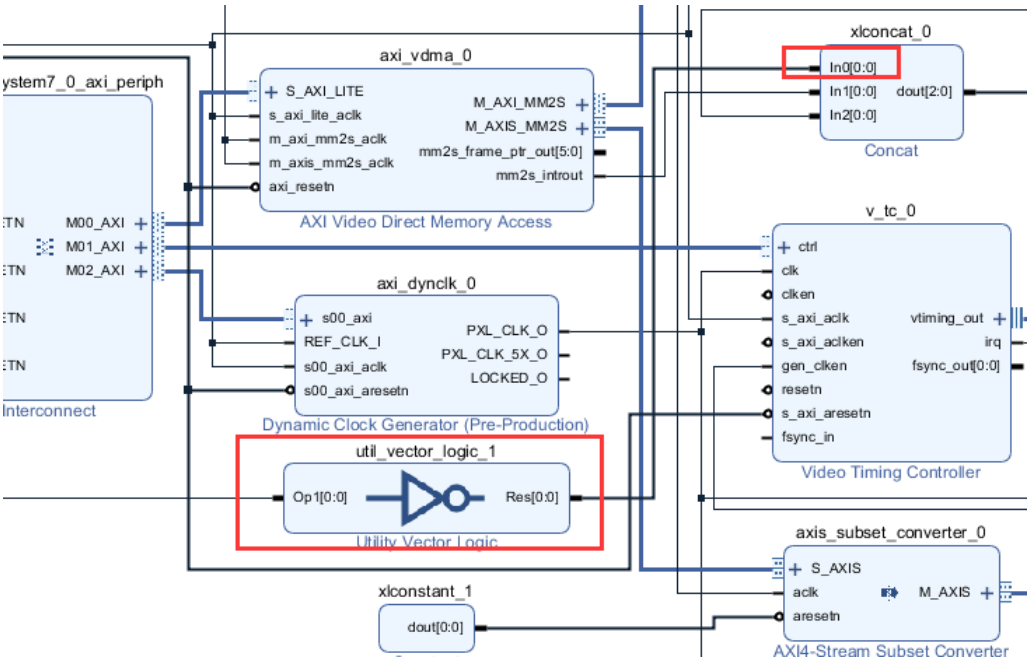
例程相关文件夹：lcd_touch

Vivado 工程在 vivado.zip 压缩包中：lcd_touch

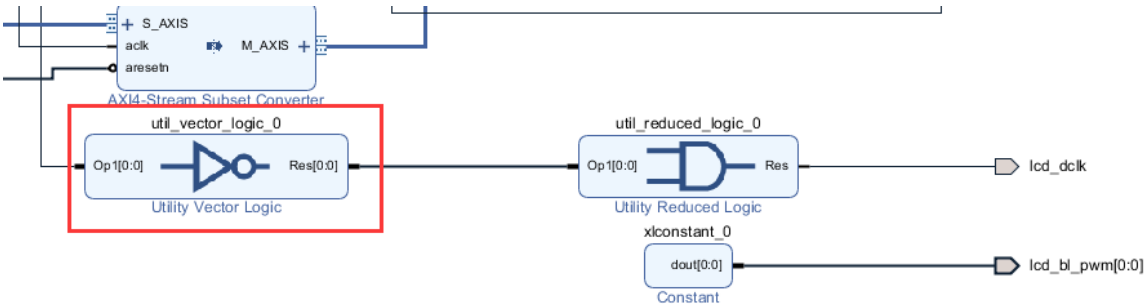
例程资料中给出了 vivado 工程，petalinux 工程。

4.1 vivado 工程简介

1) 触摸屏中断处理，由于触摸屏默认中断是低电平，而 PS 来自 PL 的中断只能是高电平或上升沿，这里加一个非门反相。中断连接到 PS 的第一个中断管脚，后面我们可以根据这些信息推断 Linux 下中断号，Linux 下中断编号可以参考 Xilinx 文档 UG585

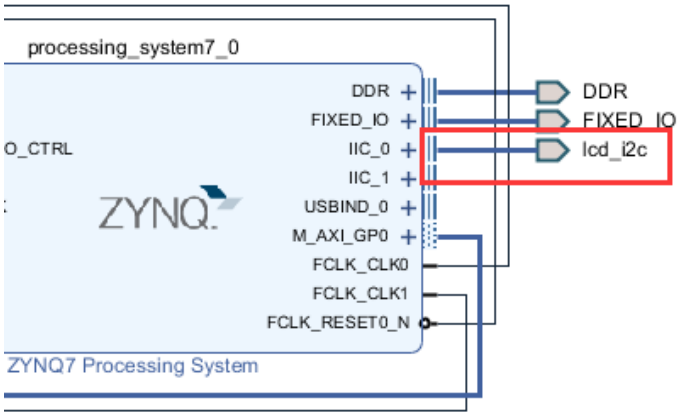


2) 液晶屏时钟处理，由于液晶屏是下降沿采样，这里使用非门把时钟反相输出，后面的与门没有什么作用，仅仅用于信号的连接。



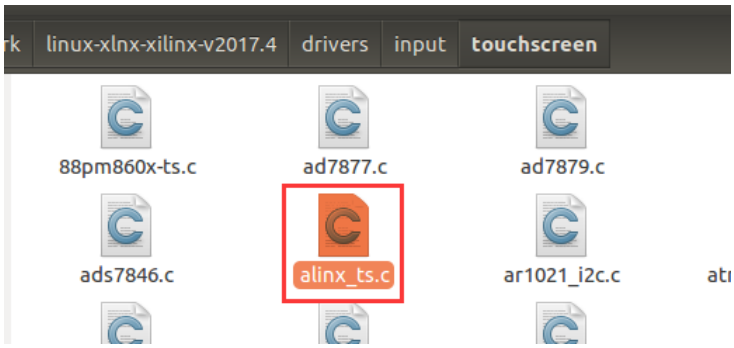
3) 液晶屏背光设置为一直高电平，就是最亮状态，可能会让液晶屏发热较多

4) IIC 用于读取液晶屏触摸信息

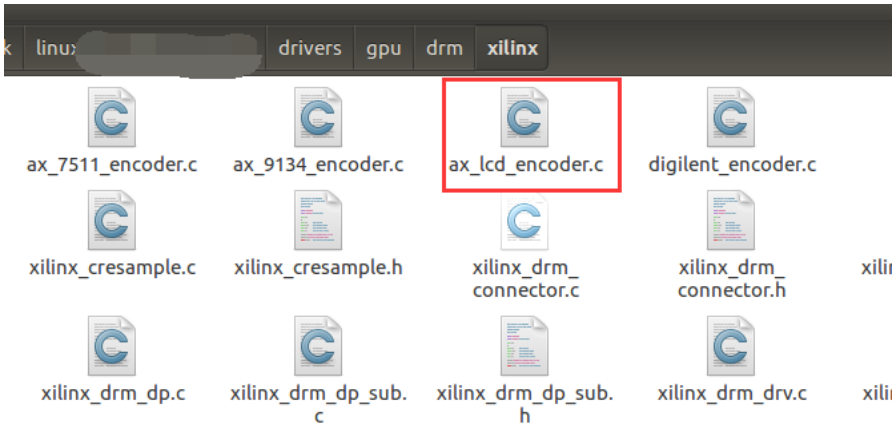


4.2 触摸屏驱动

1) alinx_ts.c 为芯驿电子提供的触摸屏驱动，位于内核目录 drivers/input/touchscreen 中

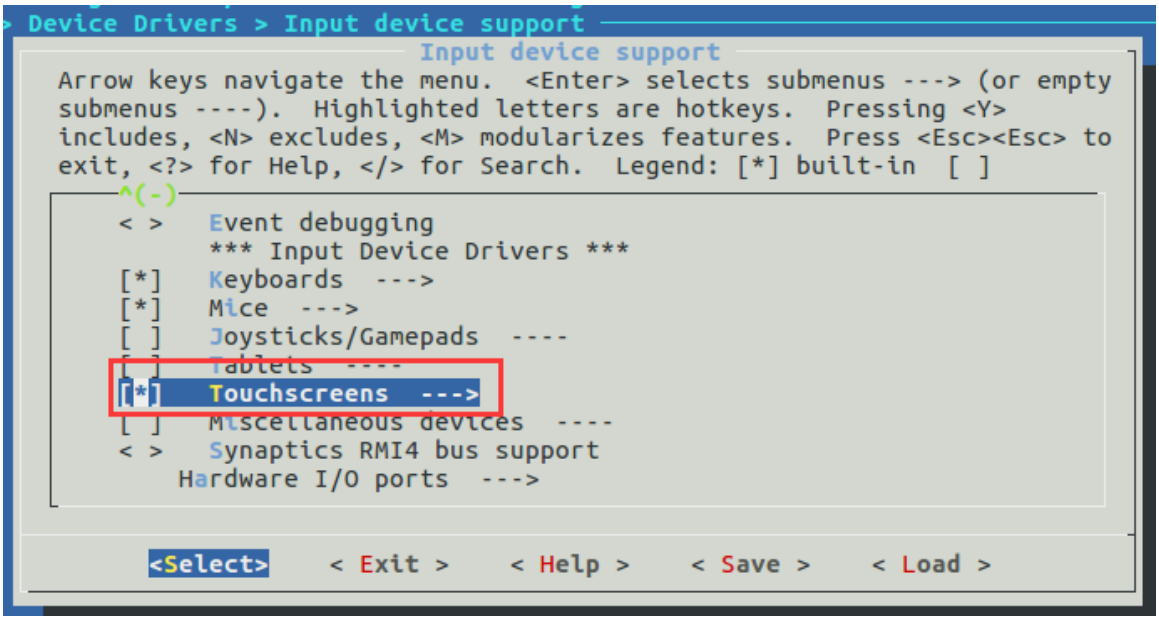


2) 触摸屏还要配套编码器才能工作，ax_lcd_encoder.c 是芯驿电子提供的 DRM 编码器驱动，位于内核 drivers/gpu/drm/xilinx

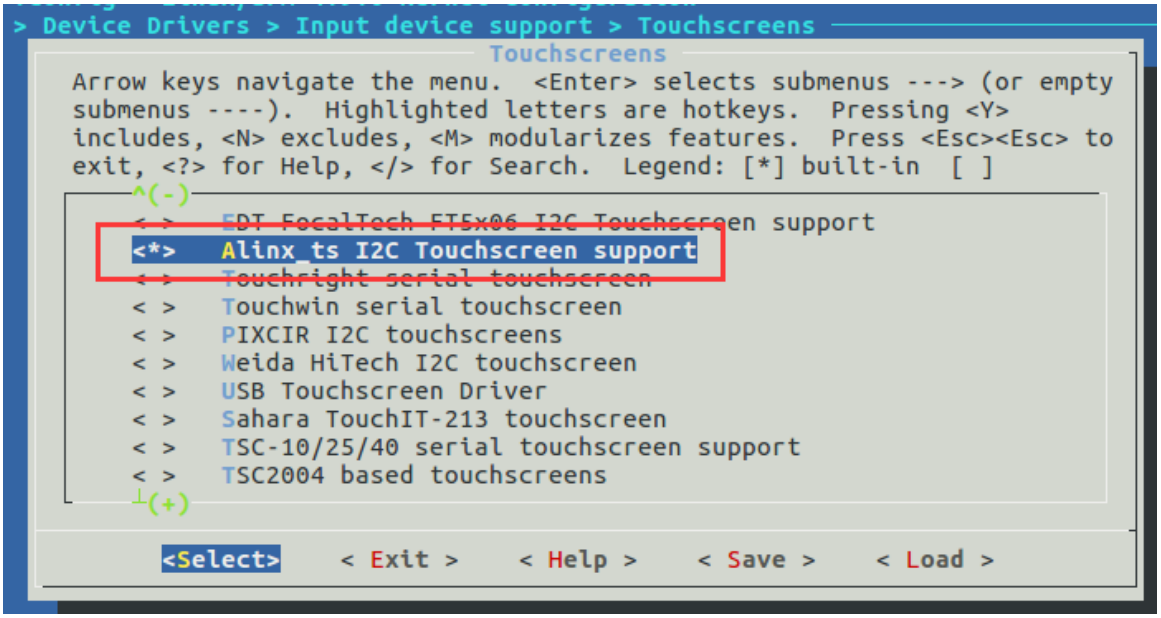


4.3 内核驱动配置

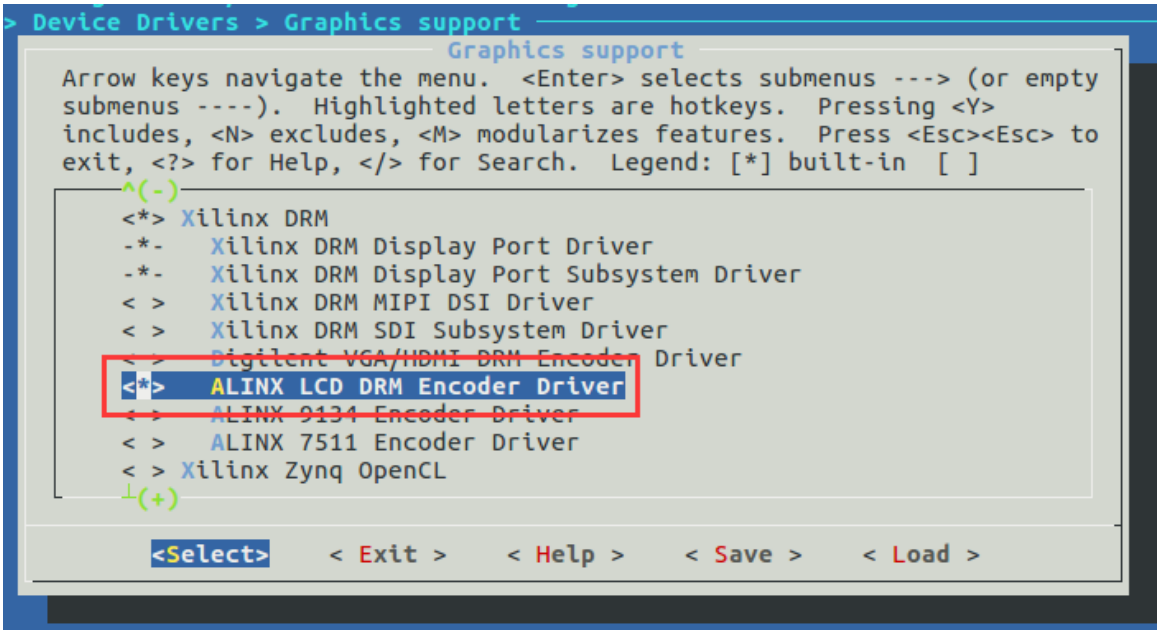
1) Device Drivers > Input device support > [*] Touchscreens 使能触摸屏



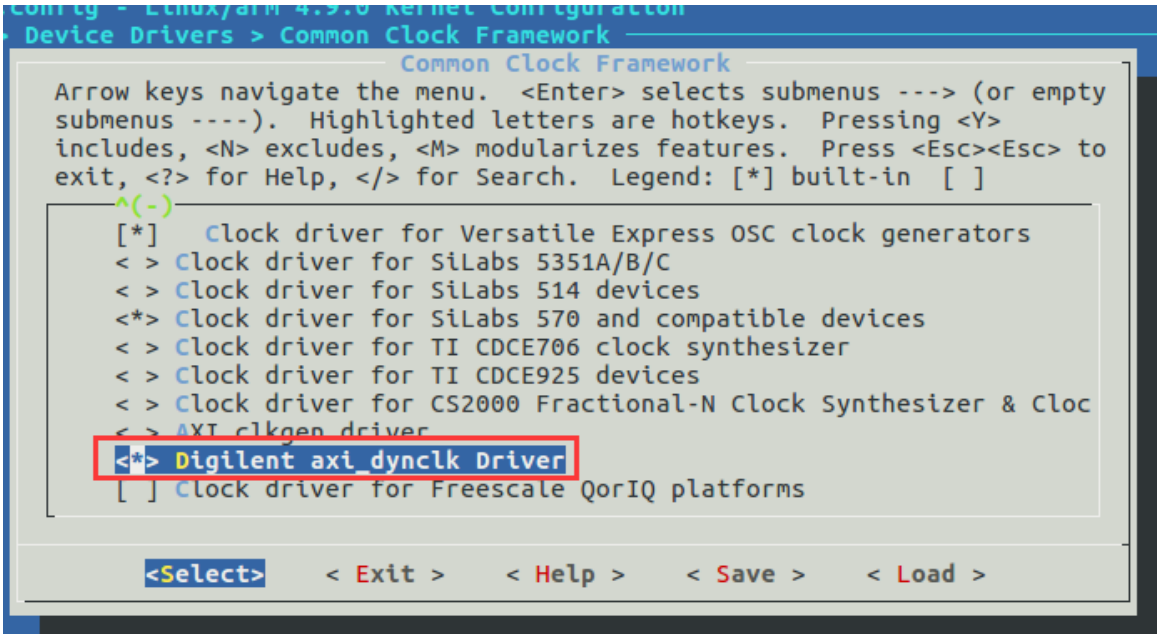
2) 选择 <*> Alinx_ts I2C Touchscreen support



3) 在 Device Drivers > Graphics support 中选择<*> ALINX LCD DRM Encoder Driver



4) 在 Device Drivers > Common Clock Framework 中选择<*> Digilent axi_dynclk Driver



5) 如果用到其他外设，请更加需要配置驱动

4.4 设备树配置

在 i2c0 下挂触摸屏模块，设备地址为 0x38（7bit 地址）。详细设备树可以参考例程 petalinux 工程目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

```
&i2c0 {
        clock-frequency = <100000>;
        alinx_an071@38 {
                compatible = "alinx,an071";
                reg = <0x38>;
                interrupt-parent = <&intc>;
                interrupts = <0 29 4>;
        };

};
```

4.5 触摸屏测试

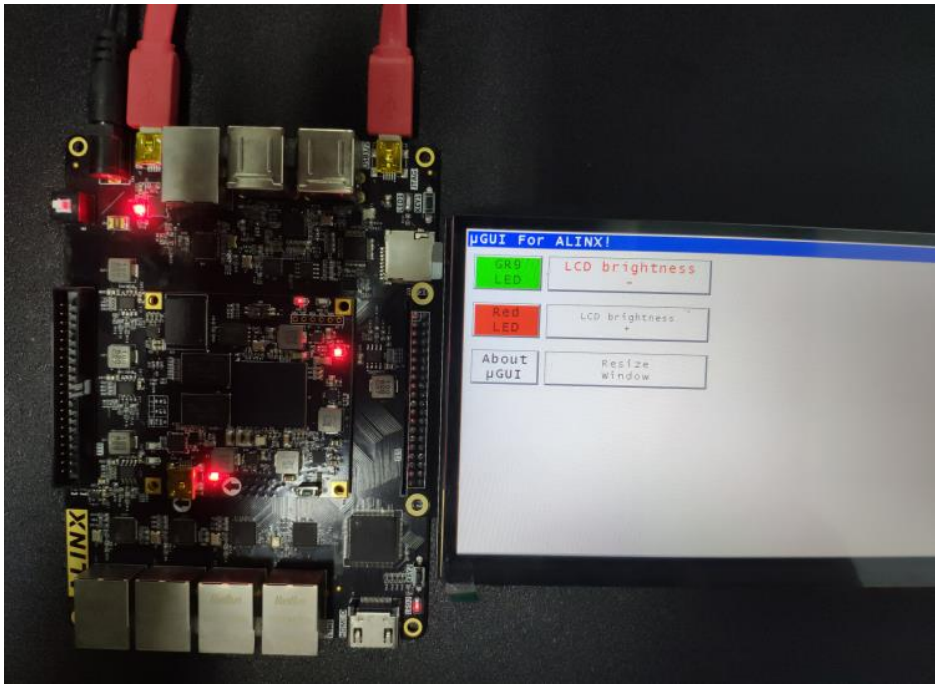
1) 用例程给的 petalinux 工程生成的 BOOT.BIN 和 image.ub，放入 SD 卡

2) 挂载 NFS，运行前面例程的 QT 测试程序

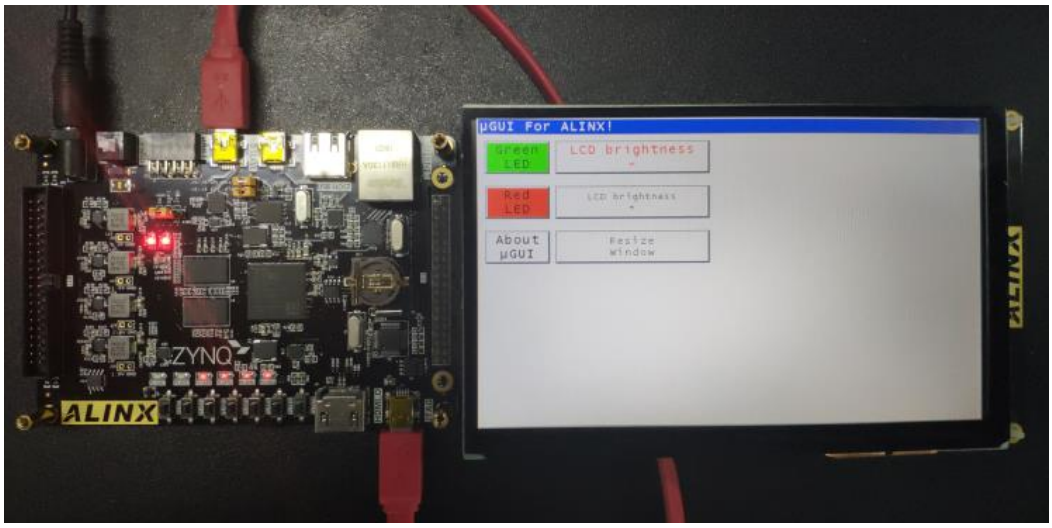
```
mount -t nfs -o nolock 192.168.1.55:/home/alinx/work /mnt
cd /mnt
mkdir /tmp/qt
mount qt_lib.img /tmp/qt
cd /tmp/qt
source ./qt_env_set.sh
cd /mnt
cd build-qt_test-ZYNQ-Debug
./qt_test
```

3) 运行效果

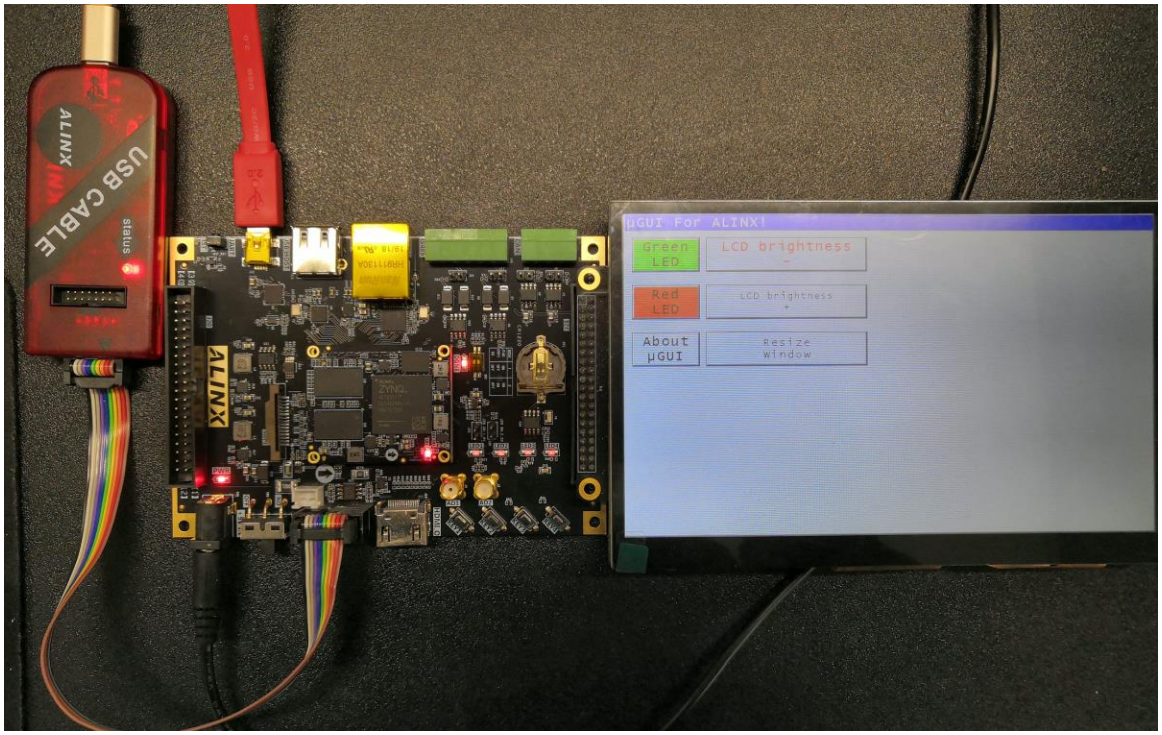




AX7021 硬件连接（J15 扩展口）请忽略显示内容



AX7020/AX7010 硬件连接（J11 扩展口）请忽略显示内容



AX7Z020/AX7Z010 硬件连接图（扩展口 J21）请忽略显示内容

第五章 AXI DMA 读写测试

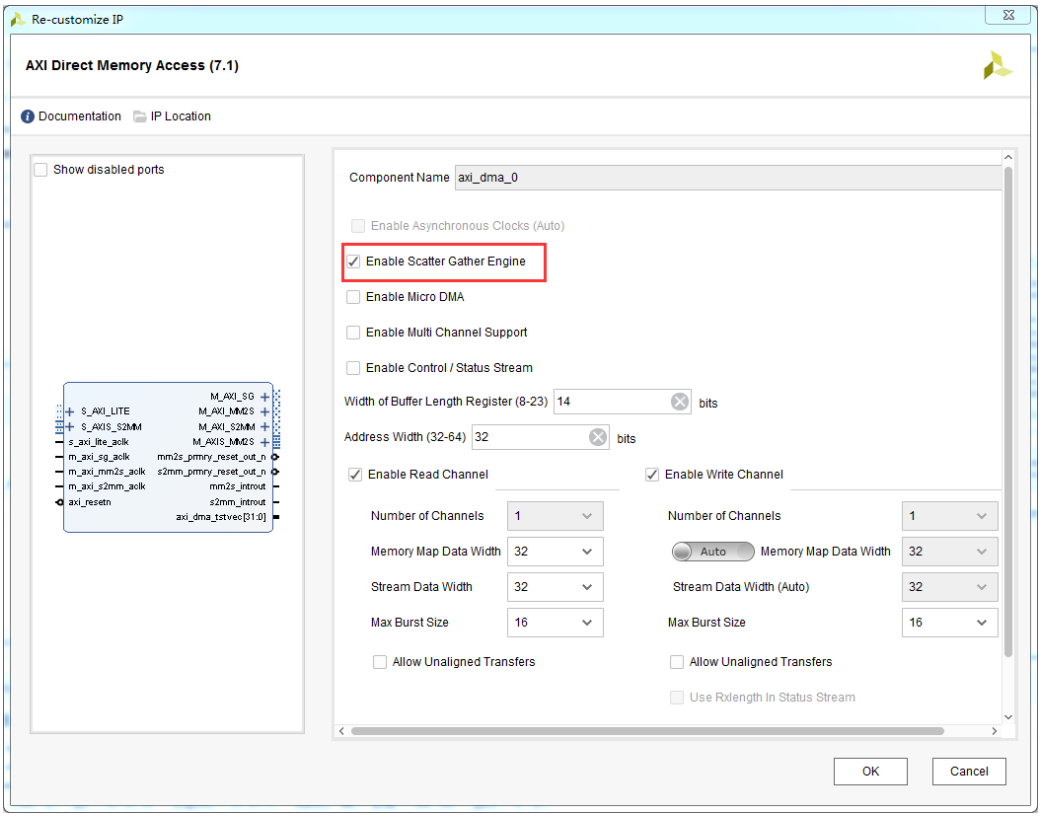
例程相关文件夹：sg_dma_test

Vivado 工程在 vivado.zip 压缩包中：sg_dma_test

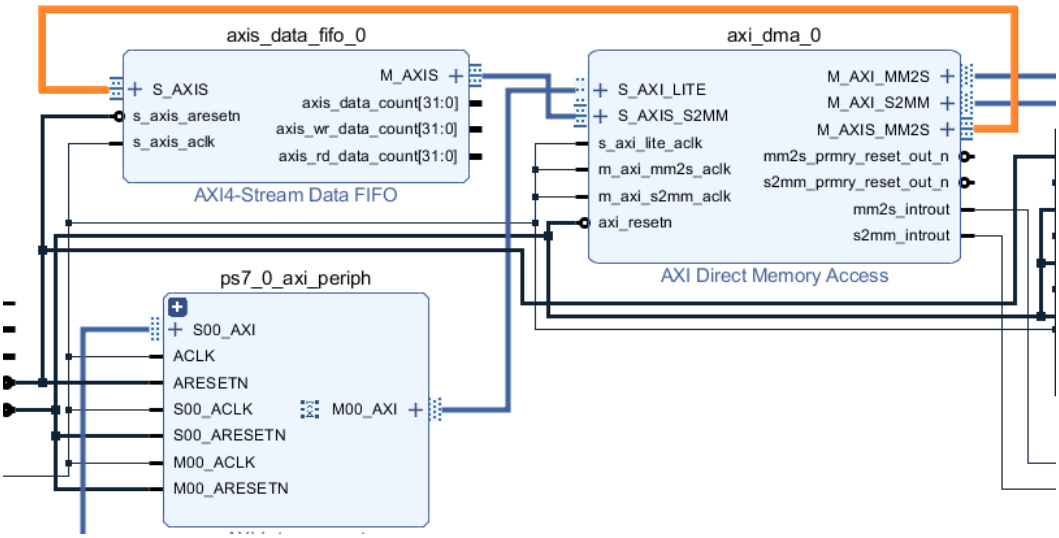
在前面的SDK裸机例程中我们大量讲解了如何使用DMA，现在我们要在Linux下使用DMA，需要说明的是，这里使用的 DMA 支持“Scatter Gather”，也就是 SG-DMA。Xilinx 提供了 DMA 驱动和 DMA 测试程序，测试程序也是一个驱动，从这可以看出，如果要使用 DMA，就算有了 DMA 驱动，我们还是要开发一个 DMA 驱动客户端，称为 DMA client，所以 DMA 的使用是有一定的门槛，对于 Linux 门外汉来说非常有挑战。

5.1 vivado 工程

1) 和 SDK 下测试 DMA 类似，添加 DMA 的 IP，选择“Enable Scatter Gather Engine”



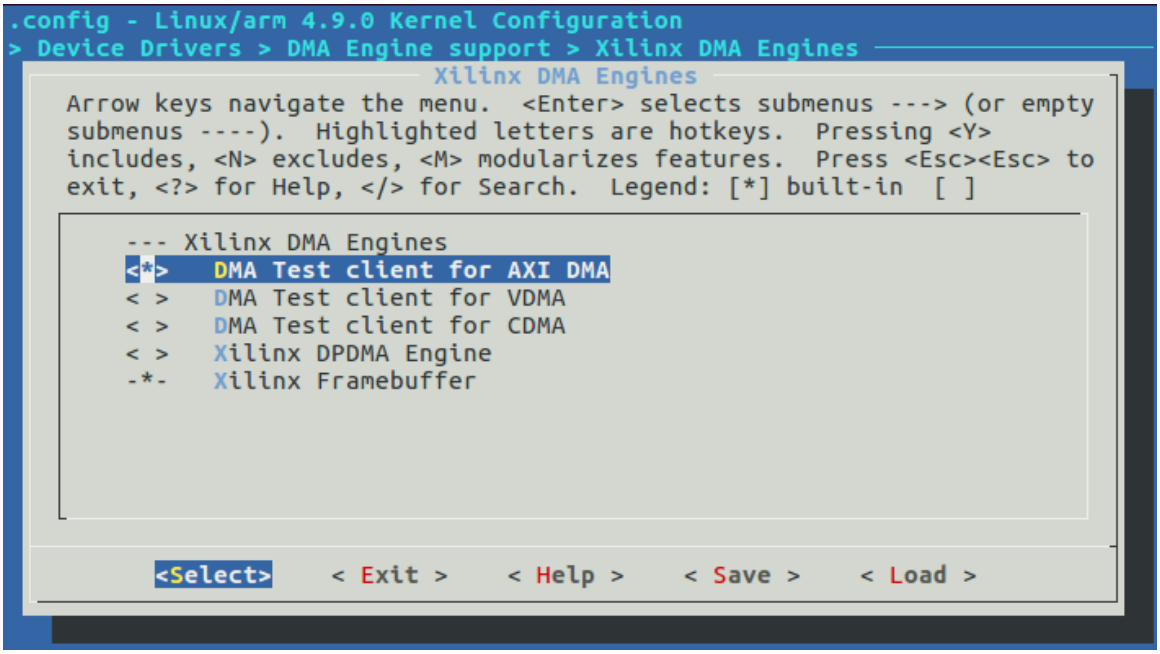
2) 添加一个“AXI4-Stream Data FIFO”，然后将 DMA 的 MM2S（读端口）和 S2MM（写端口）端口连接，组成一个读写环路，用于测试 DMA 功能。



5.2 petalinux 工程配置

1) 使用 Vivado 工程导出的 hdf 文件创建 petalinux 工程

2) 配置 DMA 测试客户端驱动，在 Device Drivers > DMA Engine support > Xilinx DMA Engines 中使能 DMA Test client for AXI DMA



3) 修改设备树，将 petalinux 工程中的 system-user.dtsi 文件修改如下

```
/include/ "system-conf.dtsi"
/ {
```

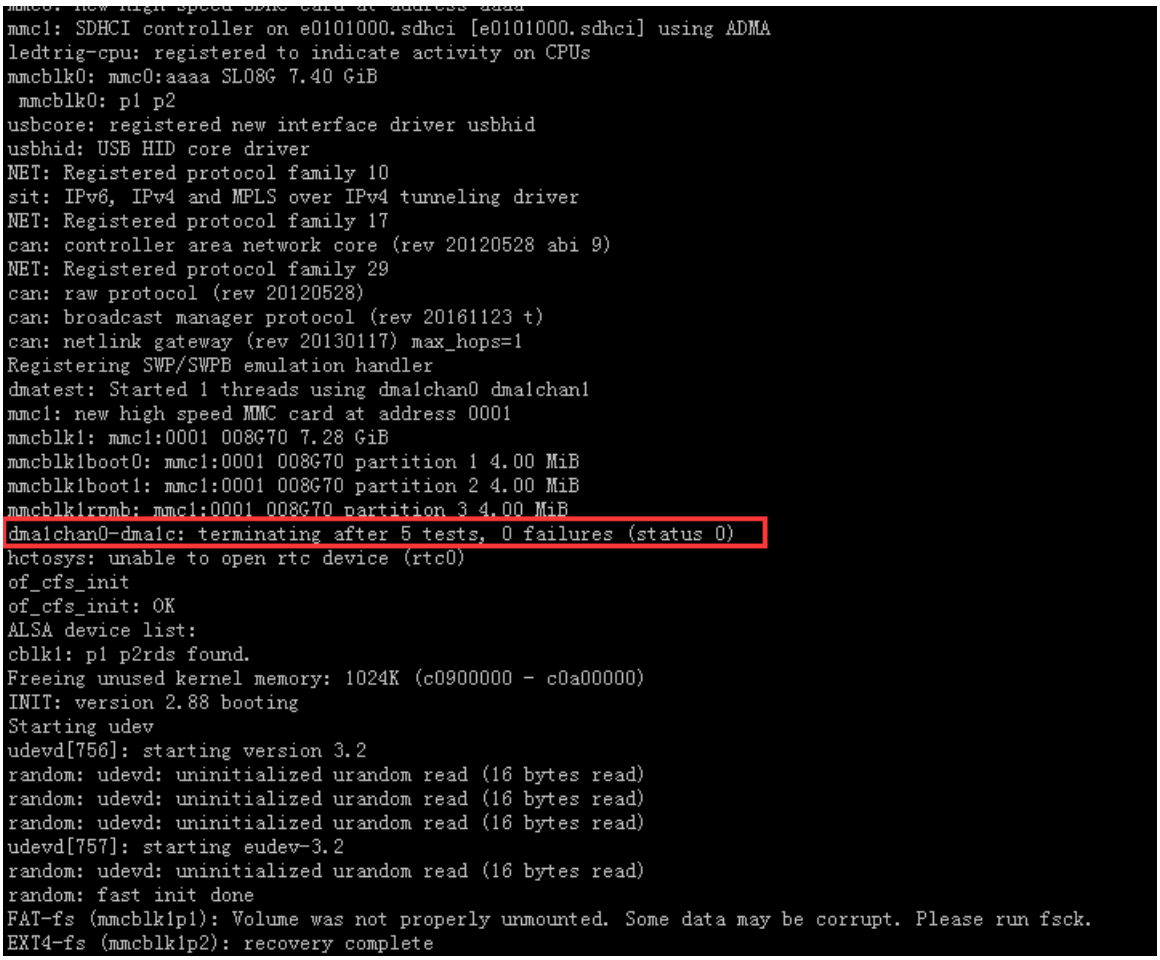
```
        axidmatest_0: axidmatest {
                                compatible ="xlnx,axi-dma-test-1.00.a";
                                dmas = <&axi_dma_0 0
                                        &axi_dma_0 1>;
                                dma-names = "axidma0", "axidma1";
        };

};
```

5.3 运行测试

1) 编译生成 BOOT.BIN 和 image.ub
2) 将 BOOT.BIN 和 image.ub 复制到 sd 卡中，开发板 sd 卡模式启动运行
3) 测试结果可以看到，DMA 读写环通测试 5 次，没有错误

```
dma1chan0-dma1c: terminating after 5 tests, 0 failures (status 0)
```



第六章 基于 DMA 的 ADC 波形显示(AN9238)

例程相关文件夹：an9238_wave

Vivado 工程在 vivado.zip 压缩包中：an9238_wave

在 SDK 裸机例程里我们用 DMA 将 ADC 模块 AN926 的数据采集到 ddr 中然后显示出波形，在上面的例程用我们也学习了 DMA 的读写测试，我们再这个读写测试的基础上写了一个 ADC 驱动，所以请大家在学习本章内容时请熟练掌握前面的例程。

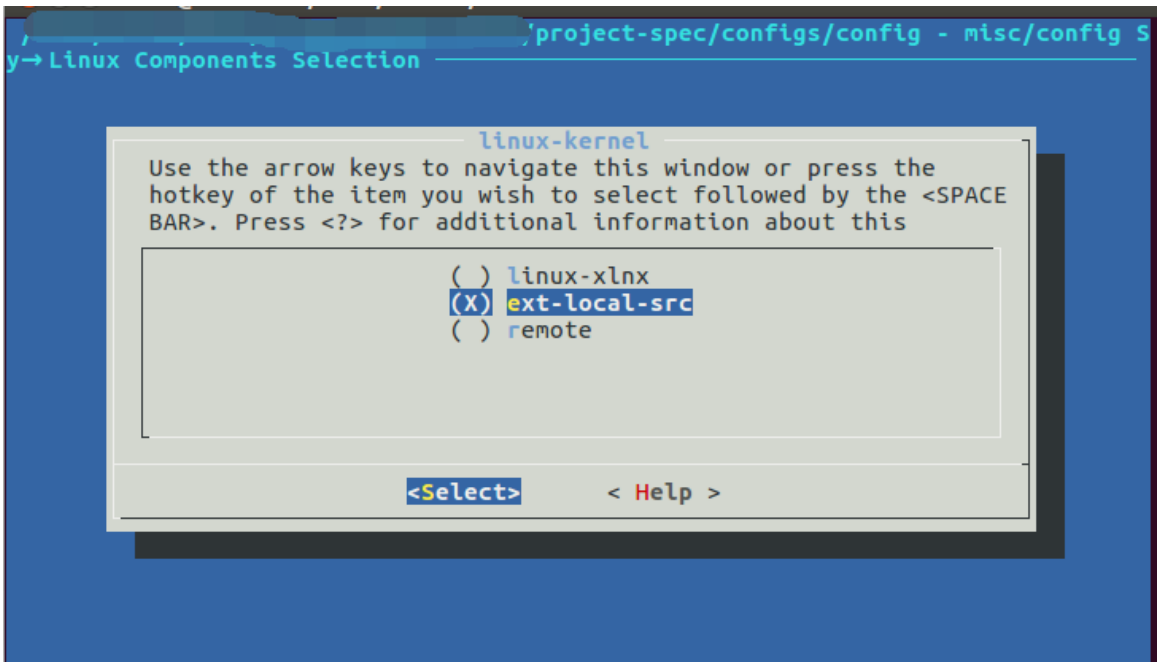
例程资料中给出了 vivado 工程，petalinux 工程，以及波形显示 app。

6.1 ADC 驱动

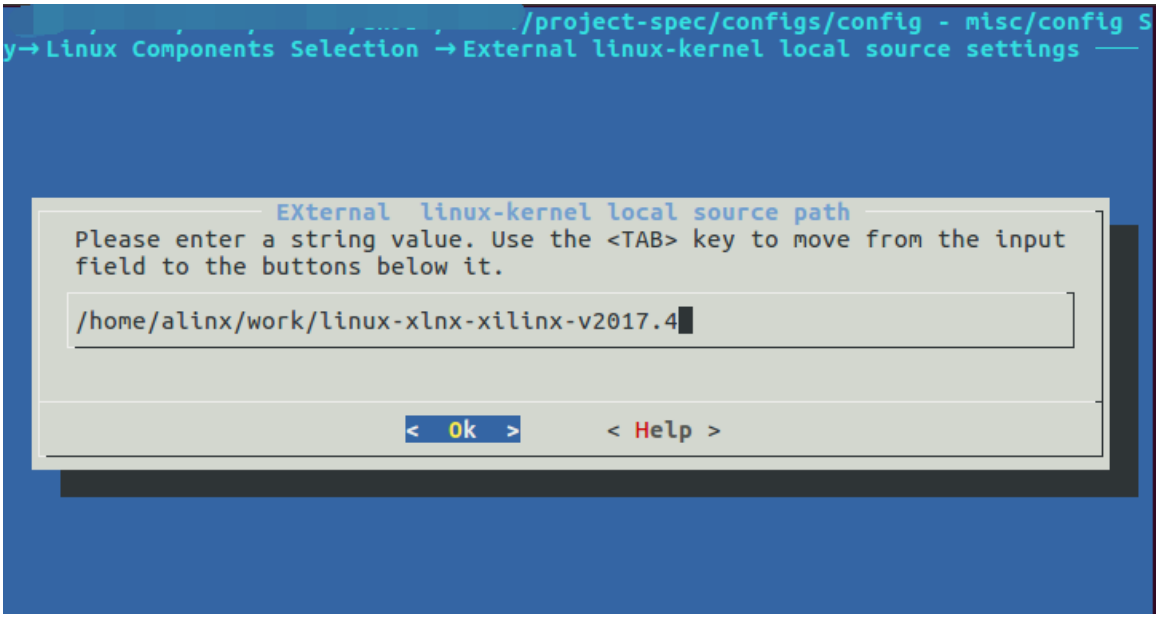
1) ADC 驱动是内核 drivers/dma/xilinx 目录中的 axi_adc_dma.c，驱动编写时参考了同目录下的 axidmatest.c 文件和 Xilinx 官方例程“xapp1183”。

6.2 petalinux 工程配置

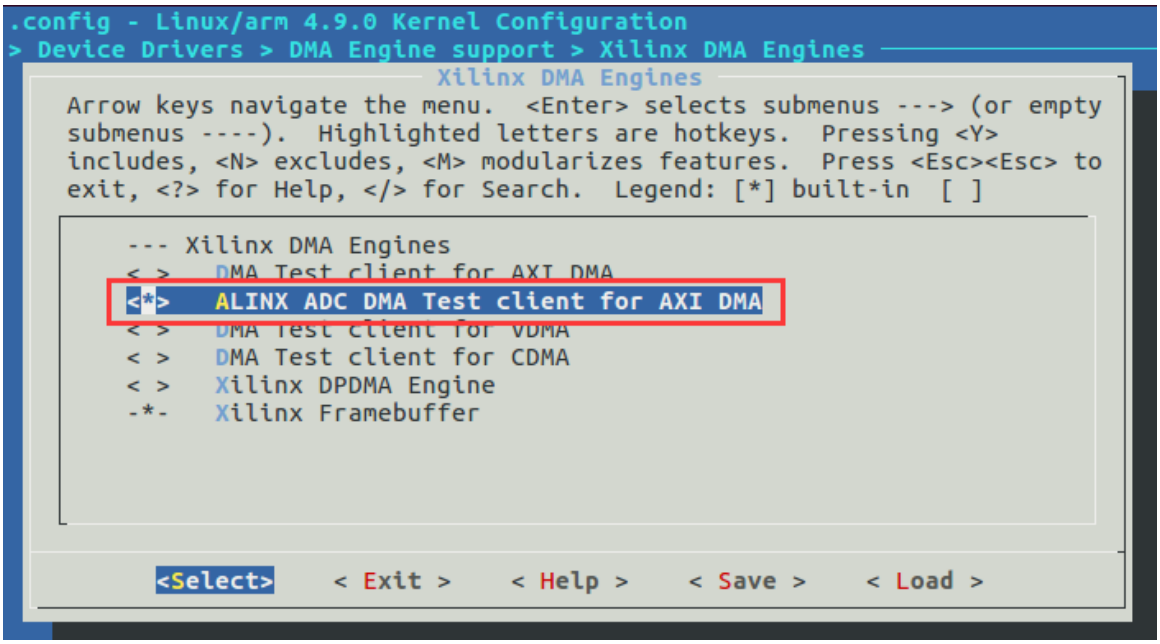
1) 配置内核选择本地代码



2) 填写内核源代码路径



3) 和前面的例程一样要配置一些外设的驱动，这里不再复述，主要是配置 ADC 的驱动，在 Device Drivers > DMA Engine support > Xilinx DMA Engines 选项中选择<*> ALINX ADC DMA Test client for AXI DMA

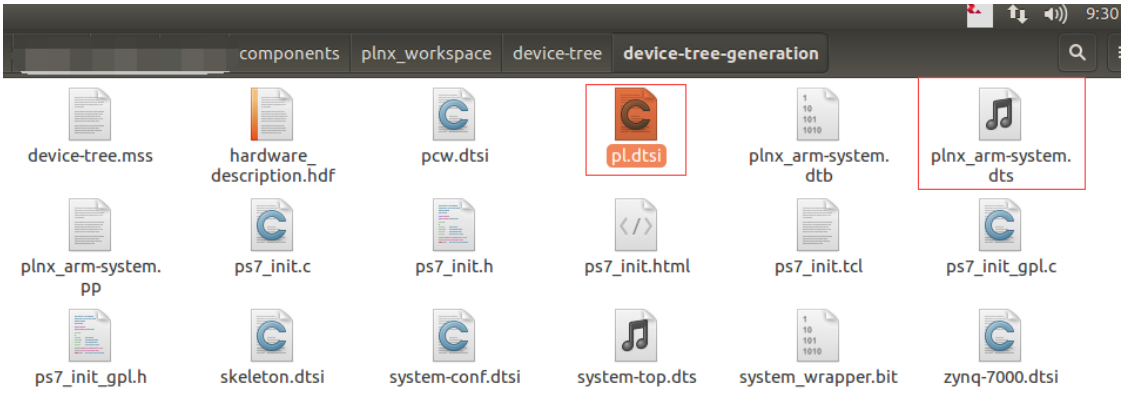


6.3 设备树配置

1) 这里是修改了驱动的匹配选项，compatible = "alinx,axi-adc-dma";用于匹配驱动程序。详细设备树可以参考例程 petalinux 工程目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

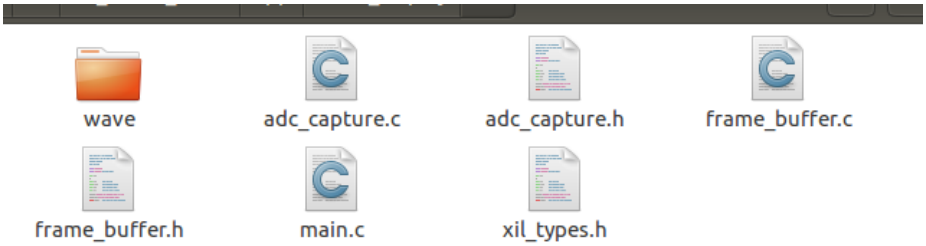
```
&ad9238_sample_0 {
        compatible = "alinx,axi-adc-dma";
        //compatible = "alinx,dma_test";
        dmas = <&axi_dma_0 0>;
        dma-names = "axidma1";
};
```

2) 设备树修改技巧，先编译一次，然后到 components/plnx_workspace/device-tree/device-tree-generation 目录下找到 pl.dtsi 这里包含了所有 PL 端外设的设备树信息，最好不要直接修改这个文件，plnx_arm-system.dts 是最后生成的文件，也是非常重要的信息，开发者可以自己好好研究一下这些文件。



6.4 应用程序

1) 应用程序采用 XSDK 编写，XSDK 建立 Linux 应用程序的方法在前面的教程中讲到。

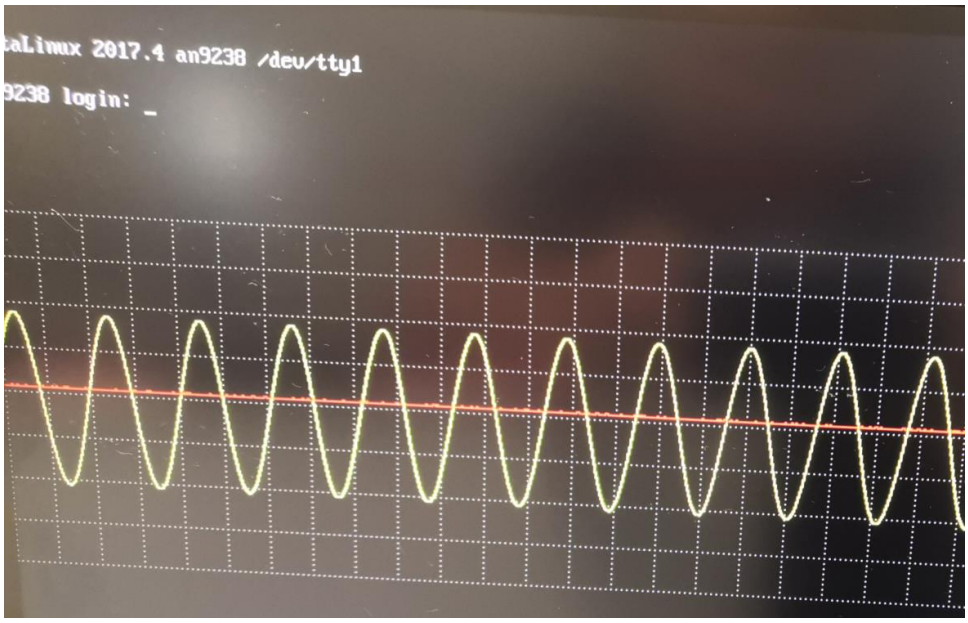


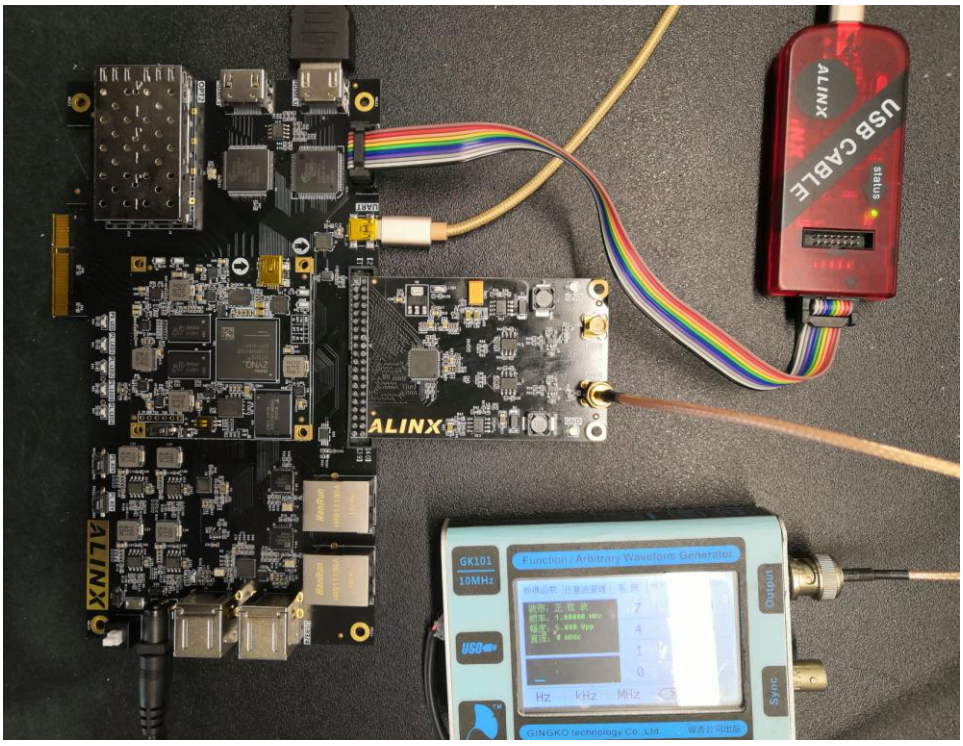
2) 程序没有使用 QT，而是直接绘制 framebuffer，frame_buffer.c 文件为 Linux 的 fb 操作。
3) adc_capture.c 为波形采集模块，主要是打开 ADC 驱动
4) 波形绘制函数完全来自裸机 SDK

6.5 运行结果

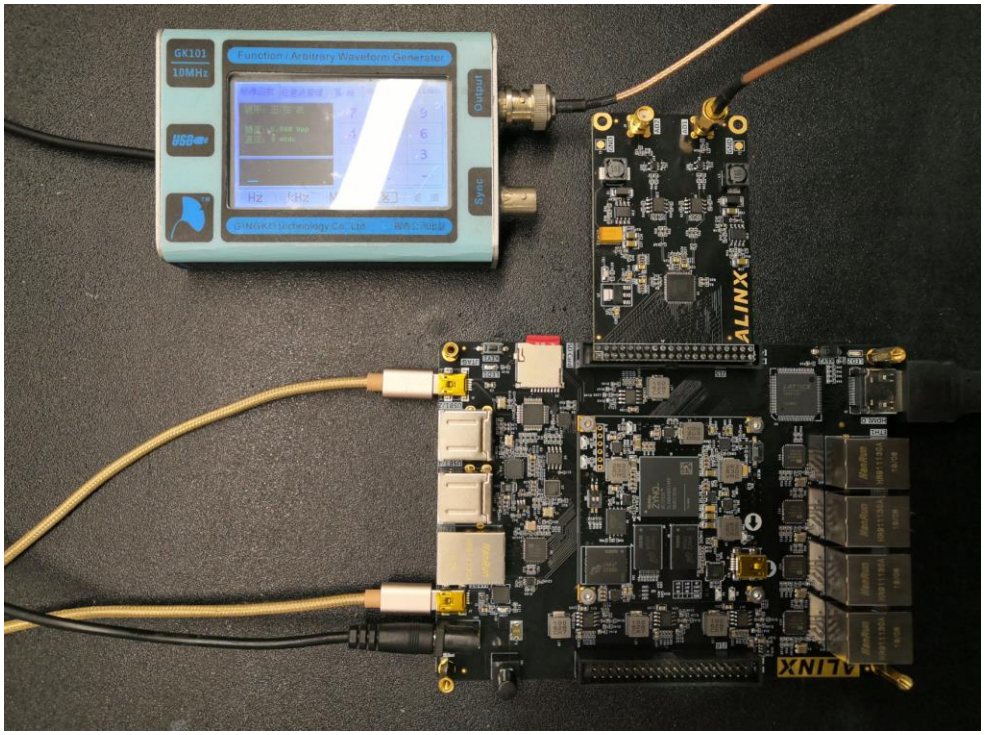
1) 资料包中给出了编译好的程序在 sd_boot 目录，所有文件复制到 sd 卡根目录运行即可。

2) 给 AN9238 第一通道输入峰峰值为 5V，频率 1Mhz 的正弦波，波形显示如下。

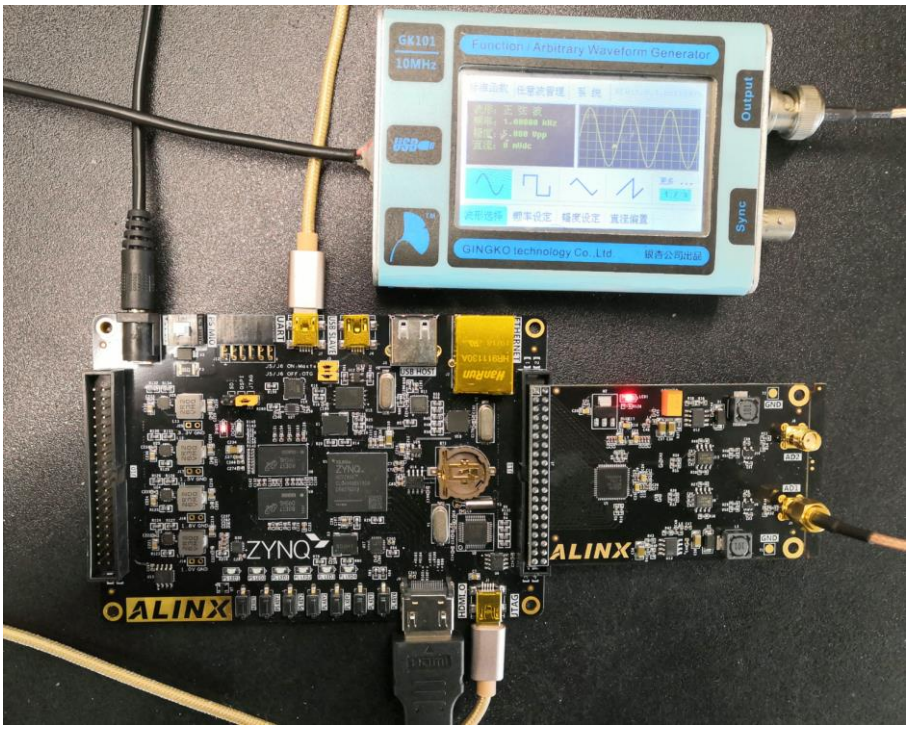




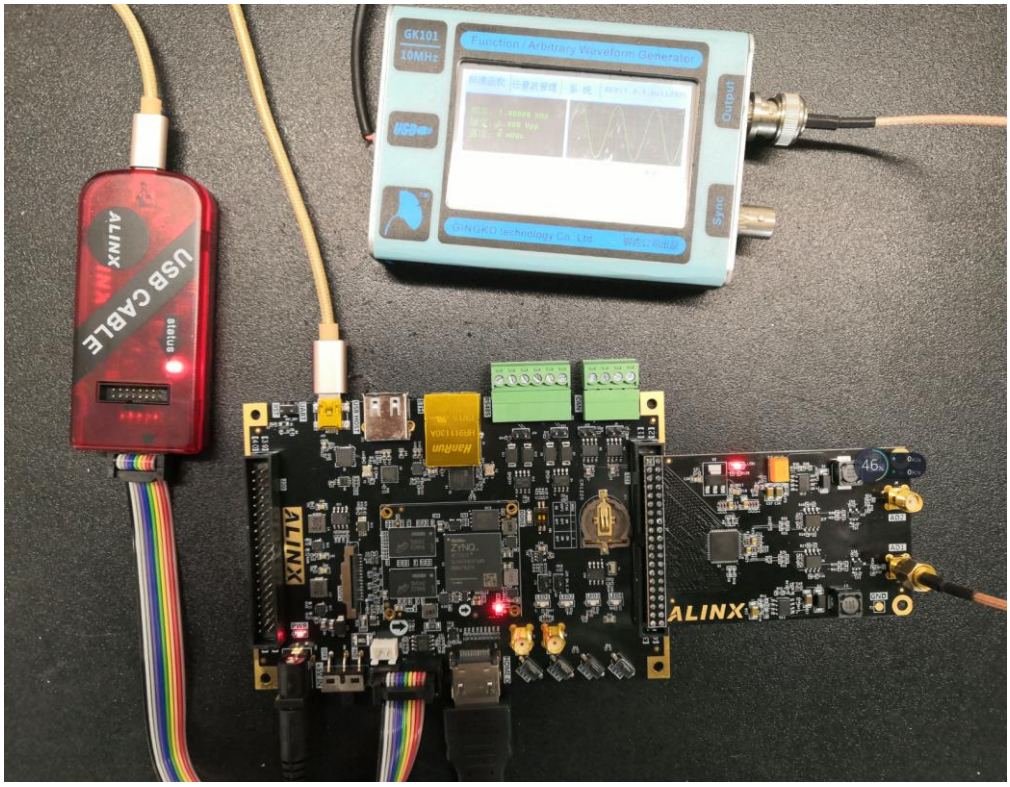
AX7015 硬件连接图



AX7021 硬件连接图（J15 扩展口）



AX7020/AX7010 硬件连接图（J11 扩展口）



AX7Z020/AX7Z010 硬件连接图（J21 扩展口）

6.6 常见问题

6.6.1 内核崩溃

启动过程中有一定几率内核崩溃，原因未知，需要重启开发板

```
task: c0a06bc0 task.stack: c0a00000
PC is at 0xffffffe
LR is at rcu_process_callbacks+0x26c/0x41c
pc : [<ffffffe>]    lr : [<c015e240>]    psr: a00e0133
sp : c0a01e70  ip : ddc9fc5c  fp : c09421c0
r10: c09421c0  r9 : 00000000  r8 : 00000000
r7 : debccfe0  r6 : c0a0cac0  r5 : c0a0cac0  r4 : debccfc0
r3 : 00000000  r2 : c0a00000  r1 : ffffffff  r0 : ddc9fc5c
Flags: NzCv  IRQs on  FIQs on  Mode SVC_32  ISA Thumb  Segment none
Control: 18c5387d  Table: 1dc6004a  DAC: 00000051
Process swapper/0 (pid: 0, stack limit = 0xc0a00210)
Stack: (0xc0a01e70 to 0xc0a02000)
1e60:                                     00000000 debcc9c0 00000000 ddc9fc5c
1e80: 0000000a c0a0302c 00000000 c0a00000 00000003 c0a00000 40000001 00000002
1ea0: c0a32bc0 0000000a c0a020a4 00000101 c0a01eb8 c011eab0 debca768 00000001
1ec0: 00200100 ffff8b80 00000002 c0a02d00 c0a020a0 00000009 00000000 00000000
1ee0: 00000000 c093fd90 00000012 de402600 c0a00000 00000000 00000000 c011ed48
1f00: 00000000 c0154114 f8f00100 c0a03b1c c0a01f38 c0a13e00 f8f01100 c01013f8
1f20: c04b2e14 600e0013 ffffffff c0a01f6c 6962c202 c010ae8c 00000000 00000000
1f40: 1e28b000 debcc9c0 69fa3377 00000000 debcbe40 00000000 6962c202 00000000
1f60: 00000000 00000000 0000000b c0a01f88 c04b2df0 c04b2e14 600e0013 ffffffff
1f80: 00000051 00000000 debcbe40 c0a28e3c 00000000 debcbe40 c0a28e3c c0a01fb8
1fa0: c0a00000 c0a0302c 00000001 c0a0307c 00000000 c014c834 c0a03000 c0900bb4
1fc0: ffffffff ffffffff 00000000 c0900670 00000000 c0931a30 00000000 c0a324d4
1fe0: c0a03018 c0931a2c c0a07db8 0000406a 413fc090 0000807c 00000000 00000000
[<c015e240>] (rcu_process_callbacks) from [<c011eab0>] (__do_softirq+0x15c/0x1bc)
[<c011eab0>] (__do_softirq) from [<c011ed48>] (irq_exit+0x68/0xd0)
[<c011ed48>] (irq_exit) from [<c0154114>] (__handle_domain_irq+0x80/0xb4)
[<c0154114>] (__handle_domain_irq) from [<c01013f8>] (gic_handle_irq+0x50/0x94)
[<c01013f8>] (gic_handle_irq) from [<c010ae8c>] (__irq_svc+0x6c/0xa8)
Exception stack(0xc0a01f38 to 0xc0a01f80)
1f20:                                                       00000000 00000000
1f40: 1e28b000 debcc9c0 69fa3377 00000000 debcbe40 00000000 6962c202 00000000
1f60: 00000000 00000000 0000000b c0a01f88 c04b2df0 c04b2e14 600e0013 ffffffff
[<c010ae8c>] (__irq_svc) from [<c04b2e14>] (cpuidle_enter_state+0xe4/0x1b0)
[<c04b2e14>] (cpuidle_enter_state) from [<c014c834>] (cpu_startup_entry+0x17c/0x19c)
[<c014c834>] (cpu_startup_entry) from [<c0900bb4>] (start_kernel+0x2f8/0x358)
Code: bad PC value
---[ end trace 2fddbbf414e7a837 ]---
Kernel panic - not syncing: Fatal exception in interrupt
CPU1: stopping
CPU: 1 PID: 0 Comm: swapper/1 Tainted: G      D         4.9.0-xilinx #1
Hardware name: Xilinx Zynq Platform
[<c010e244>] (unwind_backtrace) from [<c010a4a4>] (show_stack+0x10/0x14)
[<c010a4a4>] (show_stack) from [<c02dc2d0>] (dump_stack+0x80/0x9c)
[<c02dc2d0>] (dump_stack) from [<c010c9f8>] (ipi_cpu_stop+0x3c/0x70)
[<c010c9f8>] (ipi_cpu_stop) from [<c010d210>] (handle_IPI+0x64/0x84)
[<c010d210>] (handle_IPI) from [<c0101420>] (gic_handle_irq+0x78/0x94)
[<c0101420>] (gic_handle_irq) from [<c010ae8c>] (__irq_svc+0x6c/0xa8)
Exception stack(0xde471f70 to 0xde471fb8)
1f60:                                     00000000 00000000 1e299000 debda9c0
1f80: 792b8fc8 00000000 debd9e40 00000000 78e2382a 00000000 00000000 00000000
1fa0: 0000000b de471fc0 c04b2df0 c04b2e14 60070013 ffffffff
[<c010ae8c>] (__irq_svc) from [<c04b2e14>] (cpuidle_enter_state+0xe4/0x1b0)
[<c04b2e14>] (cpuidle_enter_state) from [<c014c834>] (cpu_startup_entry+0x17c/0x19c)
[<c014c834>] (cpu_startup_entry) from [<001014cc>] (0x1014cc)
---[ end Kernel panic - not syncing: Fatal exception in interrupt
random: fast init done
```

第七章 基于 DMA 的 ADC 波形显示(AN706)

例程相关文件夹：an706_wave

Vivado 工程在 vivado.zip 压缩包中：an706_wave

在 SDK 裸机例程里我们用 DMA 将 ADC 模块 AN706 的数据采集到 ddr 中然后显示出波形，在上面的例程用我们也学习了 DMA 的读写测试，我们再这个读写测试的基础上写了一个 ADC 驱动，所以请大家在学习本章内容时请熟练掌握前面的例程。

例程资料中给出了 vivado 工程，petalinux 工程，以及波形显示 app。

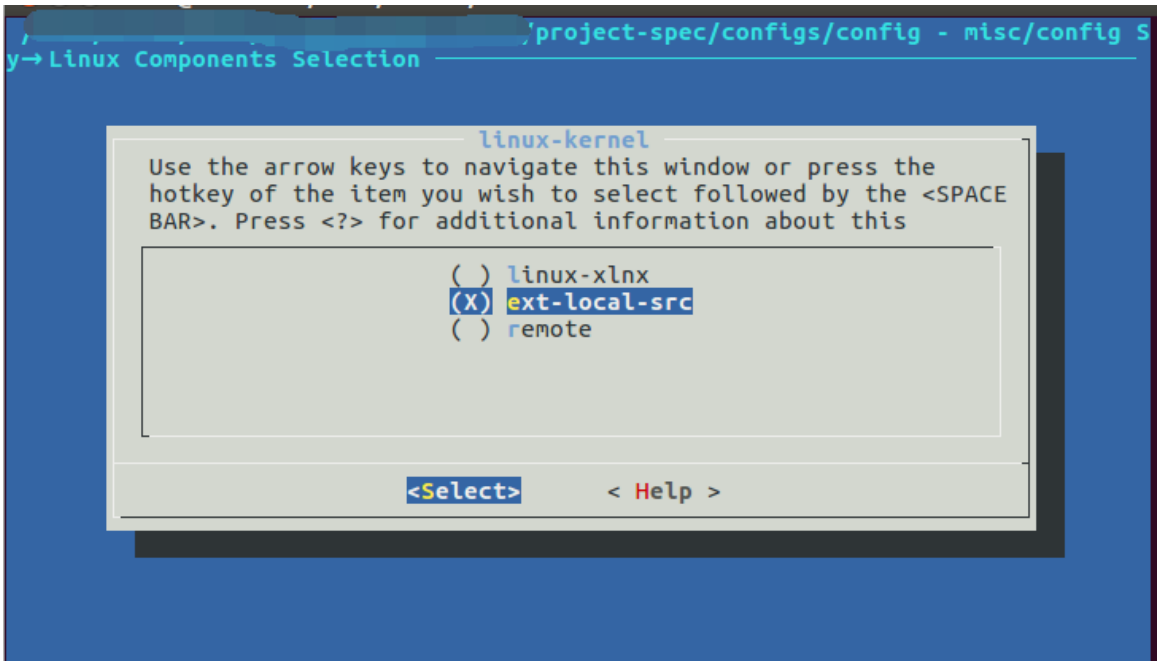
本例程和前面例程非常相似，区别在于只使用一个 DMA 传输 8 个通道的 ADC 数据，在应用层将 8 个通道的数据分离。

7.1 ADC 驱动

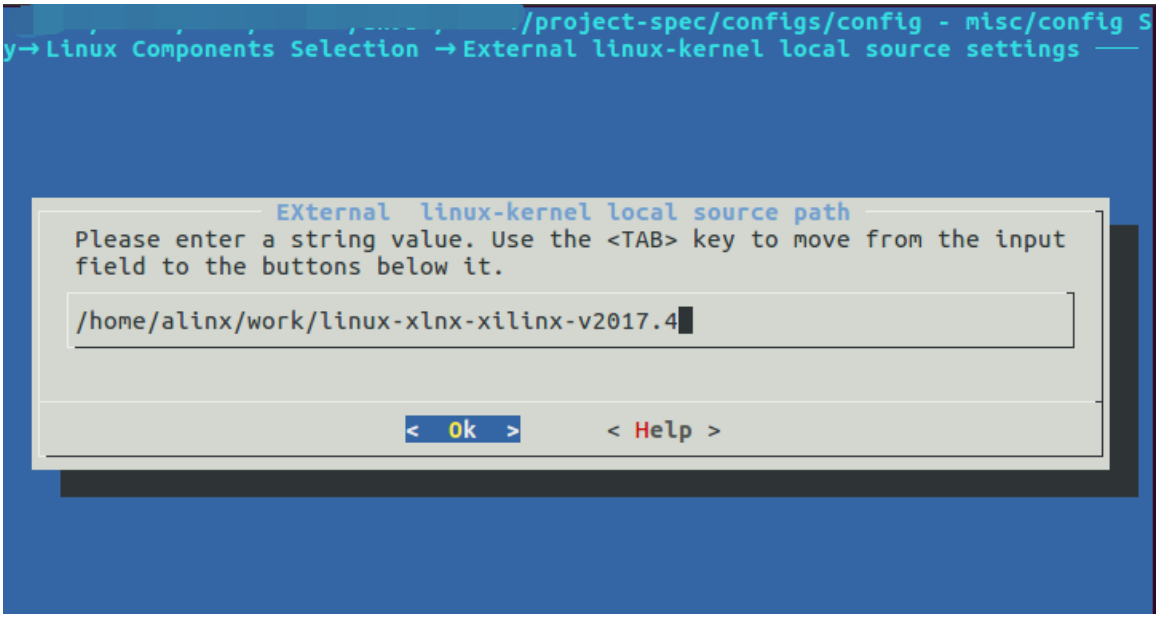
1) ADC 驱动是内核 drivers/dma/xilinx 目录中的 axi_adc_dma.c，驱动编写时参考了同目录下的 axidmatest.c 文件和 Xilinx 官方例程“xapp1183”。

7.2 petalinux 工程配置

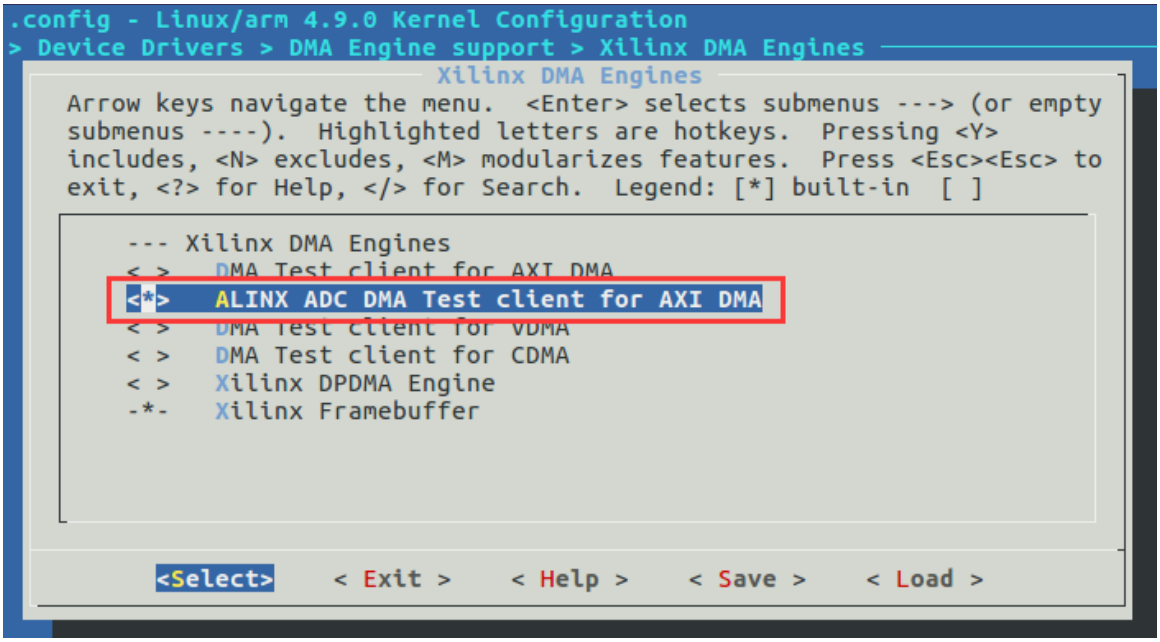
1) 配置内核选择本地代码



2) 填写内核源代码路径



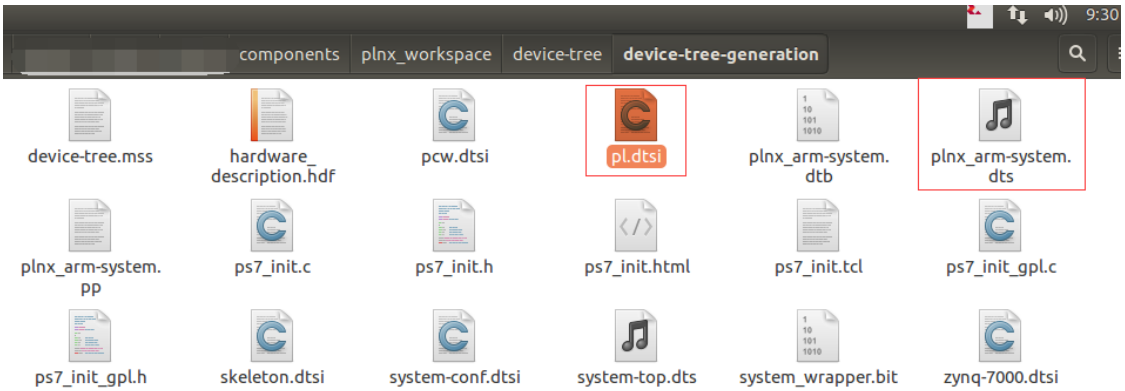
3) 和前面的例程一样要配置一些外设的驱动，这里不再复述，主要是配置 ADC 的驱动，在 Device Drivers > DMA Engine support > Xilinx DMA Engines 选项中选择<*> ALINX ADC DMA Test client for AXI DMA



7.3 设备树配置

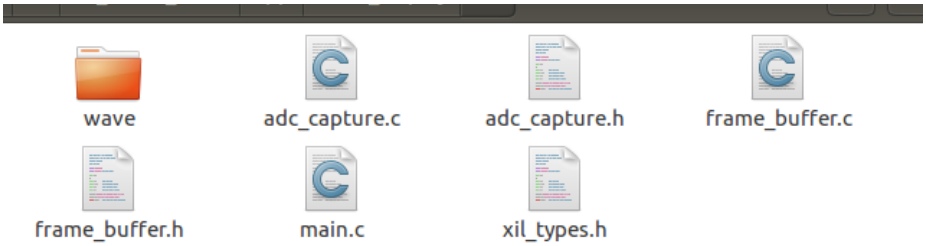
1) 这里是修改了驱动的匹配选项，compatible = "alinx,axi-adc-dma";用于匹配驱动程序。详细设备树可以参考例程 petalinux 工程目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件

2) 设备树修改技巧，先编译一次，然后到 components/plnx_workspace/device-tree/device-tree-generation 目录下找到 pl.dtsi 这里包含了所有 PL 端外设的设备树信息，最好不要直接修改这个文件，plnx_arm-system.dts 是最后生成的文件，也是非常重要的信息，开发者可以自己好好研究一下这些文件。



7.4 应用程序

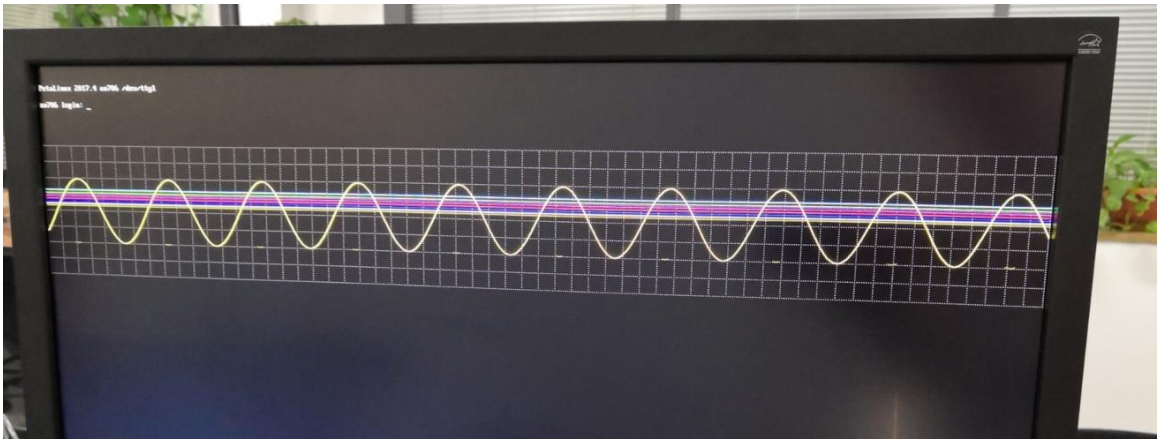
1) 应用程序采用 XSDK 编写，XSDK 建立 Linux 应用程序的方法在前面的教程中讲到。

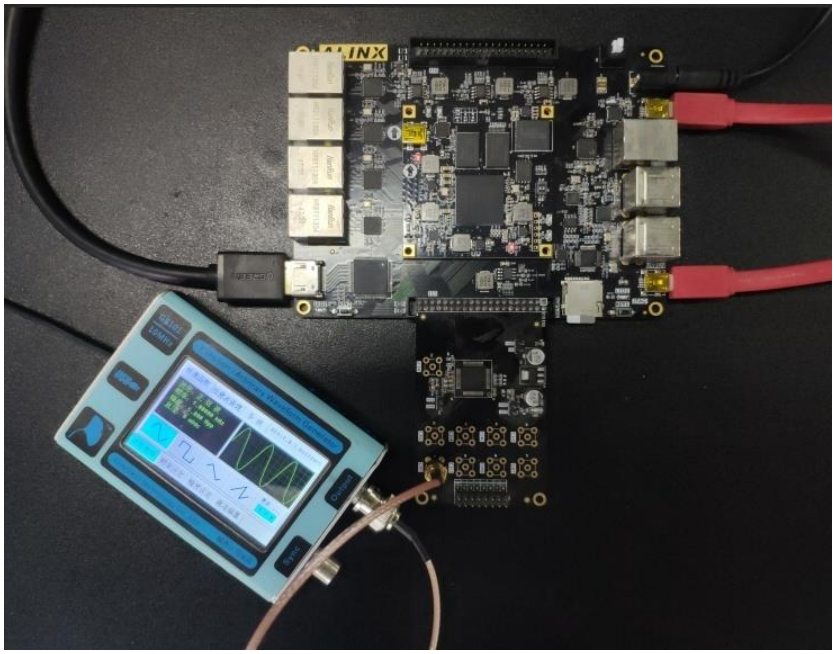


2) 程序没有使用 QT，而是直接绘制 framebuffer，frame_buffer.c 文件为 Linux 的 fb 操作。
3) adc_capture.c 为波形采集模块，主要是打开 ADC 驱动
4) 波形绘制函数完全来自裸机 SDK

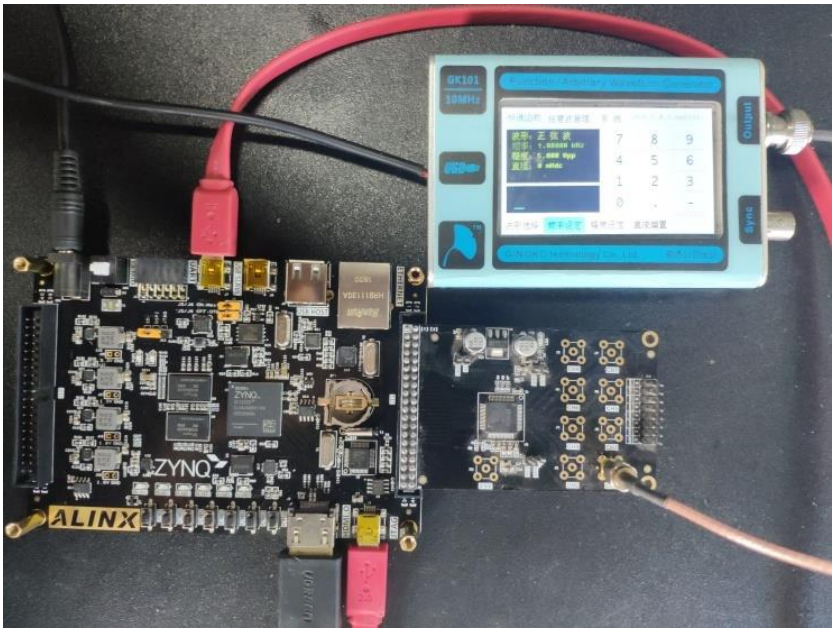
7.5 运行结果

1) 资料包中给出了编译好的程序在 sd_boot 目录，所有文件复制到 sd 卡根目录运行即可。
2) 给 AN706 第一通道输入峰峰值为 5V，频率 1khz 的正弦波，波形显示如下。

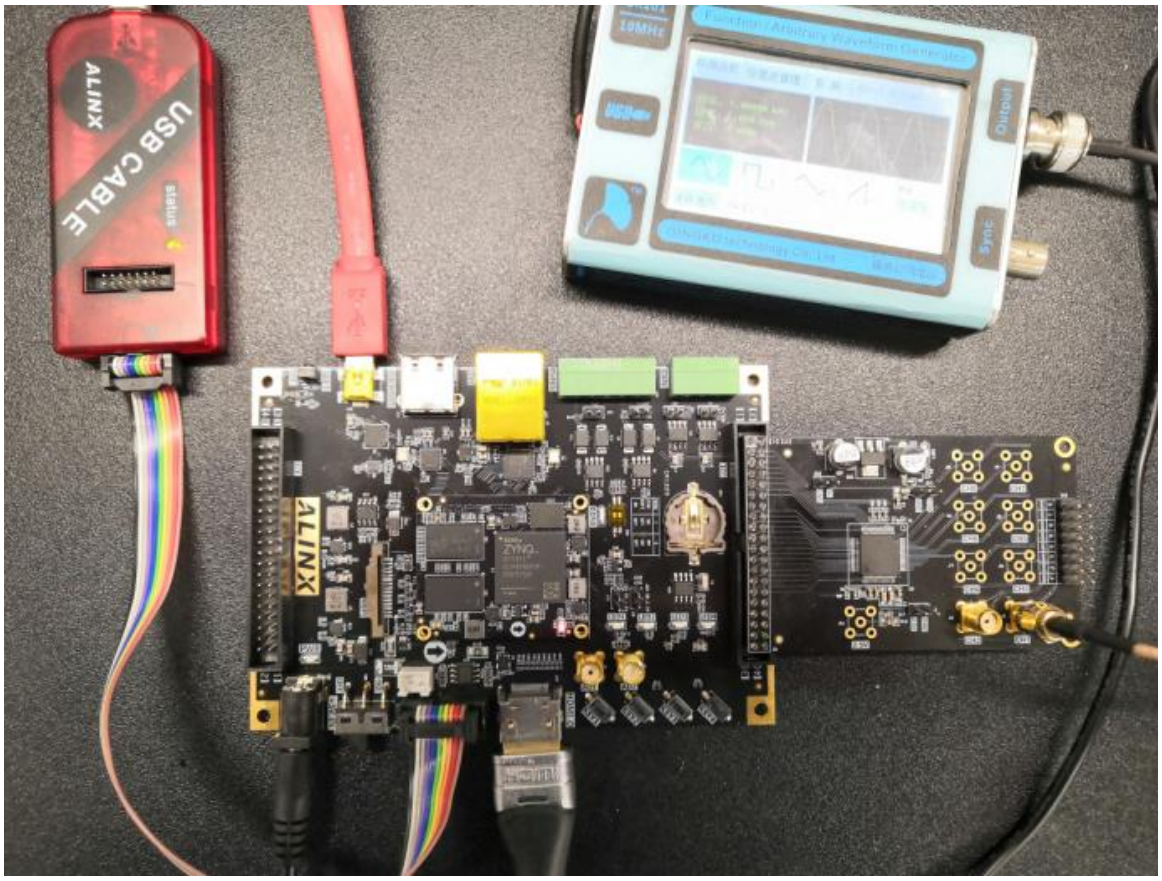




AX7021 硬件连接图（J15 扩展口）



AX7020/AX7010 硬件连接图（J11 扩展口）



AX7Z020/AX7Z010 硬件连接图（扩展口 J21）

第八章 自动运行的 petalinux 应用程序

前面的教程中我们使用了大量 petalinux 来创建 Linux 应用，而且很多时候把根文件系统选为 ramdisk 类型，不能保存我们配置数据，所以我们需要一个上电能自动运行程序的 shell 文件。

例程中给了一个做好的应用程序，可以完成上电自动挂载 sd 卡第一分区到/sd，然后运行/sd 下的 startup.sh 文件。

8.1 准备工作

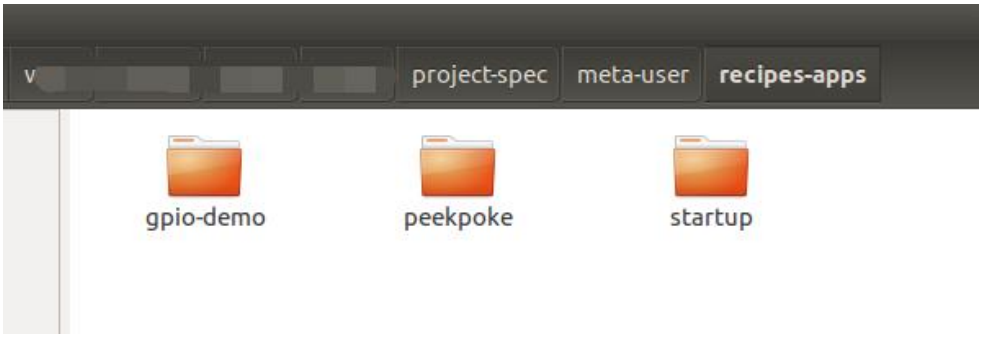
在 xilinx 提供的 petalinux 资料 UG1144 中，有个章节“Creating and Adding Custom Applications”，将如何创建一个自定义应用程序，通过下面命令来创建

```
$ petalinux-create -t apps [--template TYPE] --name --enable
```

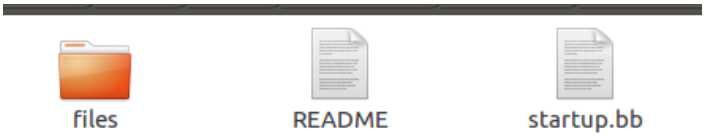
对于专业的软件开发人员来说，学习起来可能比较简单，所以这里不再描述细节，我们把已经做好的 app 提供给大家。

8.2 startup 应用程序

1) 创建好的应用程序都会在文件夹 project-spec/meta-user/recipes-apps 下出现，我们也可以直接把 app 复制到这个目录下



2) startup 文件夹下有个 startup.bb 文件



3) 用文本工具打开，do_install 函数是安装应用的 shell，我们可以大致猜测，安装过程会把 startup.sh 文件复制到根文件系统的 init.d 文件夹下，这样就可以上电自动运行这个文件。

```
#
# This file is the startup recipe.
#

SUMMARY = "Simple startup application"
SECTION = "PETALINUX/apps"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506e

SRC_URI = "file://startup.c \
       file://Makefile \
       file://startup.sh \
          "
INITSCRIPT_NAME = "startup"
INITSCRIPT_PARAMS = "defaults"

S = "${WORKDIR}"

inherit update-rc.d

do_compile() {
         oe_runmake
}

do_install() {
         install -d ${D}${bindir}
         install -m 0755 startup ${D}${bindir}
         install -d ${D}${sysconfdir}/init.d/
         install -m 0755 startup.sh ${D}${sysconfdir}/init.d/startup
}
```

4) 主要文件是 startup.sh，startup.c 没有没什么用途。



5) startup.sh 是先挂载 sd 卡第一分区到 Linux 的/sd 目录下，然后判断/sd/startup.sh 文件是不是存在，如果存在就运行。所以只要我们再 sd 的 fat 分区放一个 startup.sh，就可以实现上电自动运行。

```
start ()
{
    normal="\e[39m"
    lightred="\e[91m"
    lightgreen="\e[92m"
    green="\e[32m"
    yellow="\e[33m"
    cyan="\e[36m"
    red="\e[31m"
    magenta="\e[95m"

    echo -ne $yellow
    echo "Init Start"
    echo -ne $normal
        mkdir /sd
        mount -t vfat /dev/mmcblk0p1 /sd
        if [ -f /sd/startup.sh ]
        then
                echo -ne $lightgreen
                echo "Run startup.sh from SD card"
                echo -ne $normal
                source /sd/startup.sh
        fi

    echo -ne $yellow
    echo "Init End"
    echo -ne $normal

}
stop ()
{
        echo " Stop."
}
restart()
{
        stop
        start
}
```

8.3 使用 startup

1) 把 startup 文件夹复制到 project-spec/meta-user/recipes-apps 下

2) 修改 project-spec/meta-user/recipes-core/images/petalinux-image.bbappend 文件

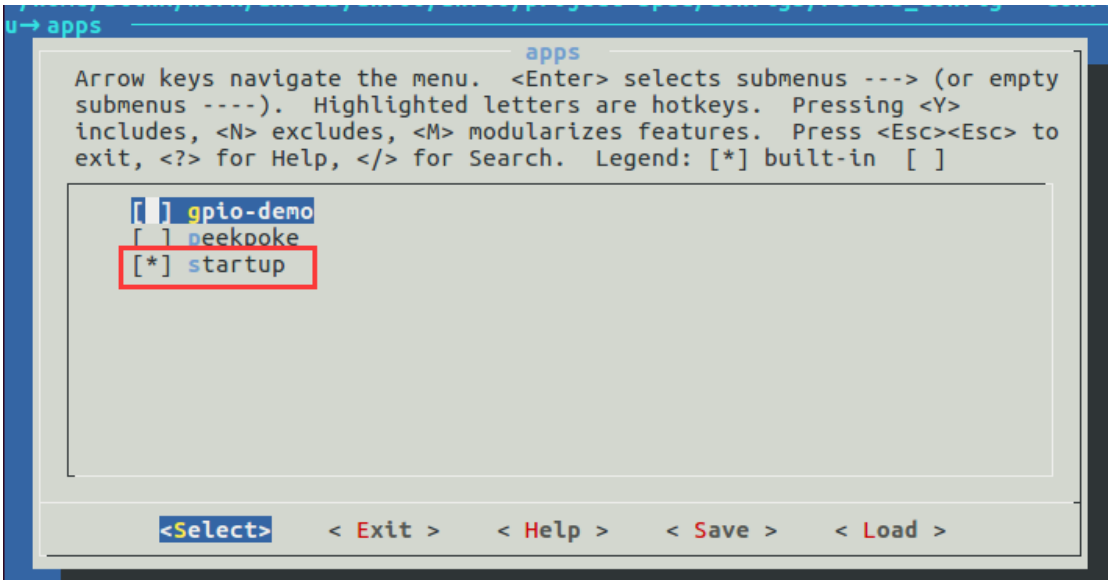
   添加一行 IMAGE_INSTALL_append = " startup"

```
#Note: Mention Each package in individual line
#      cascaded representation with line breaks are not valid in this
file.
IMAGE_INSTALL_append = " peekpoke"
IMAGE_INSTALL_append = " gpio-demo"
IMAGE_INSTALL_append = " startup"
```

3) 使用命令 petalinux-config -c rootfs 配置根文件系统

```
alinx@ubuntu:~/work/ax7015/an706/an706$ source /opt/pkg/petalinux/settings.sh
PetaLinux environment set to '/opt/pkg/petalinux'
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
alinx@ubuntu:~/work/ax7015/an706/an706$ petalinux-config -c rootfs
```

4) 在 apps 中选择 startup，保持编译



第九章 附录： Linux 常用命令

9.1 文件和文件夹操作

9.1.1 创建一个文件夹

mkdir dir

9.1.2 创建多层目录的文件夹

mkdir -p dir1/dir2

9.1.3 改变当前目录

cd newdir	进入 newdir 目录
cd ..	进入上层目录
cd -	进入上一次的路径
cd	进入 home 目录

9.1.4 复制文件

cp source_file dest_file	复制 source_file 为 dest_file
cp file1 file2 dir	复制 file1 和 file2 到文件夹 dir
cp -r source_dir dest_dir	保留属性复制，等同于 rsync -a source_dir/ dest_dir/
ln -s linked_file link	创建一个软连接

9.1.5 文件重命名

mv source_file dest_file	source_file 重命名为 dest_file

9.1.6 删除文件和文件夹

rm file1 file2	删除文件或者文件链接
rmdir dir	删除一个空文件夹
rm -rf dir	删除一个非空文件夹

9.2 文件列表

ls 最为常用，显示当前文件夹下的所有文件，不包含隐藏的文件

ls -l 显示更多的文件信息

ls -a 显示包含隐藏文件的所有文件

9.3 压缩解压

9.3.1 创建一个压缩包

tar zcvf archive.tar.gz dir/

9.3.2 解压一个 tar.gz 文件

tar zxvf archive.tar.gz

9.4 系统管理

ifconfig -a 显示所有可用网络接口

ping 192.168.1.1 测试网络和另外的主机连通性